

Benchmarking Isaac Sim Fidelity for Aerial-Robotics Control

Using a Geometric-Controller F450 Case Study

MEng Project Final Report

Zouhair Adam Hamaimou
Student Number: 1004891986

University of Toronto Institute for Aerospace Studies
Supervised by: Dr. Hugh Liu

April 27, 2026

Abstract

This report investigates the fidelity of NVIDIA Isaac Sim, coupled to the Pegasus Simulator, as a development environment for aerial-robotics research. Rather than treating the geometric tracking controller of Lee et al. [1] as the object of study, we use it as a fixed, theoretically well-understood instrument that produces a deterministic command stream from a known reference trajectory; tracking error can then be interpreted primarily as the result of modelling and numerical assumptions in the simulator and the F450 vehicle model rather than as a property of the controller itself. A detailed CAD model of the F450 was developed in OnShape and converted to a USD asset; a linear thrust model maps normalised throttle to rotor force; the geometric controller is implemented as a ROS 2 node with full feed-forward terms derived analytically from the flat outputs.

We benchmark five configurations of progressively increasing modelling complexity on a single ascending elliptic-helix reference: (i) an idealised single-rigid-body block as a baseline, (ii) the F450 articulation with thrust and moment applied at the centre of mass, (iii) the F450 with per-rotor forces and a body-applied yawing moment, and (iv)–(v) the F450 vehicle model integrated by a custom ROS 2 physics solver using fourth-order Runge–Kutta and forward Euler respectively, in place of Isaac Sim’s PhysX engine. The differences between adjacent configurations isolate the contribution of each modelling assumption to the observed tracking error. The report also documents in full the geometric controller’s stability theory and the analytical derivation of the desired body-frame angular velocity and acceleration, which together provide the theoretical basis for treating the controller as a known fixed instrument. The work delivers to the FSC Lab a reusable simulation pipeline and a quantitative characterisation of where Isaac Sim’s modelling fidelity stands relative to a custom-integrated reference.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Objectives	5
1.3	Contributions	5
1.4	Report Structure	5
2	Vehicle Modelling	6
2.1	CAD Model	6
2.2	Physical Properties	6
2.3	Linear Thrust-Curve Identification	7
2.4	Yaw-Moment Coefficient	8
3	Simulation Environment Setup	8
3.1	USD File Creation Procedure	9
3.1.1	Step 1 — Prepare the OnShape Assembly	9
3.1.2	Step 2 — Import into Isaac Sim	9
3.1.3	Step 3 — Save Flattened and Re-open	11
3.1.4	Step 4 — Restructure the Prim Hierarchy	11
3.1.5	Step 5 — Rename Rotors and Joints to the Pegasus Convention	12
3.1.6	Step 6 — Reparent Joints Under Their Rotors	14
3.1.7	Step 7 — Delete the Placeholder Environment	14
3.1.8	Step 8 — Assign Physics Properties	15
3.1.9	Step 9 — Rotor Inertia: Use a Small Non-Zero Value	16
3.1.10	Step 10 — Final Result	16
4	Geometric Tracking Control on SE(3)	17
4.1	Dynamics Model	17
4.2	Control Objective	18
4.3	Controller Structure and Inputs	18
4.4	Stability Properties	19
5	Implementation	20
5.1	Coordinate Convention and Sign of Gravity	20
5.2	Control Allocation for the F450	21
5.3	Feed-Forward Desired Angular Velocity and Acceleration	21
5.4	Pegasus Simulator Modifications	21
5.5	Custom ROS 2 Physics Solver	22
5.6	ROS 2 System Architecture	22
5.6.1	Variant A: Pegasus + Isaac Sim (E1, E2, E3)	22
5.6.2	Variant B: Custom Physics Solver (E4, E5)	23
5.7	Controller Parameter Tuning	24
6	Benchmark Results	25
6.1	Experimental Design	25
6.2	Reference Trajectory	26
6.3	Metrics	26
6.4	Results	26

6.4.1	3D Trajectory and Per-Axis Position Tracking	27
6.4.2	Position and Velocity Errors	29
6.4.3	Attitude Tracking and Errors	30
6.4.4	Control Outputs	32
6.5	Aggregate Metrics	34
6.6	Discussion	35
7	Conclusion and Future Work	36
7.1	Conclusion	36
7.2	Limitations	36
7.3	Future Work	37
A	Geometric Tracking Controller: Full Derivation and Stability Proofs	38
A.1	Dynamics Model	38
A.1.1	Configuration Manifold	38
A.1.2	Modelling Assumptions	38
A.1.3	Thrust and Moment Mapping	39
A.1.4	Equations of Motion	39
A.2	Tracking Error Definitions	39
A.2.1	Position and Velocity Errors	39
A.2.2	Attitude Error Function	39
A.2.3	Attitude Tracking Error Vector	40
A.2.4	Angular Velocity Tracking Error	40
A.3	Tracking Controller	40
A.4	Exponential Stability of the Attitude Dynamics	41
A.5	Exponential Stability of the Complete Dynamics	43
A.6	Almost Global Exponential Attractiveness	46
B	Feed-Forward Derivation of Ω_d and $\dot{\Omega}_d$	48
B.1	Definitions	48
B.2	Derivatives of A	48
B.3	Normalisation Lemma	48
B.4	Derivatives of b_{3d}	49
B.5	Derivatives of the Yaw Heading c	49
B.6	Derivatives of r and b_{2d}	49
B.7	Derivatives of b_{1d}	49
B.8	Derivation of Ω_d	50
B.9	Derivation of $\dot{\Omega}_d$	50
B.10	Summary of the Computation Chain	50
B.11	Assumptions and Degeneracy Handling	51
C	Supplementary Plots	52
C.1	Per-Axis Velocity Tracking	52
C.2	Per-Axis Velocity Error	53
C.3	Per-Axis Acceleration Tracking	54
C.4	Per-Axis Acceleration Error	55
C.5	Per-Axis Attitude-Error Components	56
C.6	Per-Axis Angular-Rate Tracking	57

C.7	Per-Axis Angular-Velocity-Error Components	58
C.8	Per-Axis Angular-Acceleration Tracking	59
C.9	Angular-Acceleration Errors	60

1 Introduction

1.1 Motivation

The Flight Systems and Control (FSC) Lab at the University of Toronto Institute for Aerospace Studies operates an F450 research quadrotor and develops new control and estimation algorithms in NVIDIA Isaac Sim, coupled to the Pegasus Simulator. Two natural questions follow: how faithful is Isaac Sim’s PhysX-backed rigid-body simulation to the physics that the algorithms will encounter on hardware, and how much of any observed discrepancy is attributable to the simulator itself versus to the user-supplied vehicle model? These cannot be answered by simply running a controller in simulation and reporting the tracking error; that error confounds controller imperfection, vehicle-model imperfection, and simulator imperfection.

This project performs a controlled experiment in which the controller and reference trajectory are held fixed and only the modelling assumptions vary. The geometric tracking controller on $SE(3)$ of Lee et al. [1] is used as the fixed instrument: it is exponentially stable on almost all of $SE(3)$ (Propositions 1–3), is defined directly on the manifold so that singularities and sign ambiguities never enter the picture, and is fully characterised analytically. With this instrument fixed, the tracking error can be interpreted primarily as the result of modelling and numerical choices.

1.2 Objectives

- (i) Develop a CAD-derived dynamic model of the F450 quadrotor.
- (ii) Implement the geometric tracking controller of [1] with full feed-forward terms as a fixed reference instrument.
- (iii) Integrate the controller into Isaac Sim through the Pegasus Simulator and a ROS 2 backend.
- (iv) Benchmark Isaac Sim’s modelling fidelity against an idealised baseline and a custom ROS 2 physics solver on a single non-symmetric reference trajectory.

1.3 Contributions

- A CAD-derived dynamic model and a measured linear thrust curve for the F450.
- A self-contained treatment of the geometric tracking controller’s stability analysis, including a corrected matrix bound in the complete-dynamics proof (Remark 3).
- An analytical derivation of Ω_d and $\dot{\Omega}_d$ from the flat outputs up to snap and yaw acceleration.
- A control-allocation matrix consistent with the F450 rotor ordering and the Isaac Sim body-frame convention.
- Extensions to the Pegasus Simulator source code (jerk publishing, angular-velocity frame correction, ROS 2 throttle backend).
- A five-configuration benchmark that isolates the contribution of articulation modelling, force-distribution modelling, and integrator choice to the tracking error in Isaac Sim.

1.4 Report Structure

Section 2 describes the CAD model and physical-parameter identification. Section 3 documents the Isaac Sim USD asset creation. Section 4 summarises the geometric controller; the full derivation,

stability proofs, and feed-forward derivation are in Appendices A and B. Section 5 details the implementation, ROS 2 integration, and the custom physics solver. Section 6 presents the benchmark. Section 7 concludes.

2 Vehicle Modelling

2.1 CAD Model

A full CAD assembly of the F450 quadrotor was built in OnShape, including the frame with landing gear, a Pixhawk flight controller, the LiPo battery, the four brushless motors, the propellers, and the wiring box. All components were modelled to scale using measurements of the physical vehicle.

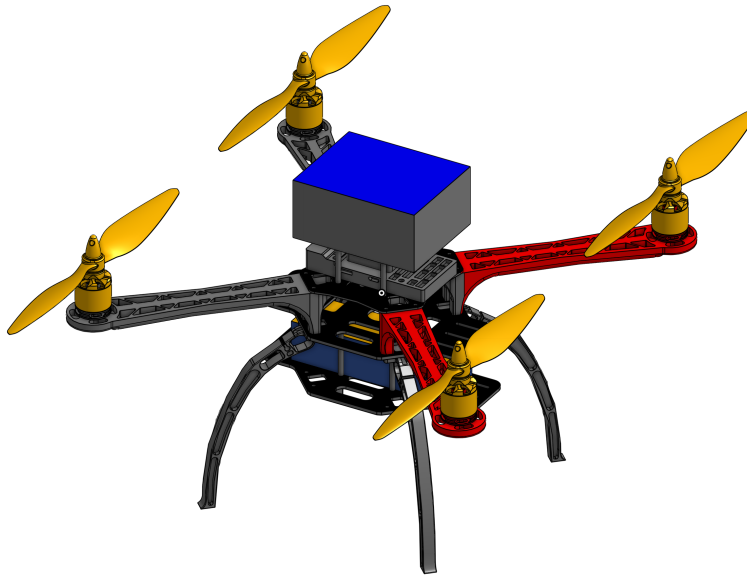


Figure 1: OnShape CAD assembly of the F450 quadrotor used to estimate mass, centre-of-gravity location, and the diagonal inertia components.

2.2 Physical Properties

The mass, centre-of-gravity (CoG), and diagonal inertia components of the complete assembly were estimated directly from OnShape’s mass-properties tool. Because the quadrotor is nearly symmetric about the body axes, off-diagonal inertia terms are small and are assumed to be zero for the purposes of the controller.

The mass breakdown of the individual components, taken from the OnShape parts list, is given in Table 1. The summed total agrees with the measured total mass of the physical vehicle to within the accuracy of the kitchen-scale measurement.

Table 1: Mass breakdown of the F450 components from the OnShape CAD assembly.

Component	Mass (kg)
F450 frame with landing gear	0.728
Pixhawk flight controller	0.104
LiPo battery	0.420
Brushless motor (per unit, $\times 4$)	0.065
Propeller (per unit, $\times 4$)	0.012
Wiring box	0.232
Miscellaneous (screws, fasteners)	0.014
Total	1.806

The centre of gravity of the complete assembly coincides with the body-frame origin in the x - y plane (i.e. it lies on the geometric centre of the rotor square when viewed from above). For the controller, the body-frame origin is defined to coincide with the CoG in all three axes, so no CoG offset appears in the equations of motion of Section 4. The vertical CoG height above the ground in the nominal resting pose is 0.165 m, which is used in simulation as the initial z -position of the vehicle when it takes off from the ground. The overall physical parameters used by the controller are summarised in Table 2.

Table 2: Physical parameters of the F450 used by the controller.

Symbol	Quantity	Value
m	Total mass (kg)	1.806
J_{xx}	Roll inertia about the CoG (kg m^2)	0.016
J_{yy}	Pitch inertia about the CoG (kg m^2)	0.017
J_{zz}	Yaw inertia about the CoG (kg m^2)	0.024
d	Arm length from CoG to rotor (m)	0.215
z_0	Ground-to-CoG height in resting pose (m)	0.165

2.3 Linear Thrust-Curve Identification

The Pegasus motor abstraction represents each rotor’s commanded input as a normalised throttle $u_i \in [0, 1]$. To close the loop between the controller (which produces forces and moments) and the simulated rotors, a mapping from u_i to thrust f_i is required. Bench tests of the F450 motor-propeller combination were performed and a linear least-squares fit of the form

$$f_i = k_f u_i, \quad k_f = 9.7237 \text{ N} \quad (1)$$

was found to capture the measured data well over the operating range used in simulation (see Figure 2). The small nonlinearities at very low throttle are acceptable because the quadrotor typically operates around hover throttle.

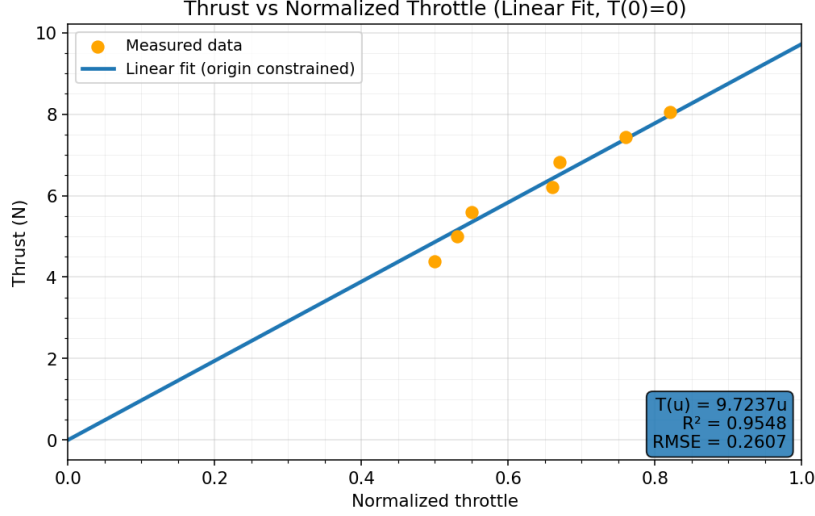


Figure 2: Measured thrust versus normalised throttle for the F450 motor-propeller assembly, with a linear least-squares fit of slope $k_f = 9.7237$ N.

2.4 Yaw-Moment Coefficient

The reactive torque produced by each rotor about the body z -axis is modelled as a linear function of the normalised throttle:

$$\tau_i = \sigma_i k_m u_i, \quad k_m = 8 \times 10^{-2} \text{ N m}, \quad (2)$$

where $i \in \{0, 1, 2, 3\}$ indexes the rotors and σ_i is a fixed ± 1 sign that depends on the rotation direction of rotor i . For the F450 as configured in this project, $(\sigma_0, \sigma_1, \sigma_2, \sigma_3) = (-1, -1, +1, +1)$: rotors 0 and 1 spin in one direction, and rotors 2 and 3 spin in the opposite direction. This pattern, together with the X-arm geometry, produces a balanced reactive torque in hover and allows yaw control by differential throttling of the two groups. The resulting yaw-moment row of the allocation matrix is given in Section 5.2.

Unlike the thrust coefficient k_f , the value of k_m used here was not identified from bench tests. It was chosen as an engineering guess of the correct order of magnitude, sufficient to obtain plausible yaw authority in simulation, and was then held fixed throughout tuning and testing. A direct measurement of k_m on a torque bench is left as future work (Section 7.3); this is an important limitation of the present model, because an incorrect k_m scales the yaw-moment column of Λ^{-1} and therefore biases the yaw channel of the controller on the real hardware.

3 Simulation Environment Setup

The simulation stack consists of NVIDIA Isaac Sim as the physics and rendering engine, the Pegasus Simulator as the quadrotor abstraction layer, and ROS 2 as the messaging middleware between the simulator and the controller. Importing the custom F450 asset into Isaac Sim requires producing a USD file with the correct articulation structure, rigid-body masses, and physics properties. The procedure developed during the project is documented below for reproducibility, with screenshots of each step that benefits from visual reference.

3.1 USD File Creation Procedure

The procedure below produces a USD asset of the F450 quadrotor that is compatible with the Pegasus Simulator. It is documented step-by-step because several of the gotchas discovered during this project are not apparent from the Isaac Sim documentation alone.

3.1.1 Step 1 — Prepare the OnShape Assembly

Step 1.1. Set the workspace units to metres using **Top left** → **Workspace units** → **Length** → **Metres**.

Step 1.2. Build a single assembly named **vehicle** in which the frame is a single part called **body**, each propeller is a separate part, and a revolute joint is defined between the **body** and each rotor. The auto-generated rotor names produced by OnShape (**rotor<1>**, **rotor<2>**, ...) are renamed later in Isaac Sim to match the Pegasus naming convention (Section 3.1.5).

Step 1.3. Align the assembly origin with the centre of mass of the drone.

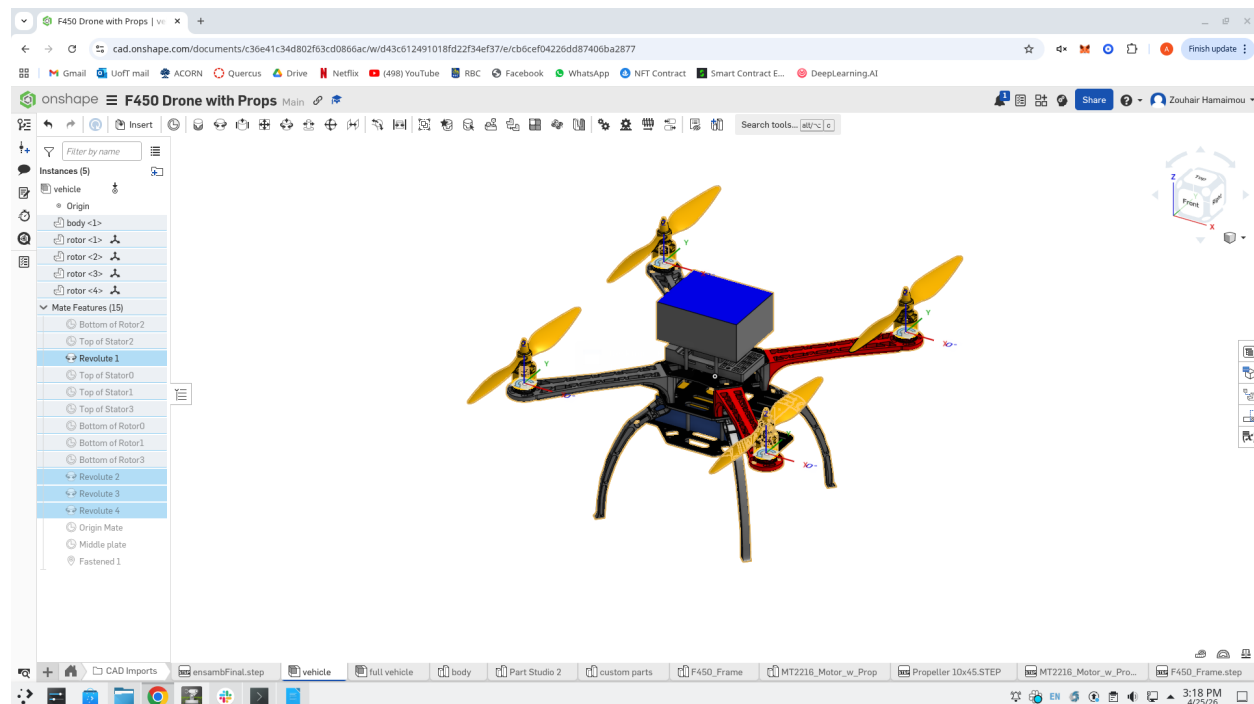


Figure 3: OnShape assembly view of the F450, with the **body**, the four rotor parts, and the four revolute joints highlighted.

3.1.2 Step 2 — Import into Isaac Sim

Step 2.1. Authenticate the OnShape account in a browser.

Step 2.2. In Isaac Sim, open **File** → **Import from OnShape**, select the project containing the F450 assembly, and choose the **vehicle** assembly.

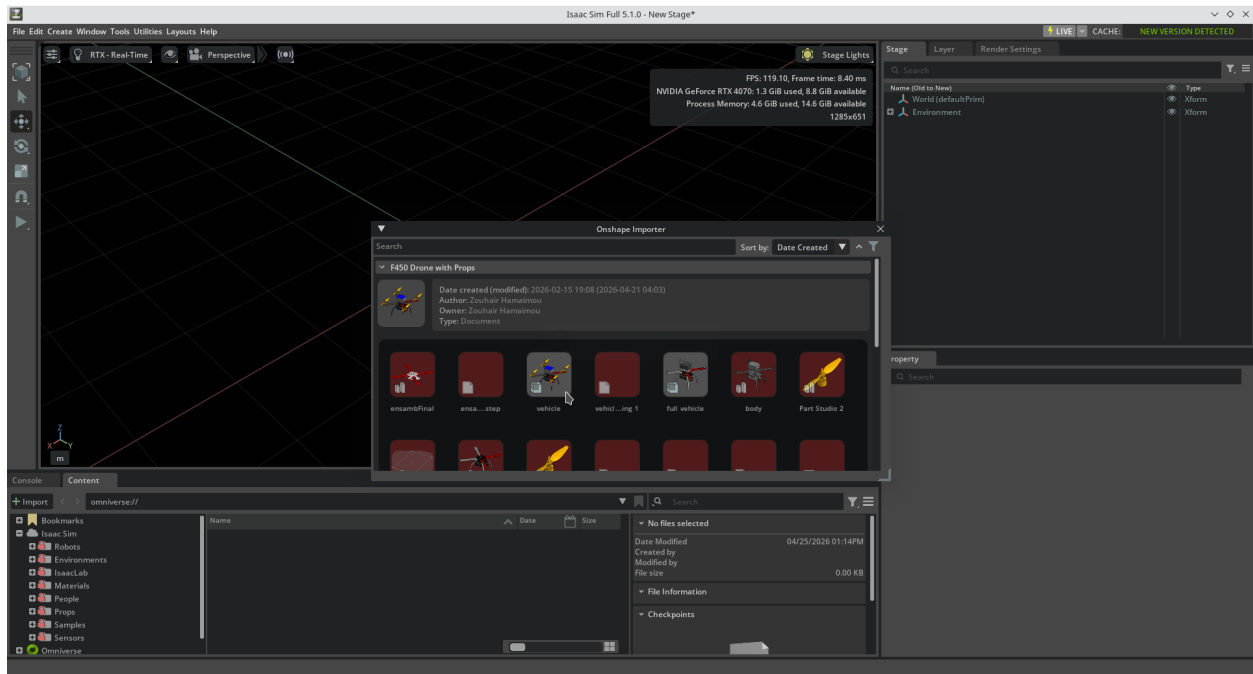


Figure 4: Isaac Sim OnShape Importer with the `vehicle` assembly selected.

Step 2.3. The importer prompts for material, mass, and tessellation settings on each part. Close this window without setting masses; the masses are assigned manually later (Step 8) once the prim hierarchy is in its final form.

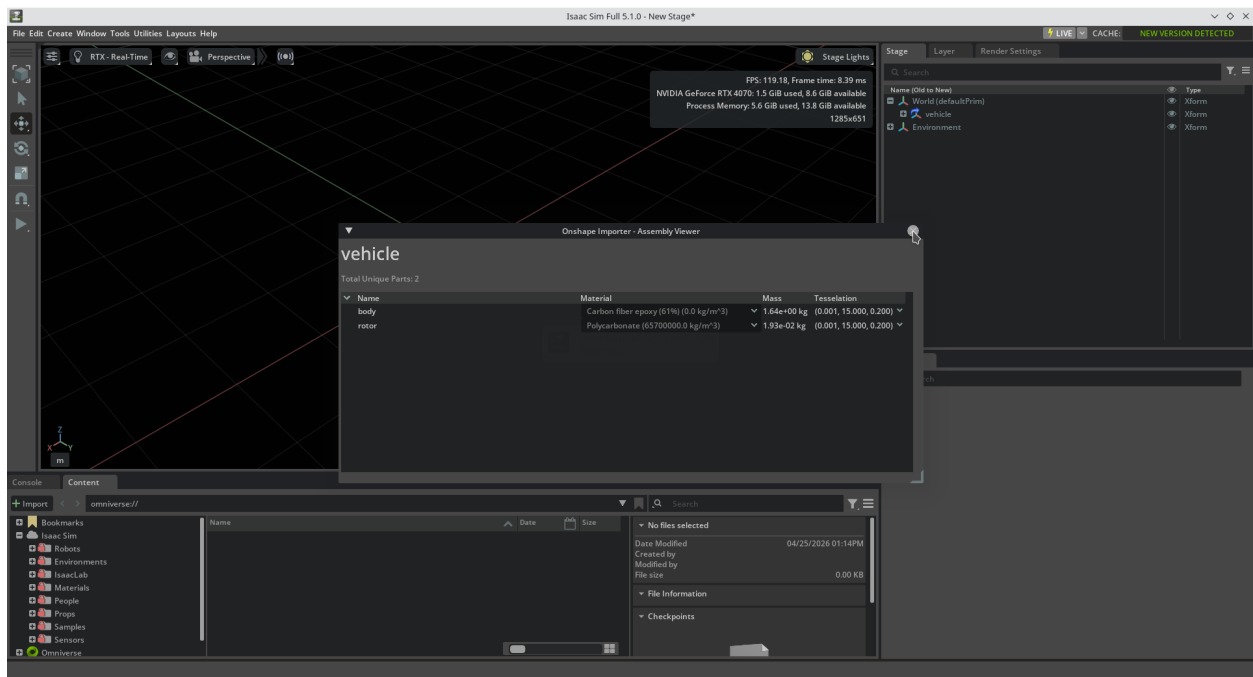
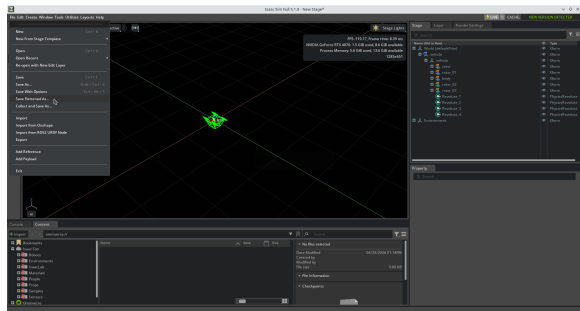


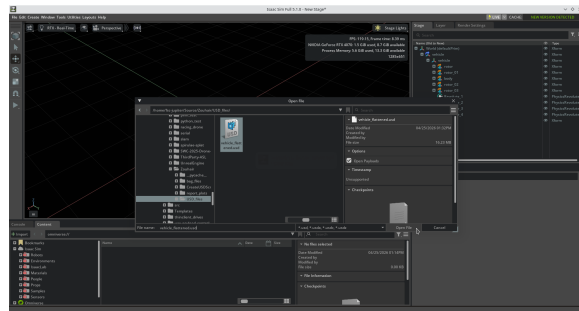
Figure 5: Importer parts list. Dismiss this dialog without setting masses; masses are assigned manually in Step 8.

3.1.3 Step 3 — Save Flattened and Re-open

The OnShape importer produces a USD scene with a deeply nested prim hierarchy that is not directly usable by Pegasus. Saving a *flattened* copy collapses the hierarchy into a form that can be edited.



(a) File → Save Flattened As...



(b) Open the flattened USD file.

Figure 6: Saving a flattened copy and re-opening it.

3.1.4 Step 4 — Restructure the Prim Hierarchy

After flattening, the hierarchy appears as in Figure 7. The Pegasus articulation root expects the rotors to live directly underneath the vehicle prim, with each revolute joint underneath its corresponding rotor. The reorganisation below produces that structure.

Step 4.1. Inspect the initial hierarchy.

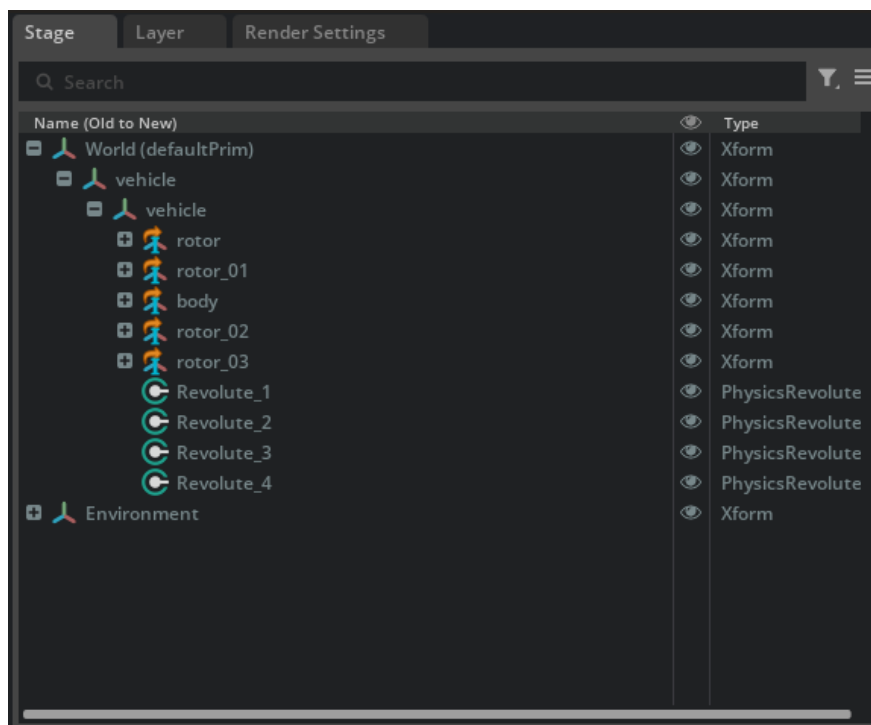


Figure 7: Initial Stage hierarchy after flattening.

Step 4.2. Move every prim under `/World/vehicle/vehicle/` up one level into `/World/vehicle/`.

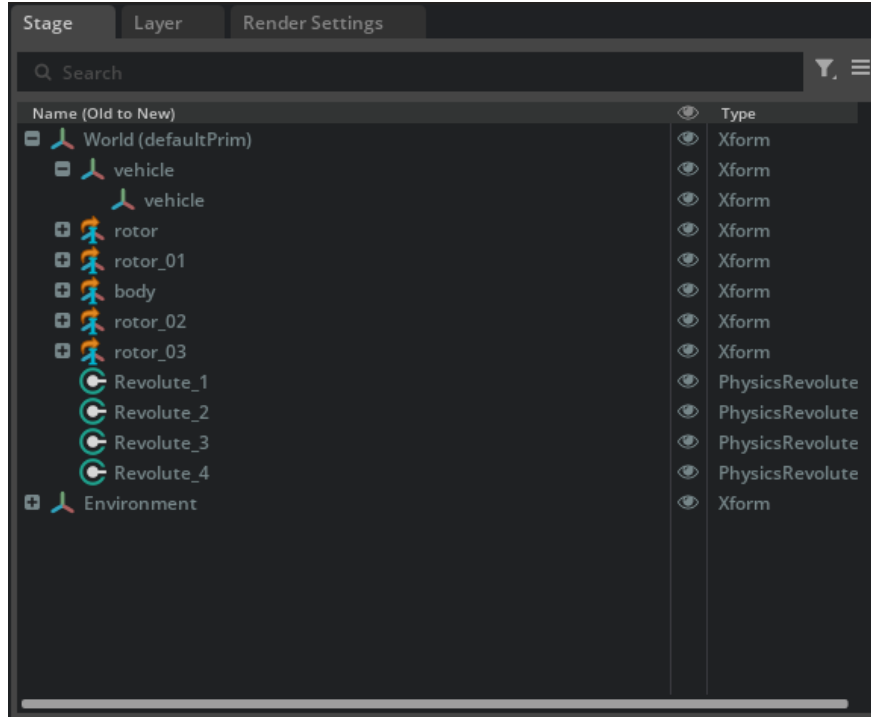


Figure 8: Stage hierarchy after promoting every child of `/World/vehicle/vehicle/` into `/World/vehicle/`. The empty inner `vehicle` prim is removed in the next step.

Step 4.3. Delete the now-empty inner `vehicle` prim, and rename `World` to `vehicle`.

3.1.5 Step 5 — Rename Rotors and Joints to the Pegasus Convention

The Pegasus simulator hard-codes a per-rotor rotation-direction sign vector (default $[-1, -1, +1, +1]$, i.e. rotors 0 and 1 rotate one way and rotors 2 and 3 rotate the other way) and uses the rotor-prim names `rotor0`, ..., `rotor3` as ROS 2 topic suffixes (`control/rotor0/ref`, etc.). The rotor names *do not* therefore have to correspond to particular physical positions; what matters is that whichever rotor is named `rotor0` actually spins in the direction indicated by $\sigma_0 = -1$, and that the controller’s allocation matrix (Section 5.2) is written in a way consistent with the chosen labelling.

In this project, the Pegasus default σ vector is preserved, and the rotor names are chosen so that the rotation-direction pattern matches the F450’s physical rotor layout: `rotor0` and `rotor1` are the two rotors that spin in the same direction, and likewise `rotor2` and `rotor3`. Each renamed joint inherits the index of the rotor it actuates (`joint0` actuates `rotor0`, and so on).

Step 5.1. For each rotor, select it, choose **Rename**, and enter the chosen Pegasus name (`rotor0`, ..., `rotor3`).

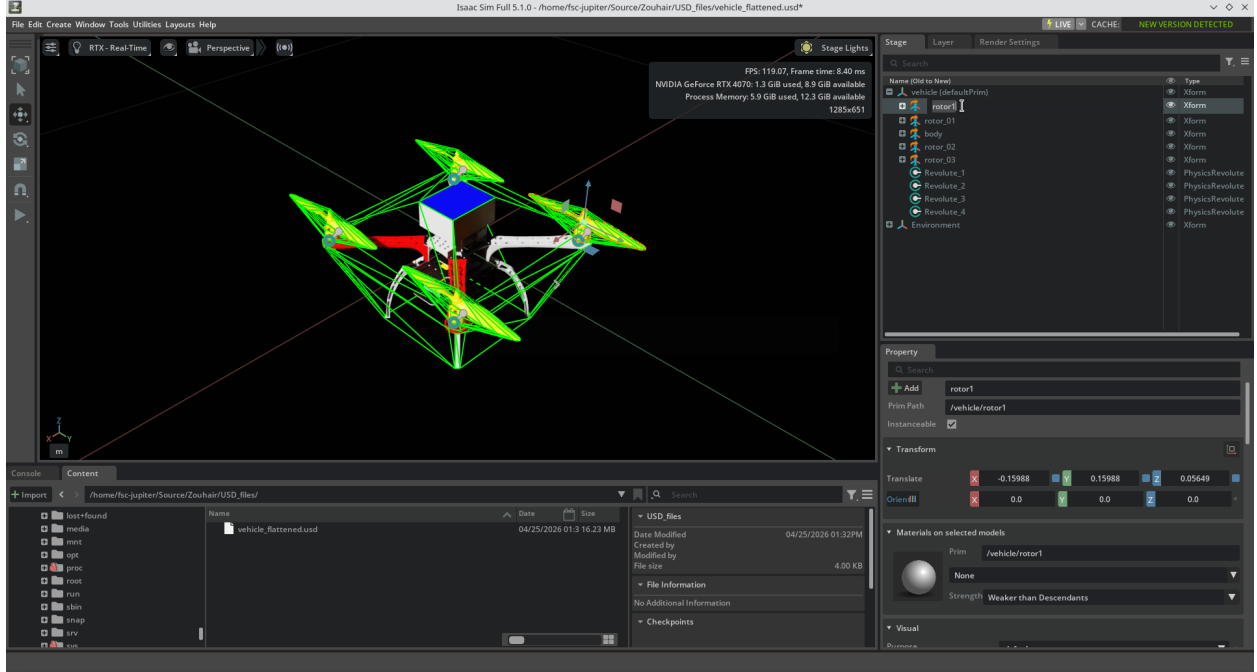


Figure 9: Renaming a rotor to match the Pegasus naming convention (**rotor0**–**rotor3**).

Step 5.2. For each revolute joint, open its property panel, read the Body 0/Body 1 paths to determine which rotor it actuates, and rename it to **jointN** where N is the index of that rotor.

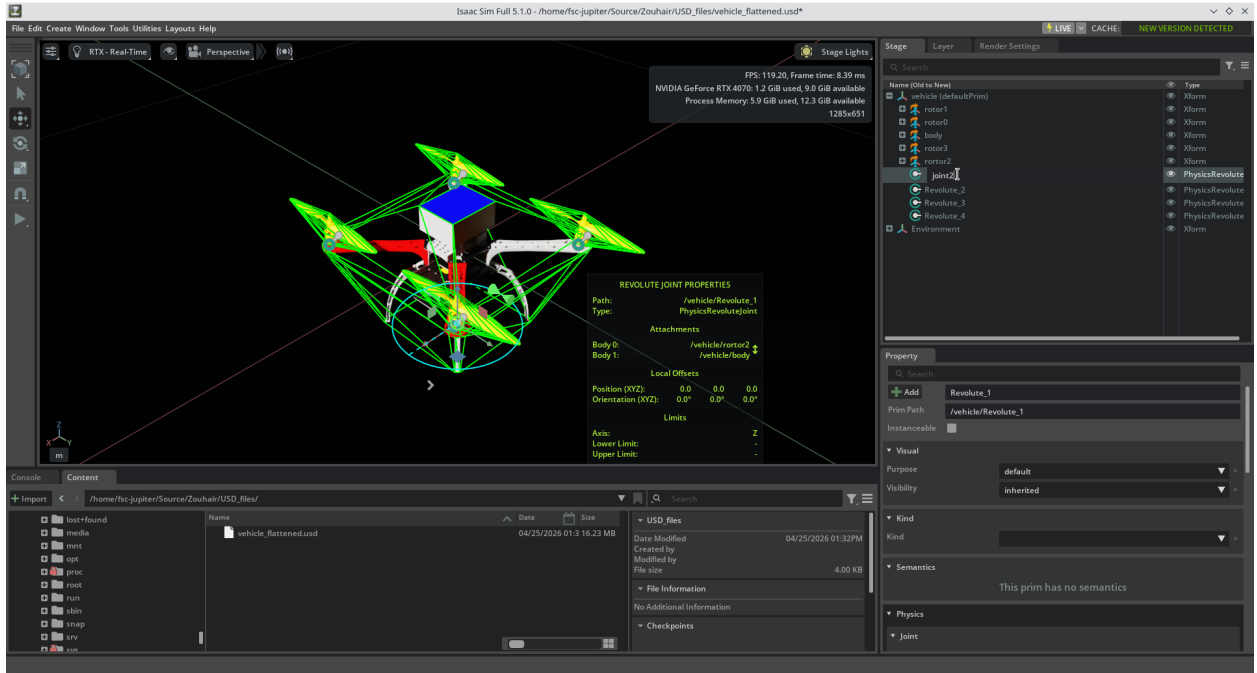


Figure 10: Inspecting a revolute joint’s Body 0 and Body 1 attachment paths to identify the rotor it actuates before renaming it to **jointN**.

3.1.6 Step 6 — Reparent Joints Under Their Rotors

The Pegasus articulation root expects each revolute joint to live underneath the rotor it actuates. Moving the joints requires disabling Isaac Sim’s instanceable flag on the affected prims.

Step 6.1. For each of `body`, `rotor0`, ..., `rotor3`, open the property panel and uncheck **Instanceable**. This is required to allow the joints to be reparented in the next step.

Step 6.2. Move each `jointN` underneath its corresponding `rotorN` prim.

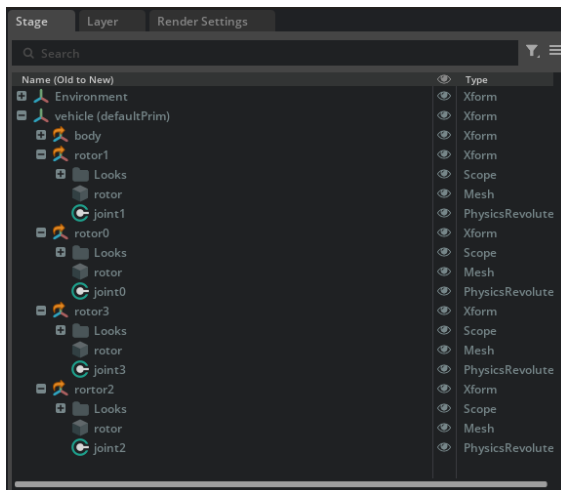


Figure 11: Stage hierarchy after Step 6: each rotor owns its actuating joint.

3.1.7 Step 7 — Delete the Placeholder Environment

The flattened scene contains a default **Environment** prim (a ground plane and a dome light). It is unrelated to the vehicle articulation and must be removed before publishing the asset.

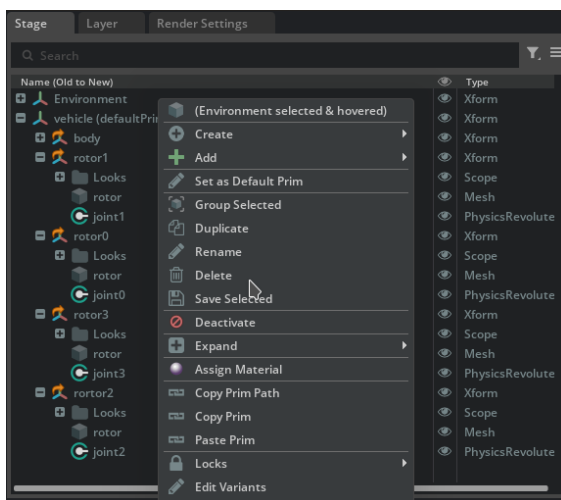


Figure 12: Deleting the auto-generated **Environment** prim.

3.1.8 Step 8 — Assign Physics Properties

Step 8.1. Right-click on the top-level `vehicle` prim and choose `Add` → `Physics` → `Articulation Root`.

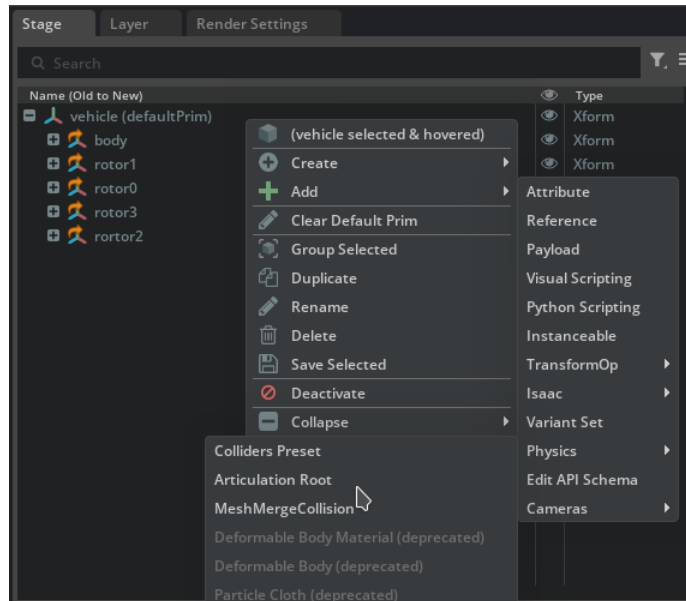


Figure 13: Adding an Articulation Root to the top-level `vehicle` prim.

Step 8.2. Right-click on each child Xform under `vehicle` (`body`, `rotor0`, `rotor1`, `rotor2`, `rotor3`) and choose `Add` → `Physics` → `Mass`.

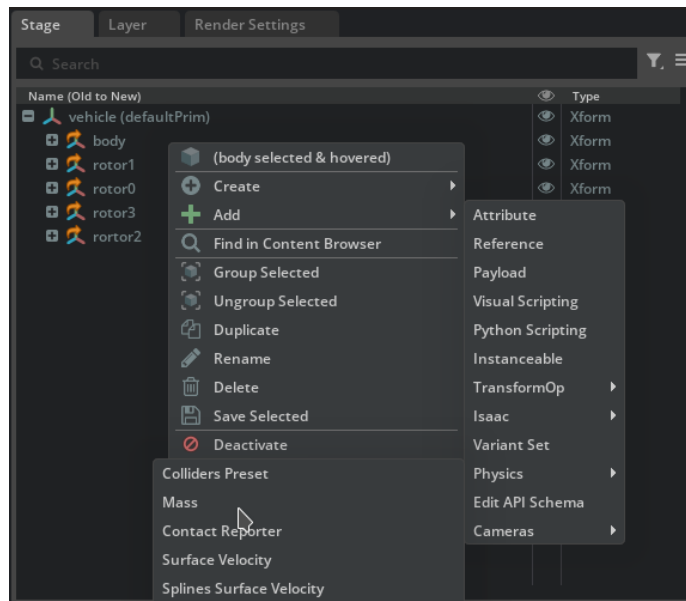


Figure 14: Adding a Mass component to each child Xform under `vehicle`.

Step 8.3. For each Xform that has a Mass component, open the property panel, find **Raw USD Properties**, and enter the centre of mass, the diagonal inertia, and the mass. The mass and inertia terms **must be strictly positive**.



(a) Before editing.



(b) After editing.

Figure 15: Raw USD mass properties for the body Xform, before and after entering the CAD-derived values from Table 2.

3.1.9 Step 9 — Rotor Inertia: Use a Small Non-Zero Value

For the rotors, only the diagonal inertia about the rotor's spin axis (z) is mechanically relevant; the other two diagonal entries are essentially zero. However, leaving the off-axis entries at exactly zero causes the Pegasus simulator to crash. The fix is to enter a small non-zero value (on the order of 10^{-7} kg m²) for the x - and y -diagonal inertia entries. When such a small value is typed into the field and the cursor leaves the field, the Raw USD Properties panel rounds the displayed value back to 0, but the underlying USD attribute is updated correctly.

3.1.10 Step 10 — Final Result

The final asset, opened from the Pegasus Simulator, is shown in Figure 16. It is now ready to be used in any of the five benchmark configurations of Section 6.

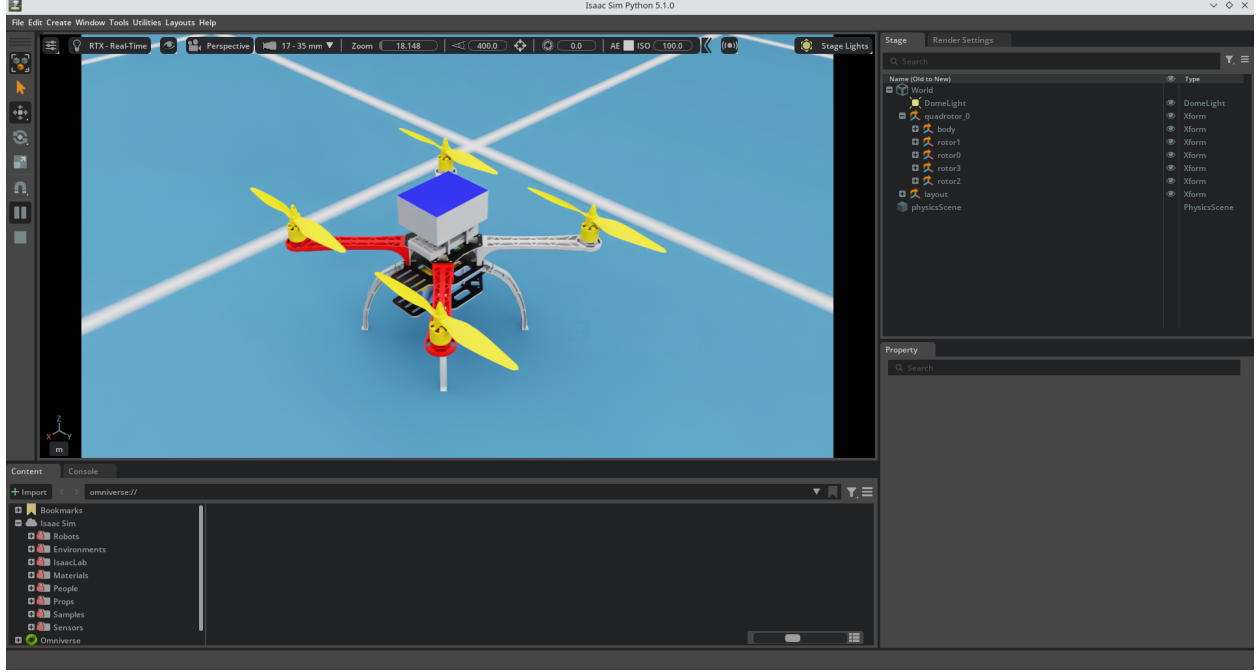


Figure 16: The completed F450 USD asset opened from the Pegasus Simulator.

4 Geometric Tracking Control on $SE(3)$

This section gives the high-level structure of the geometric tracking controller of [1] as it is used in this project. The full dynamics derivation, the tracking-error definitions on $SO(3)$, and the Lyapunov-based proofs of exponential stability and almost-global exponential attractiveness are given in Appendix A; the reader who wants to verify any specific step in the controller design or stability analysis is referred there.

4.1 Dynamics Model

Let $x \in \mathbb{R}^3$ be the position of the centre of mass in an inertial frame, $v \in \mathbb{R}^3$ its velocity, $R \in SO(3)$ the body-to-inertial rotation matrix, and $\Omega \in \mathbb{R}^3$ the body-frame angular velocity. With total thrust $f \in \mathbb{R}$ and body-frame moment $M \in \mathbb{R}^3$, the equations of motion are

$$\dot{x} = v, \quad (3)$$

$$m\dot{v} = mge_3 - fRe_3, \quad (4)$$

$$\dot{R} = R\hat{\Omega}, \quad (5)$$

$$J\dot{\Omega} + \Omega \times J\Omega = M, \quad (6)$$

where $e_3 = [0, 0, 1]^\top$ and $\hat{\cdot}: \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ is the hat map. Equations (3)–(6) have a *cascade structure*: the rotational dynamics are independent of the translational state, and the translational dynamics depend on R only through fRe_3 . The full reference-frame conventions, modelling assumptions, and the rotor-to- (f, M) allocation matrix are reproduced in Appendix A.1.

4.2 Control Objective

Given smooth desired trajectories $x_d(t) \in \mathbb{R}^3$ and a desired heading direction $\vec{b}_{1d}(t) \in S^2$, the controller is designed so that

$$x(t) \rightarrow x_d(t) \quad \text{and} \quad \text{Proj}[\vec{b}_1(t)] \rightarrow \text{Proj}[\vec{b}_{1d}(t)] \quad \text{as } t \rightarrow \infty,$$

where $\text{Proj}[\cdot]$ denotes the normalised projection onto the plane orthogonal to $\vec{b}_{3d}(t)$. The reason a projection appears is that the desired thrust direction $\vec{b}_{3d}$ is dictated by the translational stage of the controller (Section 4.3), and there is no guarantee that $\vec{b}_{1d}$ is orthogonal to $\vec{b}_{3d}$ when the user supplies it; only the component of $\vec{b}_{1d}$ in the plane normal to $\vec{b}_{3d}$ is achievable. The projection definition is illustrated in Figure 17.

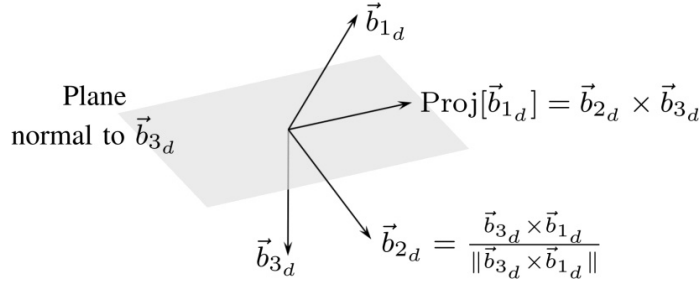


Figure 17: Definition of the desired heading direction. The desired first body-axis $\vec{b}_{1d}$ is projected onto the plane normal to $\vec{b}_{3d}$. The intermediate axis $\vec{b}_{2d} = (\vec{b}_{3d} \times \vec{b}_{1d}) / \|\vec{b}_{3d} \times \vec{b}_{1d}\|$ is computed first, from which the projected heading $\text{Proj}[\vec{b}_{1d}] = \vec{b}_{2d} \times \vec{b}_{3d}$ is recovered. Figure adapted from [1].

4.3 Controller Structure and Inputs

The controller has a two-stage cascade structure. Given smooth references $x_d(t)$ and a yaw heading $\vec{b}_{1d}(t) \in S^2$ and positive gains k_x, k_v, k_R, k_Ω :

Translational stage. Compute the virtual force vector

$$A = -k_x e_x - k_v e_v - m g e_3 + m \ddot{x}_d, \quad e_x = x - x_d, \quad e_v = v - v_d, \quad (7)$$

the desired thrust direction, and the total thrust

$$\vec{b}_{3d} = -\frac{A}{\|A\|}, \quad f = -A \cdot R e_3. \quad (8)$$

Attitude stage. Construct the desired rotation matrix from $\vec{b}_{3d}$ and $\vec{b}_{1d}$,

$$\vec{b}_{2d} = \frac{\vec{b}_{3d} \times \vec{b}_{1d}}{\|\vec{b}_{3d} \times \vec{b}_{1d}\|}, \quad R_d = [\vec{b}_{2d} \times \vec{b}_{3d}, \vec{b}_{2d}, \vec{b}_{3d}], \quad (9)$$

define the attitude and angular-velocity tracking errors on $\text{SO}(3)$,

$$e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee, \quad e_\Omega = \Omega - R^\top R_d \dot{\Omega}_d, \quad (10)$$

and apply the moment law

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J \Omega - J(\hat{\Omega} R^\top R_d \dot{\Omega}_d - R^\top R_d \dot{\Omega}_d). \quad (11)$$

The errors e_R and e_Ω are derived geometrically as the gradient of an attitude-error function $\Psi(R, R_d) = \frac{1}{2}\text{tr}(I - R_d^\top R)$ on $\text{SO}(3)$ and as the body-frame angular velocity of the relative rotation $R_d^\top R$, respectively; the full derivations are in Appendix A.2. The moment law (11) is a nonlinear PD-with-feed-forward controller on $\text{SO}(3)$: when the feed-forward terms exactly cancel the gyroscopic and reference-attitude coupling, the closed-loop attitude error dynamics reduce to $J\dot{e}_\Omega = -k_R e_R - k_\Omega e_\Omega$.

The overall block structure of the controller is summarised in Figure 18.

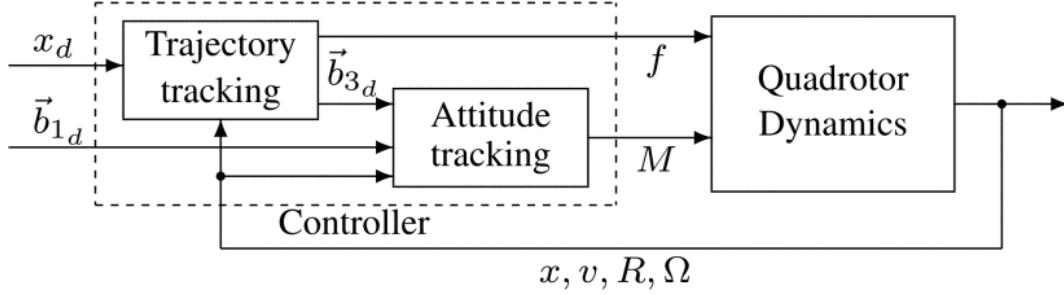


Figure 18: Controller structure. The trajectory tracking block computes the desired third body-axis $\vec{b}_{3d}$ and total thrust f from the position command x_d . The attitude tracking block computes the moment M to drive $R \rightarrow R_d$, constructed from $\vec{b}_{3d}$ and the heading command $\vec{b}_{1d}$. The state (x, v, R, Ω) is fed back to both blocks. Figure adapted from [1].

4.4 Stability Properties

The closed-loop tracking error converges exponentially to zero from almost every initial condition in $\text{SE}(3)$. Three propositions, proved in detail in Appendices A.4–A.6, are used:

- **Proposition 1 (attitude exponential stability).** For $\Psi(R(0), R_d(0)) < 2$ (initial attitude error below 180°) and a sufficiently small initial angular-velocity error, the attitude error $(e_R, e_\Omega) \rightarrow 0$ exponentially. The region of attraction *almost covers* $\text{SO}(3)$.
- **Proposition 2 (complete-dynamics exponential stability).** For $\Psi(R(0), R_d(0)) \leq \psi_1 < 1$ (initial attitude error below 90°) and a similar bound on the initial angular-velocity error, the full error $(e_x, e_v, e_R, e_\Omega) \rightarrow 0$ exponentially under suitable gain conditions.
- **Proposition 3 (almost-global exponential attractiveness).** For $1 \leq \Psi(R(0), R_d(0)) < 2$, the attitude first enters the region of attraction of Proposition 2, so exponential decay still holds eventually, with a multiplicative constant that depends on the initial condition.

The practical consequence is that, for almost every initial state on $\text{SE}(3)$ (excluding the measure-zero set of exact 180° attitude flips), the closed-loop tracking error decays exponentially. This makes the controller well-suited to act as a fixed reference instrument in the simulator-fidelity benchmark of Section 6: any error observed in simulation can be interpreted primarily as a property of the simulator and the vehicle model rather than of the controller.

5 Implementation

5.1 Coordinate Convention and Sign of Gravity

The theoretical treatment in Section 4 follows the convention of [1], in which the inertial e_3 axis points *downward*. Under this convention, gravity acts in the $+e_3$ direction, and the body-fixed $\vec{b}_3$ axis points downward (opposite to the total thrust), so the translational equation of motion reads

$$m\dot{v} = mge_3 - fRe_3. \quad (\text{Lee})$$

Isaac Sim and the Pegasus Simulator use the opposite convention: the inertial e_3 axis points *upward* (standard ENU/ Z -up world frame), and the body-fixed $\vec{b}_3$ axis is defined to align with the positive thrust direction. Under this convention, gravity acts in the $-e_3$ direction, and the translational equation of motion is

$$m\dot{v} = -mge_3 + fRe_3. \quad (\text{Impl})$$

Consequently, the virtual force vector A defined in the controller has its gravity term with the *opposite sign* from Lee’s convention:

$$A_{\text{Lee}} = -k_x e_x - k_v e_v - mge_3 + m\ddot{x}_d, \quad (12)$$

$$A_{\text{Impl}} = -k_x e_x - k_v e_v + mge_3 + m\ddot{x}_d. \quad (13)$$

The desired thrust axis and thrust magnitude are also defined with opposite signs:

$$\vec{b}_{3d}^{\text{Lee}} = -\frac{A_{\text{Lee}}}{\|A_{\text{Lee}}\|}, \quad f_{\text{Lee}} = -A_{\text{Lee}} \cdot Re_3,$$

$$\vec{b}_{3d}^{\text{Impl}} = +\frac{A_{\text{Impl}}}{\|A_{\text{Impl}}\|}, \quad f_{\text{Impl}} = +A_{\text{Impl}} \cdot Re_3.$$

These are the expressions used in the Python implementation (`geometric_control.py`) and in the feed-forward derivation of Section 5.3.

Why the stability proofs still apply. The change between (Lee) and (Impl) is purely a choice of world-frame z -axis orientation; it is equivalent to the coordinate transformation $e_3 \mapsto -e_3$ (with a corresponding re-orientation of R). Since the closed-loop error dynamics of Proposition 1 reduce to $J\dot{e}_\Omega = -k_R e_R - k_\Omega e_\Omega$ independently of the translational dynamics, and the translational error dynamics in Proposition 2 reduce (in Step 2) to $m\dot{e}_v = -k_x e_x - k_v e_v - X$ where all the frame-dependent constants have cancelled out, the coordinate flip does not enter any of the Lyapunov inequalities. All sign changes are self-consistent: the $-mge_3$ term in (Impl), the $+mge_3$ term in (13), the $+$ sign in $\vec{b}_{3d}^{\text{Impl}}$, and the $+$ sign in f_{Impl} together leave the closed-loop error dynamics unchanged. Propositions 1–3 therefore apply verbatim to the implemented controller, with the understanding that e_3 and R in the proofs are replaced by their Z -up-convention counterparts.

For the remainder of Section 5, all equations follow the implementation (Isaac Sim / Z -up) convention.

5.2 Control Allocation for the F450

The F450 rotors (indexed $i \in \{0, 1, 2, 3\}$) are arranged in an X-configuration. Using the linear thrust and moment models from Section 2.3 and Section 2.4, the allocation from normalised throttles $u = [u_0, u_1, u_2, u_3]^\top$ to (f, M_1, M_2, M_3) is

$$\begin{bmatrix} f \\ M_1 \\ M_2 \\ M_3 \end{bmatrix} = \underbrace{\begin{bmatrix} k_f & k_f & k_f & k_f \\ \frac{k_f d}{\sqrt{2}} & \frac{k_f d}{\sqrt{2}} & \frac{k_f d}{\sqrt{2}} & -\frac{k_f d}{\sqrt{2}} \\ -\frac{k_f d}{\sqrt{2}} & \frac{k_f d}{\sqrt{2}} & -\frac{k_f d}{\sqrt{2}} & \frac{k_f d}{\sqrt{2}} \\ -k_m & -k_m & k_m & k_m \end{bmatrix}}_{\Lambda} \begin{bmatrix} u_0 \\ u_1 \\ u_2 \\ u_3 \end{bmatrix}. \quad (14)$$

The yaw-moment row $[-k_m, -k_m, +k_m, +k_m]$ reflects the rotation-direction pattern $(\sigma_0, \sigma_1, \sigma_2, \sigma_3) = (-1, -1, +1, +1)$ established in Section 2.4. The controller outputs (f, M) via (8), (11), which are inverted by Λ^{-1} (computed once at initialisation) to obtain the per-rotor normalised throttles. The pair (Λ, Λ^{-1}) is verified numerically against $\Lambda\Lambda^{-1} = I$ at start-up.

5.3 Feed-Forward Desired Angular Velocity and Acceleration

The moment law (11) requires the desired body-frame angular velocity Ω_d and acceleration $\dot{\Omega}_d$ as feed-forward terms. Rather than numerically differentiating the desired attitude—which amplifies noise—these are derived analytically from the flat outputs $x_d(t)$ and $\psi_d(t)$ and their derivatives up to snap and yaw-acceleration, following the differential-flatness approach of Mellinger and Kumar [5].

The full derivation is given in Appendix B. The high-level computation chain is:

- (i) Compute $A, \dot{A}, \ddot{A}$ from the controller errors and the flat-output derivatives (a_d, j_d, s_d) .
- (ii) Apply a normalisation lemma to obtain b_{3d} and its first two time derivatives from A .
- (iii) Differentiate the yaw heading vector $c = [\cos \psi_d, \sin \psi_d, 0]^\top$ twice to get $\dot{c}, \ddot{c}$.
- (iv) Form $r = b_{3d} \times c$ and obtain $b_{2d}, \dot{b}_{2d}, \ddot{b}_{2d}$ via the same normalisation lemma.
- (v) Form $b_{1d} = b_{2d} \times b_{3d}$ and obtain $\dot{b}_{1d}, \ddot{b}_{1d}$ by the product rule.
- (vi) Recover Ω_d and $\dot{\Omega}_d$ as

$$\Omega_d = \begin{bmatrix} b_{3d}^\top \dot{b}_{2d} \\ b_{1d}^\top \dot{b}_{3d} \\ b_{2d}^\top \dot{b}_{1d} \end{bmatrix}, \quad \dot{\Omega}_d = \begin{bmatrix} \dot{b}_{3d}^\top \dot{b}_{2d} + b_{3d}^\top \ddot{b}_{2d} \\ \dot{b}_{1d}^\top \dot{b}_{3d} + b_{1d}^\top \ddot{b}_{3d} \\ \dot{b}_{2d}^\top \dot{b}_{1d} + b_{2d}^\top \ddot{b}_{1d} \end{bmatrix}. \quad (15)$$

The derivation assumes $\|A\| \neq 0$ and $\|b_{3d} \times c\| \neq 0$. The first fails only when the controller commands exactly zero net force; the second occurs when the commanded body- z axis becomes horizontal, i.e. at a 90° pitch or roll attitude. Both are guarded in the implementation by small numerical thresholds with safe fall-backs.

5.4 Pegasus Simulator Modifications

The Pegasus Simulator required three source modifications to support the controller:

1. **Jerk output.** The simulator now publishes body-frame linear jerk $j = \dot{a}$, required by the $\ddot{A}$ computation in Section 5.3.

2. **Angular velocity reference frame.** The original simulator published angular velocity expressed in the world frame while mislabelling it as body-frame. This was corrected so that the published Ω matches the definition used in (5).
3. **ROS 2 control backend.** A backend was added that subscribes to per-rotor normalised throttle commands on the ROS 2 topic `control/rotor{i}/ref` and applies them directly to the simulated rotors.

5.5 Custom ROS 2 Physics Solver

For the integrator-comparison experiments in Section 6, a separate ROS 2 node was developed that replaces Isaac Sim’s PhysX engine entirely with a hand-written rigid-body integrator. The node subscribes to the same per-rotor throttle topics as the Pegasus backend, evaluates the F450 dynamics (3)–(6) using the same mass and inertia parameters as the Isaac Sim asset (Table 2), advances the state by one time-step using either fourth-order Runge–Kutta (RK4) or forward Euler integration at the same 250 Hz rate as the controller, and publishes the new state on the standard `state/*` topics. In these experiments Isaac Sim is not running at all; the custom node is the sole source of physics, providing a controlled comparison between the simulator’s commercial-grade integrator and a transparent, fully-specified textbook integrator on identical inputs.

5.6 ROS 2 System Architecture

The closed-loop pipeline runs in two slightly different topologies depending on whether the experiment uses Isaac Sim as the physics engine (E1, E2, E3) or the custom ROS 2 solver (E4, E5). Both topologies share the same controller and trajectory-publisher nodes; they differ only in what supplies and consumes the rotor commands. The two variants are shown in Figures 19 and 20. Arrows are labelled in standard control-theory notation: $\mathbf{x}$ for the measured vehicle state, $\mathbf{x}_d$ for the reference trajectory, $\mathbf{u}$ for the per-rotor commands, and f, M for the lumped total thrust and body-frame moment that the controller also publishes (used by Pegasus only in the lumped-input experiments E1 and E2). Each label corresponds to one or more ROS 2 topics, listed in Table 3. All ROS 2 nodes run at 250 Hz, matched to the Isaac Sim physics step in variant A and to the custom-solver step in variant B.

5.6.1 Variant A: Pegasus + Isaac Sim (E1, E2, E3)

In this variant, Isaac Sim is the physics engine and the Pegasus Simulator backend bridges between Isaac Sim’s internal state and the ROS 2 topic graph. Pegasus reads the vehicle state from Isaac Sim through the Dynamic Control Python interface (not ROS 2) and republishes it on the state-bundle topics. Conversely, the rotor commands received on ROS 2 are converted by Pegasus into the appropriate forces and moments and applied to Isaac Sim through the same Python interface.

The exact translation that the Pegasus backend performs depends on which experiment is running:

- **E1, E2 (lumped (f, M)).** Pegasus subscribes to the controller’s f and M topics and applies them at the body’s CoG.
- **E3 (distributed).** Pegasus subscribes to the four rotor-throttle topics $\mathbf{u}$, converts each one to a per-rotor force using the thrust curve of (1), applies each force to the corresponding rotor link, and applies only the reactive yawing moment M_z to the body.

The geometric controller publishes both $\mathbf{u}$ and (f, M) in all three experiments; Pegasus only reads whichever is appropriate for the current configuration.

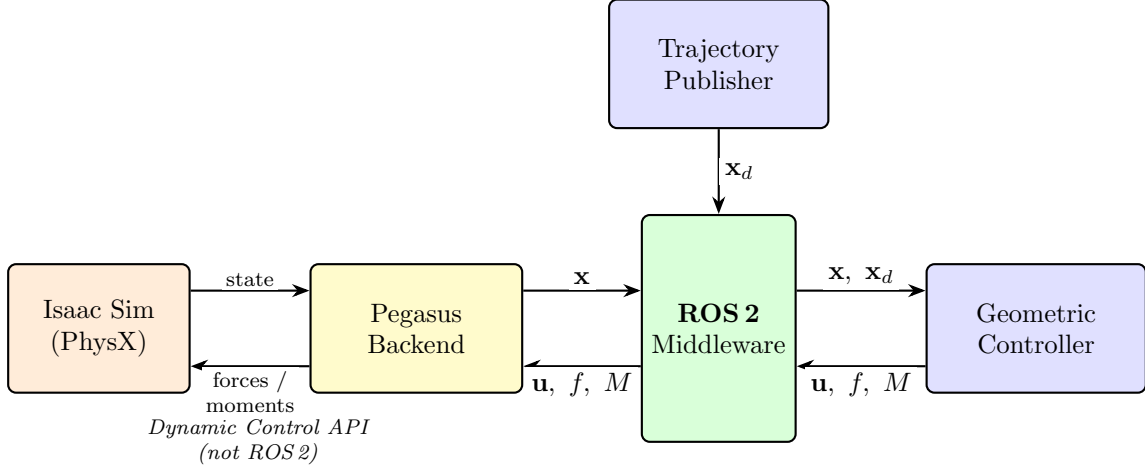


Figure 19: Variant A architecture (E1, E2, E3).

5.6.2 Variant B: Custom Physics Solver (E4, E5)

In this variant, Isaac Sim is not running. A custom ROS 2 node integrates the rigid-body dynamics (3)–(6) (RK4 in E4, forward Euler in E5) directly inside ROS 2 and publishes the integrated state on the same state-bundle topics that Pegasus would have used in variant A. Because the custom node receives per-rotor commands and produces a state, it functions as a one-block replacement for the entire (Pegasus, Isaac Sim) pair. The geometric controller and trajectory publisher are unchanged.

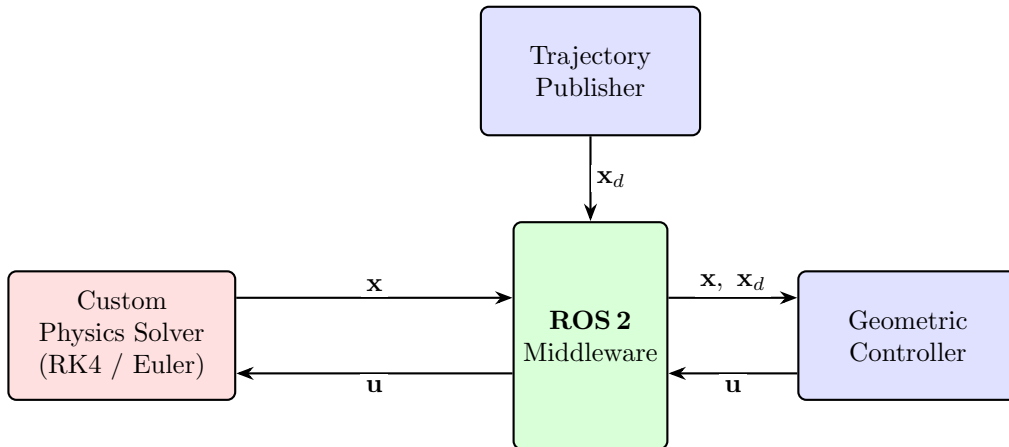


Figure 20: Variant B architecture (E4, E5).

Table 3: ROS 2 topics carried by each arrow-symbol of Figures 19 and 20. All topics are namespaced under /uav_0/, with the namespace path given once per group beneath the symbol.

Symbol	Topic	Description
x	namespace: state/	
	pose	World-frame pose (position + orientation) of the body.
	twist	Body-frame linear and angular velocity.
	twist_inertial	World-frame linear and angular velocity.
	accel	Body-frame linear and angular acceleration.
	jerk	Body-frame linear jerk $\dot{a}$, used by the $\ddot{A}$ feed-forward.
x_d	namespace: trajectory/desired/	
	position	Reference position $x_d(t)$.
	velocity	Reference velocity $\dot{x}_d(t)$.
	acceleration	Reference acceleration $\ddot{x}_d(t) = a_d$.
	jerk	Reference jerk $\dddot{x}_d(t) = \dot{a}_d$, for $\dot{A}$.
	snap	Reference snap $\ddddot{x}_d(t) = \dot{s}_d$, for $\ddot{A}$.
	yaw	Reference yaw angle $\psi_d(t)$.
	yaw_rate	Reference yaw rate $\dot{\psi}_d(t)$.
	yaw_acceleration	Reference yaw acceleration $\ddot{\psi}_d(t)$.
u	namespace: control/	
	rotor0/ref	Normalised throttle command for rotor 0.
	rotor1/ref	Normalised throttle command for rotor 1.
	rotor2/ref	Normalised throttle command for rotor 2.
	rotor3/ref	Normalised throttle command for rotor 3.
f, M	namespace: control/output/	
	f	Total thrust commanded by the controller (used by Pegasus in E1, E2).
	M	Body-frame moment commanded by the controller (used by Pegasus in E1, E2).

5.7 Controller Parameter Tuning

The gains used for all simulation results in Section 6 are listed in Table 4. The outer-loop position gains were tuned first with an approximate damping ratio of $\zeta \approx 0.7$, then the inner-loop attitude gains were chosen to give a roughly 10 : 1 bandwidth separation. Separate yaw gains were used so that the yaw response could be tuned independently from roll and pitch. The yaw gains were chosen slightly smaller to avoid overly aggressive yaw transients.

Table 4: Controller gains used for all trajectory tests.

Gain group	Parameters	Values
Position control	(k_x, k_v)	(1.0, 1.7)
Roll/pitch attitude control	(k_R, k_Ω)	(1.5, 0.4)
Yaw attitude control	$(k_{R,\psi}, k_{\Omega,\psi})$	(1.0, 0.35)
Control loop	Publish rate	250 Hz

6 Benchmark Results

6.1 Experimental Design

The benchmark holds the geometric controller (Section 4) and the reference trajectory fixed across all experiments and varies only the configuration of the simulated body and physics engine. Five configurations are tested, listed in order of increasing modelling complexity in Table 5.

Table 5: Benchmark configurations used in the fidelity study.

ID	Body model	Forces / moments applied	Physics integrator
E1	Single rigid body	(f, M) applied at the CoG	Isaac Sim PhysX
E2	F450 articulation	(f, M) applied at body	Isaac Sim PhysX
E3	F450 articulation	Per-rotor forces on rotor links + M_z on body	Isaac Sim PhysX
E4	F450 rigid body (parameters only)	Per-rotor forces + M_z on body	Custom ROS 2 RK4
E5	F450 rigid body (parameters only)	Per-rotor forces + M_z on body	Custom forward Euler

The geometric controller, controller gains (Table 4), and reference trajectory are identical across all five configurations.

The purpose of each configuration, and the modelling effect that is isolated by comparing it against the previous one in the table, is summarised below.

- **E1 (Ideal block)** is the purest test of the geometric controller in Isaac Sim. The body is a single rigid block with the F450’s mass and inertia, and the controller applies its computed thrust and moment directly at the CoG. There is no rotor articulation, no force-distribution model, no thrust-curve mapping. Any tracking error here is attributable to the controller, the trajectory, and Isaac Sim’s integrator alone. This is the baseline.
- **E2 (F450 lumped)** replaces the block with the full F450 articulation (frame, four rotor links, four revolute joints) but still applies the resultant (f, M) at the body’s CoG. The difference E2–E1 isolates the effect of articulating the body into a multi-link rigid system, even when the same net force and moment are applied.
- **E3 (F450 distributed)** applies the four computed per-rotor forces directly to the four rotor links via the allocation matrix of Section 5.2, and applies only the reactive yawing moment M_z to the body. This is the most physically faithful Isaac Sim configuration considered in this work. The difference E3–E2 isolates the effect of distributing the forces across the rotor links rather than lumping them at the CoG.
- **E4 (Custom RK4)** replaces Isaac Sim’s PhysX integrator with the custom ROS 2 solver of Section 5.5, integrating (3)–(6) with fourth-order Runge–Kutta at 250 Hz. The vehicle parameters are identical to E3. The difference E4–E3 isolates the effect of the integrator: PhysX (commercial, contact-aware, optimised) versus a textbook RK4 with full insight into every step.
- **E5 (Custom Euler)** is identical to E4 except that the integrator is forward Euler. The difference E5–E4 isolates the effect of integrator order on a 250 Hz controller: a high-order method versus the lowest-order method that still produces stable trajectories.

6.2 Reference Trajectory

The benchmark uses an ascending elliptic helix as the single reference trajectory:

$$x_d(t) = a \sin(\omega t), \quad (16)$$

$$y_d(t) = b(1 - \cos(\omega t)), \quad (17)$$

$$z_d(t) = z_0 + v_z t, \quad (18)$$

$$\psi_d(t) = \omega t, \quad (19)$$

with $a = 1$ m, $b = 2$ m, $\omega = 0.5$ rad/s, $v_z = 0.1$ m/s, and $z_0 = 0.165$ m corresponding to the resting CoG height of the vehicle (Section 2). The trajectory exercises all four controllable degrees of freedom of the quadrotor: x and y trace an ellipse of unequal semi-axes (so the lateral acceleration is not symmetric in the body frame), z ramps linearly upward, and the yaw ψ rotates continuously. It runs for three full revolutions (duration ≈ 37.7 s), long enough that any unstable mode in any of the five configurations would have time to grow visibly.

6.3 Metrics

The natural error variables for the benchmark are those that already appear in the controller and its stability analysis (Section 4 and Appendix A): the position error $e_x = x - x_d$ and velocity error $e_v = v - v_d$, and the attitude and angular-velocity errors on SO(3),

$$e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee, \quad e_\Omega = \Omega - R^\top R_d \Omega_d. \quad (20)$$

The aggregate metrics reported per experiment are summarised in Table 6.

Table 6: Aggregate tracking metrics computed for each experiment after the initial ramp-up transient.

Metric	Symbol	Definition
RMS position error	RMS_{e_x}	$\sqrt{\frac{1}{N} \sum_k \ e_x(t_k)\ ^2}$
Max position error	$\max \ e_x\ $	$\max_k \ e_x(t_k)\ $
RMS velocity error	RMS_{e_v}	$\sqrt{\frac{1}{N} \sum_k \ e_v(t_k)\ ^2}$
RMS attitude error	RMS_{e_R}	$\sqrt{\frac{1}{N} \sum_k \ e_R(t_k)\ ^2}$
Max attitude error	$\max \ e_R\ $	$\max_k \ e_R(t_k)\ $
RMS angular-velocity error	RMS_{e_Ω}	$\sqrt{\frac{1}{N} \sum_k \ e_\Omega(t_k)\ ^2}$

6.4 Results

All plots that follow are overlays in which the five experiments E1–E5 and the reference signal share the same axes. The plots in this subsection constitute the main-report set; the per-axis velocity, acceleration, angular-rate, and angular-acceleration breakdowns, together with per-axis attitude and angular-velocity error components, are deferred to Appendix C. The numerical summary is in Table 7; this subsection describes what is visible in the plots, and Section 6.6 draws the conclusions.

6.4.1 3D Trajectory and Per-Axis Position Tracking

Figure 21 shows the reference helix and the trajectories produced by the five benchmark configurations. The paths are visually indistinguishable except very near the start, where E_1 is briefly visible underneath the others as it trails the rising helix; the differences between experiments are best seen in the per-axis tracking plots and in the error norms.

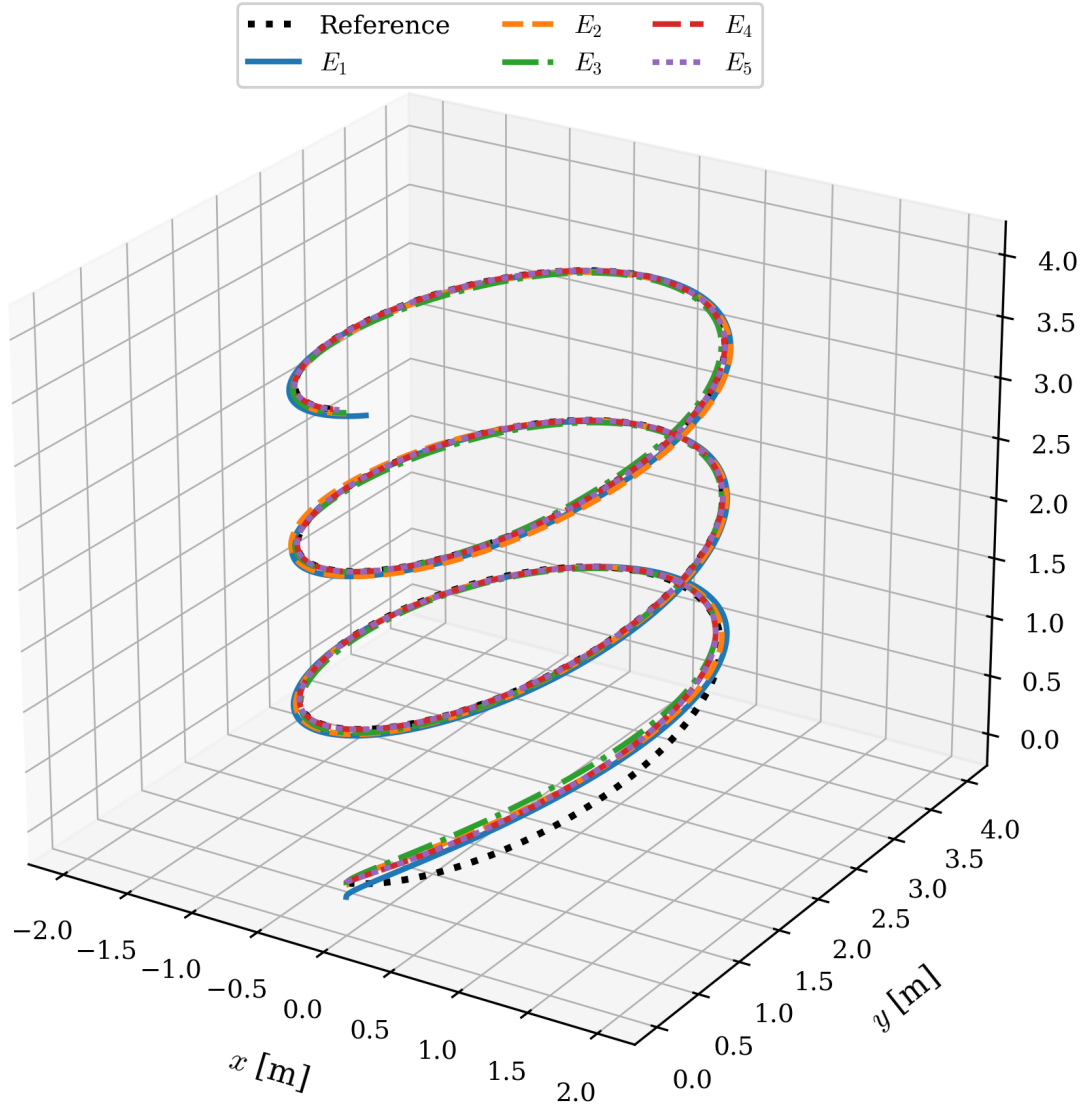
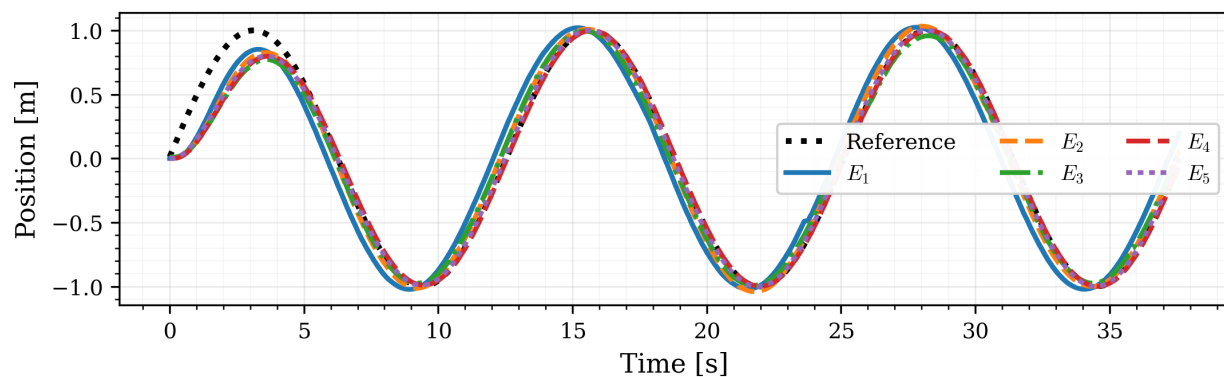
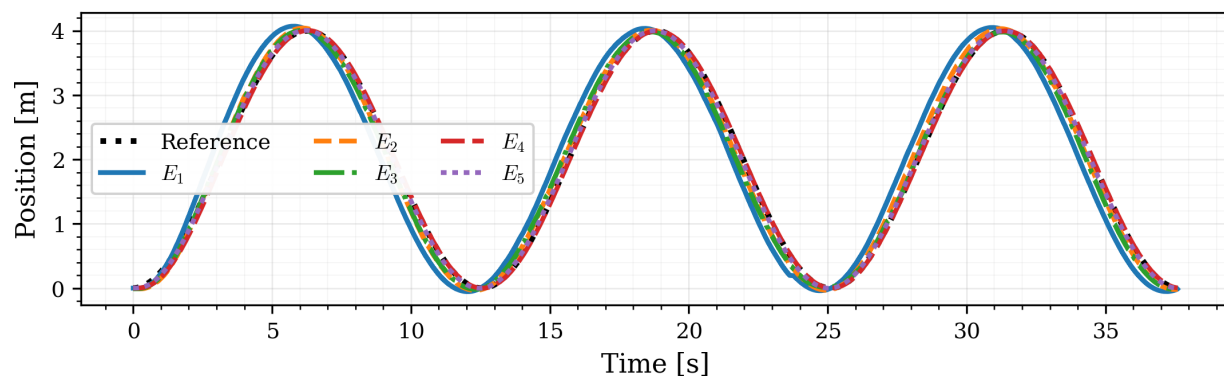


Figure 21: 3D path traced by the vehicle in each of the five configurations, overlaid against the reference helix.

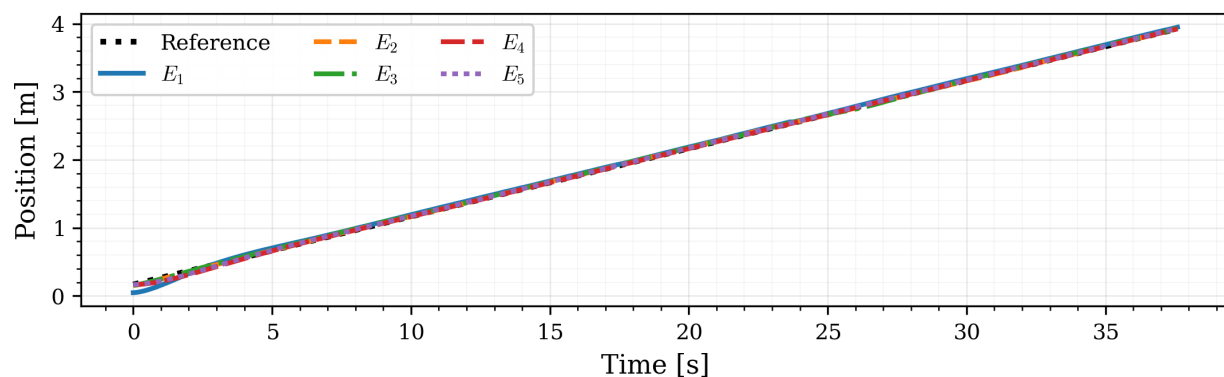
In the per-axis tracking plots in Figure 22, E1 (blue) lags the reference visibly in both x and y ; E2 and E3 also lag but by a smaller amount; E4 and E5 lie underneath the reference within line width. The z -axis is tracked tightly by all five experiments, since it is excited only by the constant climb rate $v_z = 0.1$ m/s.



(a) $x(t)$.



(b) $y(t)$.

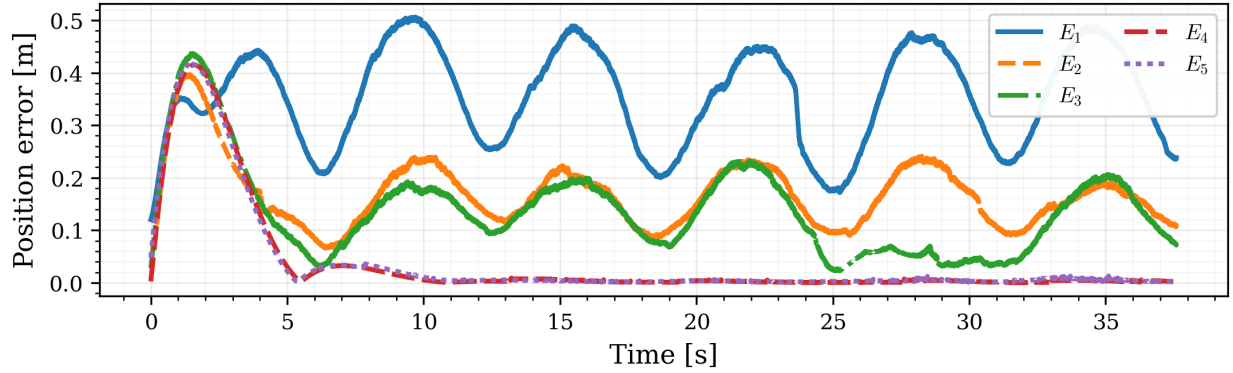


(c) $z(t)$.

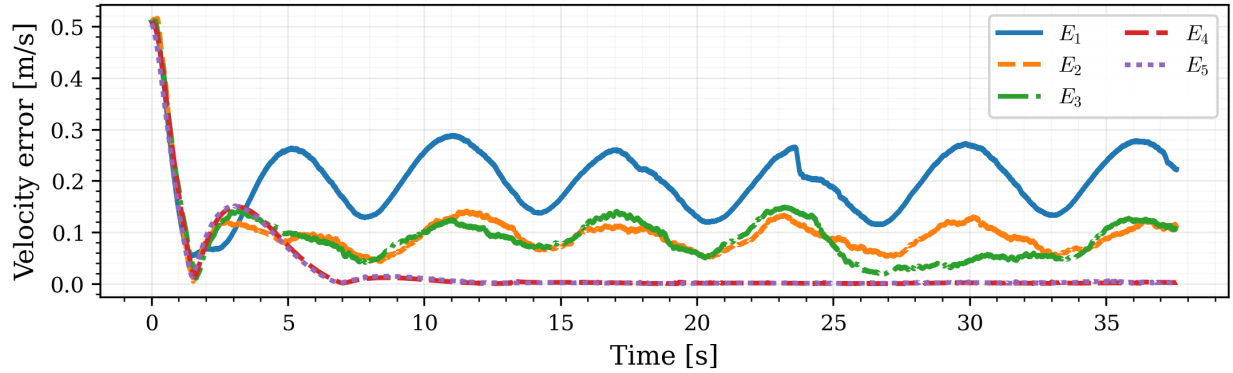
Figure 22: Per-axis position tracking against the reference (dashed).

6.4.2 Position and Velocity Errors

The translational error norms in Figure 23 provide the clearest separation between the five benchmark configurations. After the initial takeoff transient, the responses separate into three groups. E1 has the largest sustained error, with $\|e_x\|$ remaining roughly in the 0.2–0.5 m range. E2 and E3 form an intermediate band, generally oscillating between about 0.10 and 0.25 m. E4 and E5 have the smallest errors and remain close to zero after the transient, indicating that the custom rigid-body solvers reproduce the controller’s nominal model more closely than the Isaac Sim configurations. The velocity-error norm follows the same grouping, with the amplitude of the oscillations scaled by 0.5, the characteristic frequency of the reference trajectory.



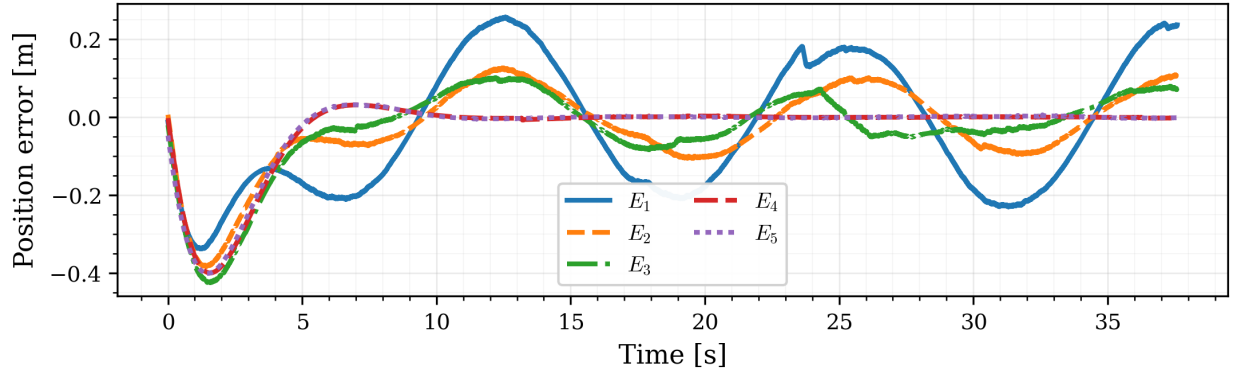
(a) Position-error norm $\|e_x(t)\|$.



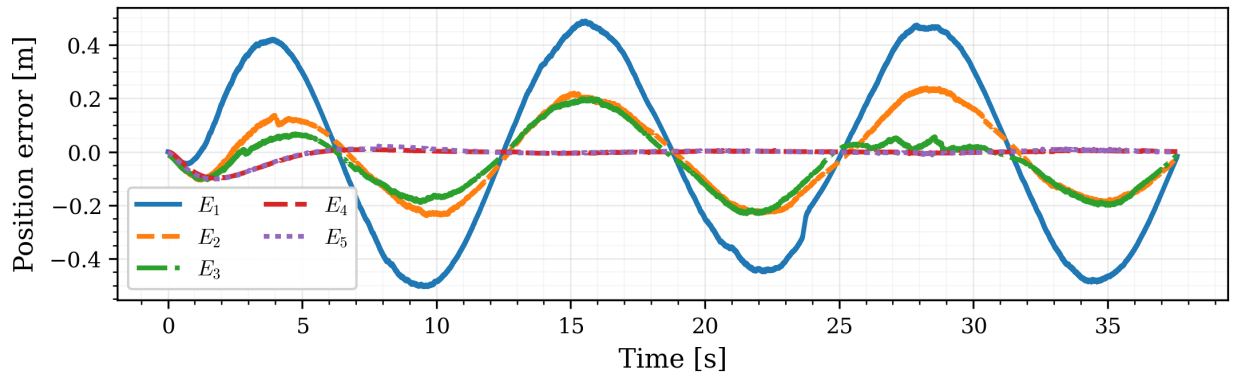
(b) Velocity-error norm $\|e_v(t)\|$.

Figure 23: Translational tracking errors. The early peak corresponds to the initial transient from the resting pose; afterwards each curve traces the steady-state tracking error of its experiment.

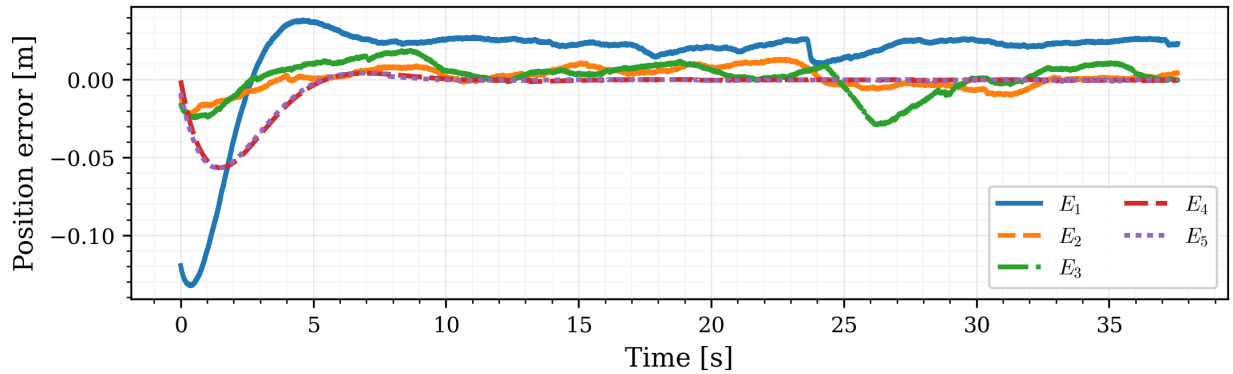
The component-wise position errors in Figure 24 show that the sustained translational error is dominated by the lateral axes. For E1, the x - and y -components exhibit the largest oscillations, with e_x reaching roughly -0.4 to 0.25 m and e_y reaching roughly ± 0.5 m. E2 and E3 show the same oscillatory structure but with smaller amplitude, while E4 and E5 remain close to zero after the initial transient. In contrast, the vertical error e_z is comparatively small for all configurations after takeoff, staying on the order of a few centimetres. This indicates that the main differences between experiments arise from lateral tracking behaviour rather than from the vertical thrust/gravity balance.



(a) $e_x(t)$.



(b) $e_y(t)$.



(c) $e_z(t)$.

Figure 24: Position-error components along the inertial axes. Here e_x , e_y , and e_z denote the scalar components of the position-error vector.

6.4.3 Attitude Tracking and Errors

Only yaw is plotted as an attitude-tracking signal in Figure 25, because the yaw reference is provided directly by the trajectory generator. Roll and pitch are not independent reference inputs: they are implied by the desired rotation matrix R_d computed inside the geometric controller. Therefore, the roll/pitch response is better assessed through the geometric attitude error e_R , rather than by comparing Euler-angle traces against a single shared roll/pitch reference.

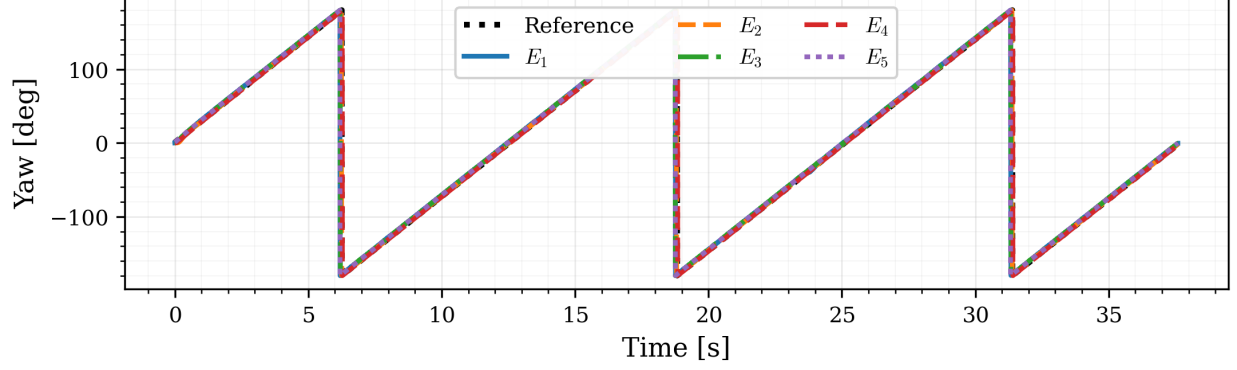
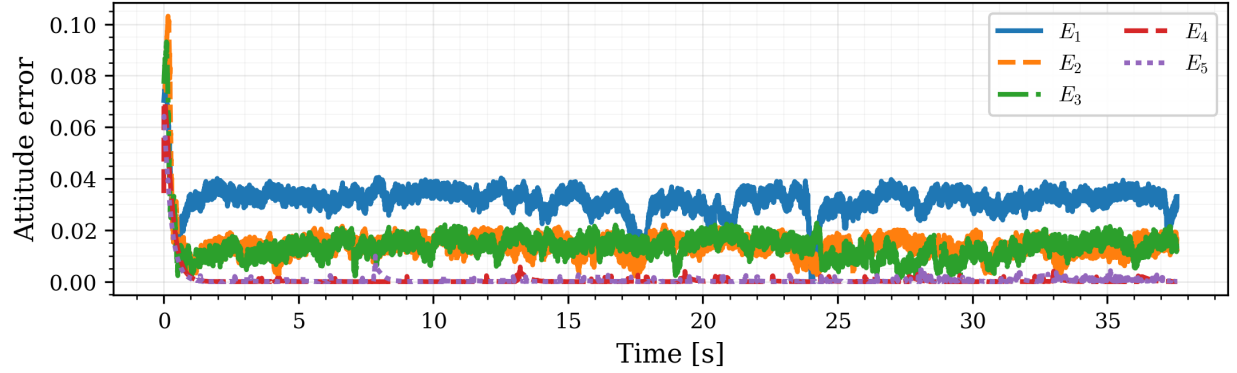
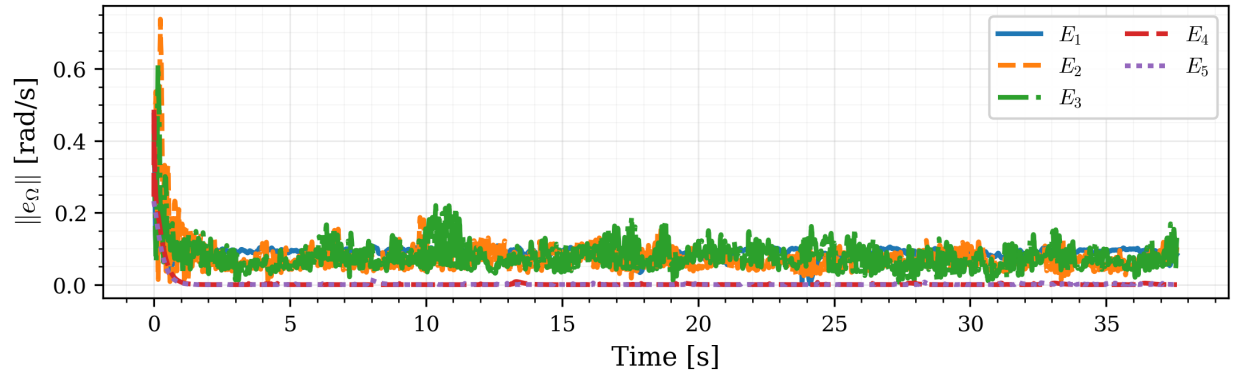


Figure 25: Yaw tracking against the trajectory-generator yaw reference. The sawtooth shape is caused by wrapping the yaw angle to $[-180^\circ, 180^\circ]$; it is a plotting convention and does not represent a discontinuity in the underlying rotation matrix.

All five experiments track the yaw reference nearly indistinguishably in Figure 25. The more meaningful attitude comparison is therefore given by the geometric rotational errors in Figure 26, which compare each experiment against its own controller-computed R_d and Ω_d .



(a) Attitude-error norm $\|e_R(t)\|$.



(b) Angular-velocity-error norm $\|e_\Omega(t)\|$.

Figure 26: Rotational tracking errors. The Isaac Sim configurations show larger sustained oscillations than the custom rigid-body solvers, especially in angular velocity.

The attitude-error norm separates the experiments into the same broad groups observed in the translational-error plots. E1 has the largest sustained attitude error, settling around $\|e_R\| \approx 0.03$. E2 and E3 form an intermediate band around $\|e_R\| \approx 0.01$ – 0.02 , while E4 and E5 remain close to zero after the initial transient. The angular-velocity-error norm shows the same trend but makes the high-frequency behaviour more visible: the Isaac Sim/Pegasus configurations, especially E2 and E3, exhibit persistent small-amplitude oscillations after the initial transient, whereas the custom RK4 and Euler solvers remain nearly flat. This suggests that the additional rotational activity is associated with the Isaac Sim/Pegasus physics and force-application path rather than with the controller itself.

6.4.4 Control Outputs

The control-output plots provide a useful diagnostic for distinguishing smooth model-following behaviour from high-frequency corrective action by the controller. The first second is omitted in Figures 27–28 so that the initial takeoff transient does not compress the steady-state scale.

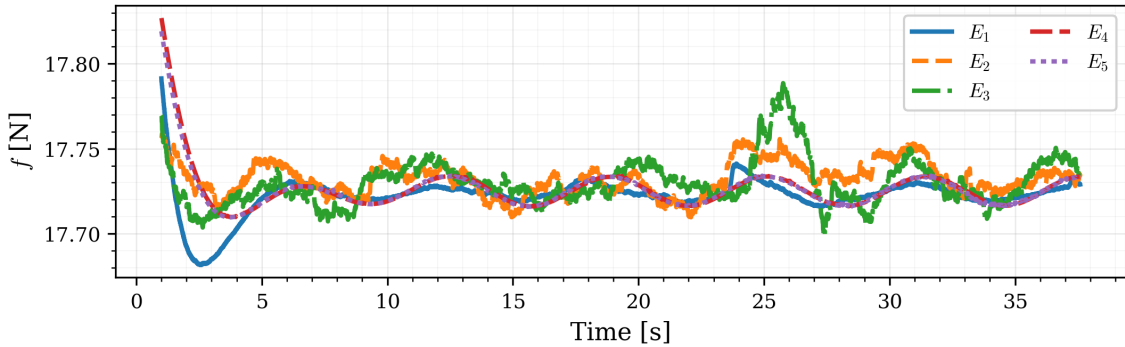
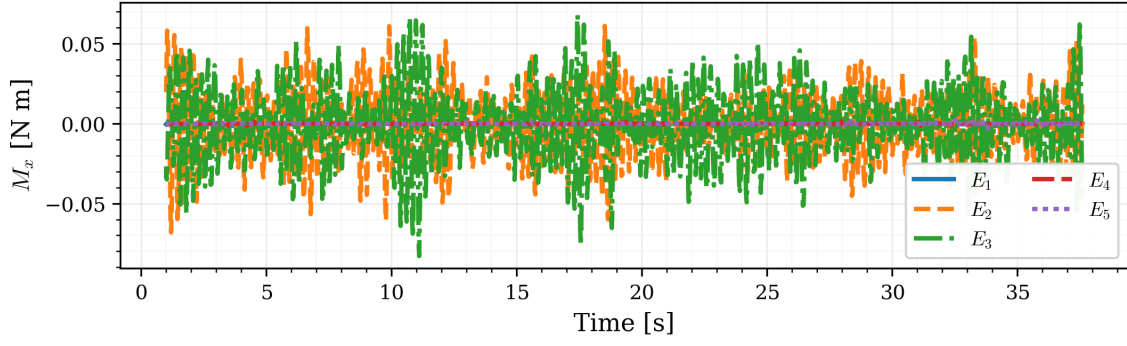


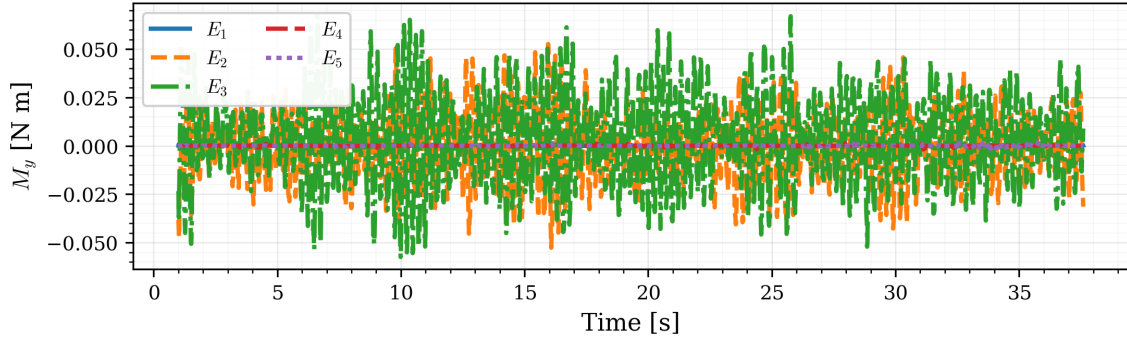
Figure 27: Commanded total thrust $f(t)$, plotted from $t = 1$ s onward.

Figure 27 shows that the commanded thrust remains close to the vehicle weight, $mg \approx 17.7$ N, in all five configurations. The custom-solver cases E4 and E5 are the smoothest, while the Isaac Sim/Pegasus cases show slightly larger ripple. E3 has the most visible thrust variation, especially around $t \approx 25$ – 27 s, consistent with the controller compensating for the noisier distributed-force vehicle response. Overall, none of the configurations requires large thrust excursions; the main differences between experiments appear more clearly in the moment commands.

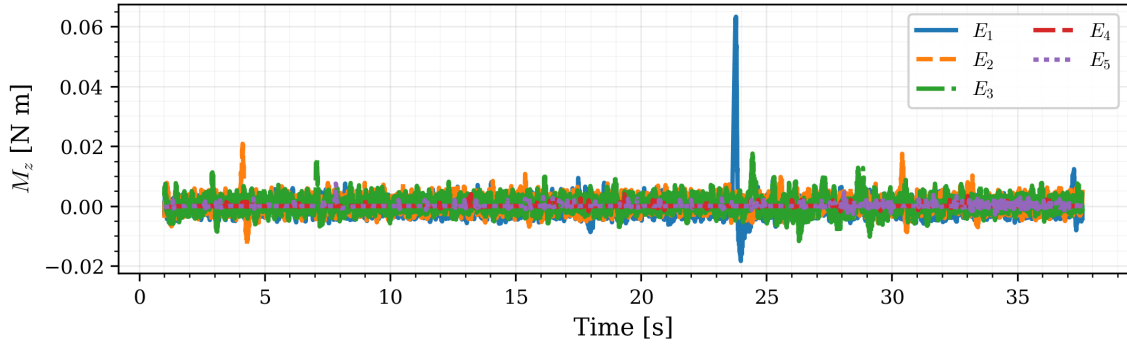
The moment commands in Figure 28 show a stronger separation between the Isaac Sim/Pegasus configurations and the custom rigid-body solvers. In M_x and M_y , E2 and E3 contain persistent high-frequency oscillations, while E4 and E5 remain close to zero with very little chatter. This is consistent with the custom solvers presenting a cleaner rigid-body model to the controller, whereas the Isaac Sim/Pegasus configurations introduce additional small corrective demands through the physics and force-application path. E1 is smoother than E2/E3 in the roll and pitch moments but contains an isolated yaw-moment spike near $t \approx 23$ – 24 s. Since the yaw angle wraps multiple times during the trajectory but this spike appears only once, it should not be attributed simply to yaw wrapping. It is better treated as an isolated transient or numerical/logging anomaly unless further debugging identifies a specific cause.



(a) Roll moment $M_x(t)$.



(b) Pitch moment $M_y(t)$.



(c) Yaw moment $M_z(t)$.

Figure 28: Commanded body-frame moments, plotted from $t = 1$ s onward.

The rotor commands in Figure 29 remain tightly clustered around the hover operating point, approximately 0.46–0.48, and stay well inside the saturation limits $[0, 1]$ for every configuration. This confirms that the observed tracking differences are not caused by actuator saturation. The same isolated E1 event near $t \approx 23$ –24 s appears in the rotor commands as a brief redistribution of throttle between rotors, matching the spike seen in M_z .

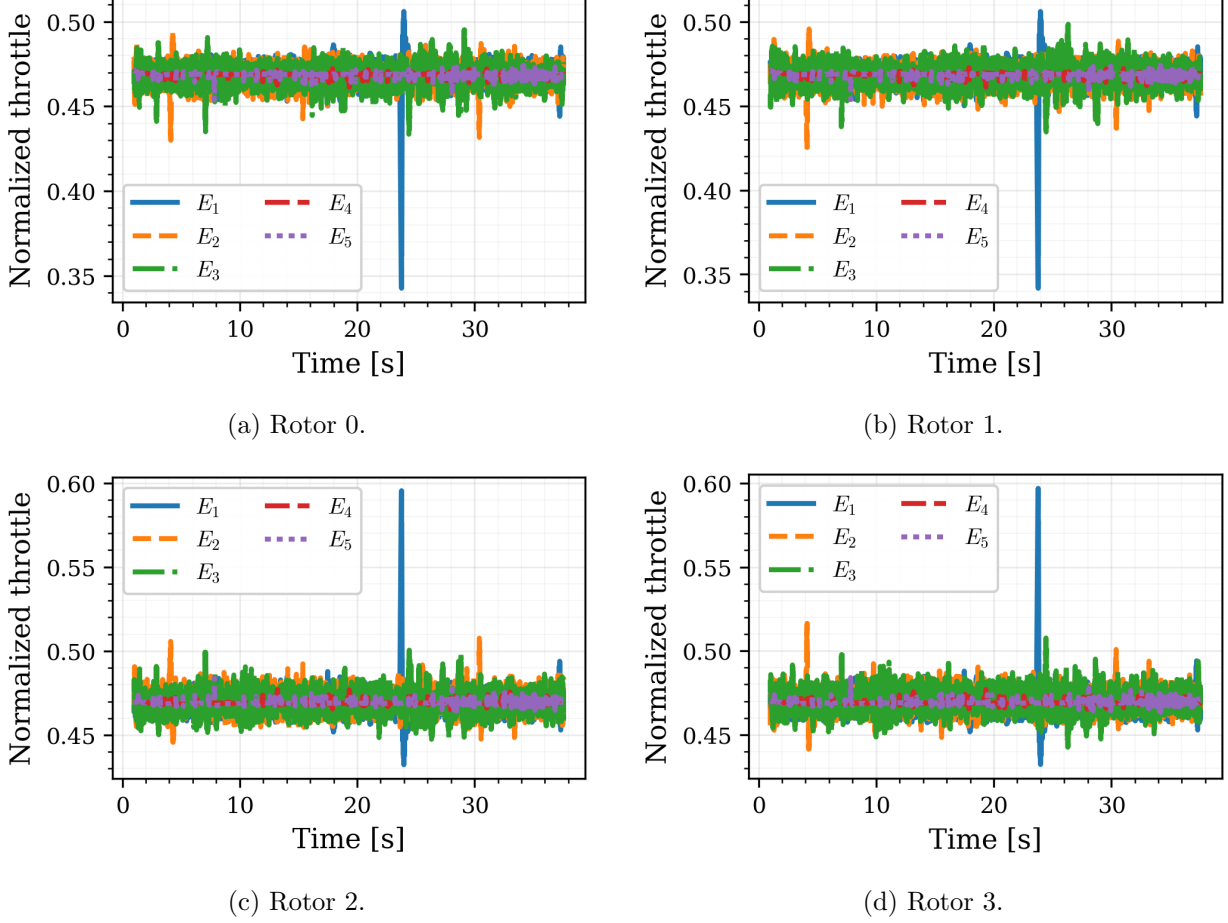


Figure 29: Per-rotor normalised throttle commands u_0, u_1, u_2, u_3 , plotted from $t = 1$ s onward.

6.5 Aggregate Metrics

The aggregate tracking metrics across all five experiments are summarised in Table 7. All metrics in this table are computed after the initial transient, using only samples with $t \geq 5$ s. This makes the numerical comparison consistent with the steady-state portions of the plots in Sections 6.

Table 7: Post-transient aggregate tracking metrics on the elliptic-helix benchmark, all five configurations. Errors are the controller-internal error variables defined in Section 6.3; all values are computed using samples with $t \geq 5$ s.

Configuration	RMS_{e_x} [m]	$\max \ e_x\ $ [m]	RMS_{e_v} [m/s]	RMS_{e_R} [-]	$\max \ e_R\ $ [-]	RMS_{e_Ω} [rad/s]
E1: Ideal block	0.368	0.506	0.208	0.0323	0.0405	0.0895
E2: F450 lumped	0.165	0.241	0.0934	0.0149	0.0220	0.0740
E3: F450 distributed	0.133	0.232	0.0887	0.0147	0.0235	0.0859
E4: Custom RK4	0.00929	0.0326	0.0100	0.000505	0.00570	0.00124
E5: Custom Euler	0.0107	0.0379	0.00956	0.000818	0.00995	0.00188

6.6 Discussion

The plots and post-transient metrics in Table 7 separate the five experiments into three broad performance bands. E1 has the largest sustained tracking error, E2 and E3 form an intermediate Isaac Sim/Pegasus band, and E4 and E5 form a low-error custom-solver band. The differences within the E2/E3 pair and within the E4/E5 pair are small compared with the gap between the Isaac Sim/Pegasus configurations and the custom rigid-body solvers.

E2 vs. E1 (effect of articulation). Replacing the single rigid block with the F450 articulation reduced the RMS position error from 0.368 m to 0.165 m, a factor of about 2.2. The RMS attitude error shows a similar reduction, from 0.0323 to 0.0149. This improvement is consistent with the position, velocity, and rotational error plots. A plausible explanation is that the ideal block in E1 carries the F450’s total inertia J at the centre of mass, whereas the articulated F450 in E2 distributes part of that inertia into the rotor links. As a result, the effective dynamics seen by the controller are not identical, even though the same nominal controller parameters are used. The main takeaway is that E1 should not be interpreted as a physically superior baseline simply because it is simpler. It is a useful sanity check for the controller and PhysX under a single-rigid-body model, but its dynamics are not equivalent to the articulated F450 model.

E3 vs. E2 (effect of force distribution). Distributing the four rotor forces across the rotor links instead of applying the full wrench at the body produced a modest improvement in translational tracking but little change in rotational tracking. RMS position error decreases from 0.165 m to 0.133 m, while the maximum position error changes only slightly from 0.241 m to 0.232 m. The attitude metrics remain nearly unchanged: RMS e_R is 0.0149 for E2 and 0.0147 for E3. The angular-velocity metric is slightly worse in E3, increasing from 0.0740 rad/s to 0.0859 rad/s, which is consistent with the higher-frequency activity seen in the Isaac Sim/Pegasus control-output plots. Overall, the force-distribution choice matters less than the broader difference between the Isaac Sim/Pegasus physics path and the custom rigid-body solvers.

E4 vs. E3 (Isaac Sim/Pegasus physics path vs. custom rigid-body solver). Replacing the Isaac Sim/Pegasus articulated-vehicle path with the custom rigid-body solver produced the largest change in the benchmark. RMS position error drops from 0.133 m to 0.00929 m, a factor of about 14. The maximum position error drops from 0.232 m to 0.0326 m, a factor of about 7. The rotational metrics show an even larger separation: RMS attitude error drops from 0.0147 to 0.000505, and RMS angular-velocity error drops from 0.0859 rad/s to 0.00124 rad/s. The control-output plots provide a complementary view of the same effect: the Isaac Sim/Pegasus cases require visibly noisier roll and pitch moment commands, while the custom-solver cases are nearly flat after the transient. The data therefore indicate that the dominant observed gap is between the Isaac Sim/Pegasus physics path and the clean custom rigid-body solver, not between the two Isaac Sim force-application models.

E5 vs. E4 (RK4 vs. Euler). Switching from RK4 to forward Euler within the custom solver produced only a small change at the controller’s 250 Hz update rate. E5 has a slightly larger RMS position error than E4 (0.0107 m vs. 0.00929 m) and a slightly larger RMS attitude error (0.000818 vs. 0.000505), while its RMS velocity error is marginally smaller (0.00956 m/s vs. 0.0100 m/s). These differences are very small in absolute terms compared with the gap between E3 and E4. For this smooth trajectory and this controller bandwidth, integration order within the custom solver is not the limiting factor.

Headline finding. The dominant difference observed in this benchmark is between the Isaac Sim/Pegasus physics path and the custom rigid-body solver. E2 and E3 show that, once the articulated F450 model is used, distributing the rotor forces across the rotor links instead of applying a lumped wrench has only a secondary effect on the aggregate metrics. E4 and E5 show that, for this smooth trajectory at 250 Hz, the custom solver is insensitive to whether RK4 or forward Euler is used. The most physically detailed Isaac Sim configuration considered here, E3, has roughly $14\times$ the RMS position error, $7\times$ the maximum position error, $29\times$ the RMS attitude error, and $69\times$ the RMS angular-velocity error of the custom RK4 solver. For controller-law verification against an ideal rigid-body model, the custom solver is therefore the cleaner reference. For applications requiring contact dynamics, articulation, sensor simulation, or visual realism, Isaac Sim remains the appropriate environment, but the user should expect additional closed-loop tracking error and control chatter relative to a clean rigid-body integrator.

7 Conclusion and Future Work

7.1 Conclusion

This project delivered a complete pipeline from CAD modelling through to a controlled simulator-validation benchmark for the F450 quadrotor in NVIDIA Isaac Sim. By using a theoretically well-understood geometric tracking controller as a fixed instrument and varying the body model, force-application model, and physics provider, the project quantified how these choices affect closed-loop tracking error in simulation. The five-experiment ladder—ideal block, F450 lumped, F450 distributed, custom RK4, and custom Euler—provides the FSC Lab with a practical basis for deciding when Isaac Sim/Pegasus is useful as a high-fidelity robotics environment and when a transparent custom rigid-body solver is more appropriate as a controller-verification reference.

The benchmark shows that the largest observed gap is not between lumped and distributed force application inside Isaac Sim, nor between RK4 and Euler integration inside the custom solver. Instead, the largest gap is between the Isaac Sim/Pegasus physics path and the custom rigid-body solver. The Isaac Sim/Pegasus configurations retain substantially larger post-transient position, attitude, and angular-velocity errors, while the custom solvers closely reproduce the nominal rigid-body behaviour assumed by the controller. The CAD-derived vehicle model, linear thrust identification, analytical feed-forward derivation, and ROS 2 integration form a reusable basis for future controller and simulator-validation studies.

7.2 Limitations

This benchmark validates Isaac Sim under nominal rigid-body assumptions only. The experiments do not include aerodynamic drag, rotor wake, ground effect, motor dynamics, sensor noise, actuator delay, or hardware flight data. In addition, the yaw moment coefficient was estimated rather than measured. The E3–E4 comparison also changes more than the numerical integrator: it replaces the Isaac Sim/Pegasus articulated-vehicle path with a custom rigid-body solver. Therefore, the results should be interpreted as a controlled simulator/model-fidelity benchmark, not as a complete simulation-to-hardware validation or a pure isolated comparison of numerical integration schemes.

7.3 Future Work

Several directions build naturally on this work:

- **Hardware validation.** The natural next experiment is a sixth configuration: the same controller running on the FSC Lab F450 with a motion-capture or VIO state estimator. The gap between the hardware and the closest of the five simulated configurations would quantify the simulation-to-hardware transfer error, completing the validation ladder.
- **Accurate rotor-moment identification.** The rolling-moment coefficient k_m was estimated as an order-of-magnitude placeholder. A direct measurement on a torque bench is needed before the simulation results can be compared meaningfully to hardware, since an incorrect k_m scales the yaw-moment column of the allocation matrix.
- **Aerodynamic effects.** All five configurations are aerodynamics-free rigid-body simulations. Adding rotor wash, drag, and ground effect would reveal which Isaac Sim features, extensions, or third-party plugins close the remaining simulation-to-hardware gap.
- **More demanding trajectories.** The elliptic helix is non-symmetric but still smooth and far from the controller’s region-of-attraction boundaries. Aggressive manoeuvres, such as flips and large initial attitude errors, would test the controller’s almost-global guarantees of Proposition 3 and stress the simulator physics path further.
- **Disturbance and noise modelling.** Adding state-estimator noise and external-disturbance models to the benchmark would extend the validation from “is Isaac Sim’s nominal physics accurate?” to “are the noise and disturbance properties of Isaac Sim representative of hardware?”
- **Adaptive control.** Lee et al.’s adaptive extension accounts for unknown mass and inertia and would benefit from the validation pipeline established here, since modelling discrepancies between the controller’s internal model and the simulated body are exactly what the adaptive law is meant to compensate for.

References

- [1] T. Lee, M. Leok, and N. H. McClamroch, “Geometric tracking control of a quadrotor UAV on $SE(3)$,” in *Proc. IEEE Conf. Decision and Control (CDC)*, 2010, pp. 5420–5425.
- [2] M. Jacinto, J. Pinto, J. Patrikar, J. Keller, R. Cunha, S. Scherer, and A. Pascoal, “Pegasus Simulator: An Isaac Sim framework for multiple aerial vehicles simulation,” *arXiv preprint arXiv:2307.05263*, 2023.
- [3] NVIDIA Corporation, *Isaac Sim*, <https://developer.nvidia.com/isaac-sim>.
- [4] Open Robotics, *ROS 2 Documentation*, <https://docs.ros.org/en/humble/>.
- [5] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 2520–2525.

A Geometric Tracking Controller: Full Derivation and Stability Proofs

This appendix gives the complete derivation, error-vector definitions, and stability analysis for the geometric tracking controller of [1] that is summarised in Section 4 of the main report. It is intended to be read as a self-contained reference; the main report contains only the high-level structure and a statement of results.

A.1 Dynamics Model

This subsection collects the configuration manifold, modelling assumptions, and the rotor-to- (f, M) allocation that underlie the equations of motion stated in the main report. The reference-frame setup is illustrated in Figure 30.

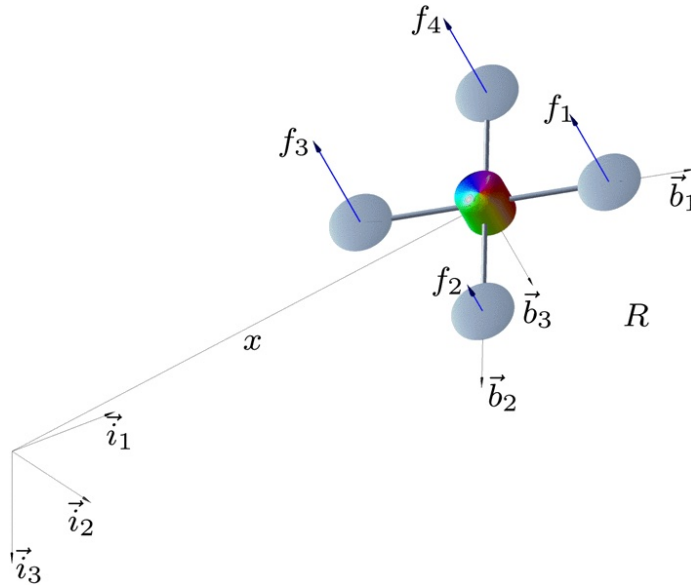


Figure 30: Quadrotor model showing the inertial frame $\{\vec{i}_1, \vec{i}_2, \vec{i}_3\}$, the body-fixed frame $\{\vec{b}_1, \vec{b}_2, \vec{b}_3\}$, the position x of the centre of mass, the rotation matrix R , and the four rotor thrust vectors $f_1, \dots, f_4$. Figure adapted from [1].

A.1.1 Configuration Manifold

The configuration of the quadrotor is fully described by $(x, R) \in \text{SE}(3)$, the special Euclidean group,

$$\text{SE}(3) = \mathbb{R}^3 \rtimes \text{SO}(3), \quad \text{SO}(3) = \{R \in \mathbb{R}^{3 \times 3} \mid R^\top R = I, \det R = 1\}.$$

A.1.2 Modelling Assumptions

1. Thrust of each propeller is directly controlled; rotor and propeller dynamics are ignored.
2. Propellers 1 and 3 rotate *clockwise*; propellers 2 and 4 rotate *counter-clockwise*.
3. The torque generated by each propeller is proportional to its thrust, $\tau_i = (-1)^i c_{\tau f} f_i$, for a fixed constant $c_{\tau f} > 0$.

A.1.3 Thrust and Moment Mapping

The total thrust f and body-fixed moment $M = [M_1, M_2, M_3]^\top$ are related to the individual rotor thrusts by

$$\begin{bmatrix} f \\ M_1 \\ M_2 \\ M_3 \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & -d & 0 & d \\ d & 0 & -d & 0 \\ -c_{\tau f} & c_{\tau f} & -c_{\tau f} & c_{\tau f} \end{bmatrix}}_{\mathcal{A}} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \end{bmatrix}.$$

The determinant of $\mathcal{A}$ is $8c_{\tau f}d^2 \neq 0$, so we treat $(f, M) \in \mathbb{R} \times \mathbb{R}^3$ as the four effective control inputs.

A.1.4 Equations of Motion

Using Newton's second law and Euler's rotation equations,

$$\begin{aligned} \dot{x} &= v, \\ m\dot{v} &= mge_3 - fRe_3, \\ \dot{R} &= R\hat{\Omega}, \\ J\dot{\Omega} + \Omega \times J\Omega &= M, \end{aligned}$$

where $e_3 = [0, 0, 1]^\top$ and $\hat{\cdot}: \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ is the hat map. These are the same as (3)–(6) in the main report and have a *cascade structure*: the rotational dynamics are independent of the translational state, while the translational dynamics depend on R only through fRe_3 .

A.2 Tracking Error Definitions

A.2.1 Position and Velocity Errors

$e_x = x - x_d$ and $e_v = v - v_d$.

A.2.2 Attitude Error Function

Since $R, R_d \in \text{SO}(3)$ cannot be subtracted directly, define

$$\Psi(R, R_d) = \frac{1}{2} \text{tr}(I - R_d^\top R). \quad (21)$$

Properties of Ψ . $\Psi \geq 0$, with $\Psi = 0$ iff $R = R_d$. Applying the Rodrigues formula

$$Q = I + \sin \theta \hat{k} + (1 - \cos \theta) \hat{k}^2, \quad (22)$$

(for axis k with $\|k\| = 1$ and angle $\theta \in [0, \pi)$) to $Q = R_d^\top R$, together with the hat-map identities

$$\text{tr}(\hat{x}) = 0, \quad (23)$$

$$\text{tr}(\hat{x}\hat{y}) = -2x^\top y, \quad (24)$$

yields

$$\Psi = 1 - \cos \theta. \quad (25)$$

Hence $\Psi < 2$ iff $\theta < 180^\circ$, defining the sublevel set $\mathcal{L}_2 = \{R, R_d \in \text{SO}(3) \mid \Psi(R, R_d) < 2\}$, which *almost covers* $\text{SO}(3)$.

A.2.3 Attitude Tracking Error Vector

The tangent space at $R \in \text{SO}(3)$ is $T_R\text{SO}(3) = \{R\hat{\eta} \mid \eta \in \mathbb{R}^3\}$, so a small variation of R has the form $\delta R = R\hat{\eta}$. Differentiating (21) in this direction gives

$$D_R\Psi(R, R_d) \cdot R\hat{\eta} = -\frac{1}{2}\text{tr}\left[R_d^\top R\hat{\eta}\right].$$

Using the decomposition of any matrix A into symmetric and skew-symmetric parts, one shows $\text{tr}(A_S\hat{\eta}) = 0$, so

$$\text{tr}(A\hat{\eta}) = \text{tr}\left[\frac{1}{2}(A - A^\top)\hat{\eta}\right]. \quad (26)$$

Applying (26) with $A = R_d^\top R$ and simplifying using (24),

$$D_R\Psi(R, R_d) \cdot R\hat{\eta} = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee \cdot \eta, \quad (27)$$

where the *vee map* $(\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ is the inverse of the hat map. Since (27) holds for all $\eta \in \mathbb{R}^3$, we define the **attitude tracking error vector**

$$e_R = \frac{1}{2}(R_d^\top R - R^\top R_d)^\vee. \quad (28)$$

Remark 1 (Geometric interpretation). Writing $Q = R_d^\top R$ with Rodrigues formula (22), $e_R = \sin \theta k$, so $\|e_R\| = |\sin \theta|$. Using (25),

$$\Psi(2 - \Psi) = \sin^2 \theta = \|e_R\|^2. \quad (29)$$

A.2.4 Angular Velocity Tracking Error

$\dot{R} \in T_R\text{SO}(3)$ and $\dot{R}_d \in T_{R_d}\text{SO}(3)$ lie in different tangent spaces. Transporting $\dot{R}_d$ into $T_R\text{SO}(3)$ gives $\dot{R}_d(R_d^\top R) = R(R^\top R_d \Omega_d)^\wedge$, leading to

$$e_\Omega = \Omega - R^\top R_d \Omega_d. \quad (30)$$

Differentiating $R_d^\top R$ yields the useful identity

$$\frac{d}{dt}(R_d^\top R) = (R_d^\top R)\widehat{e_\Omega}. \quad (31)$$

A.3 Tracking Controller

Given smooth commands $x_d(t)$, $\vec{b}_{1d}(t)$ and positive gains k_x, k_v, k_R, k_Ω , the desired third body-axis is

$$\vec{b}_{3d} = -\frac{-k_x e_x - k_v e_v - m g e_3 + m \ddot{x}_d}{\| -k_x e_x - k_v e_v - m g e_3 + m \ddot{x}_d \|}, \quad (32)$$

assuming the denominator is non-zero. We also assume

$$\| -m g e_3 + m \ddot{x}_d \| < B \quad (33)$$

for some $B > 0$. The control inputs are

$$f = -(-k_x e_x - k_v e_v - m g e_3 + m \ddot{x}_d) \cdot R e_3, \quad (34)$$

$$M = -k_R e_R - k_\Omega e_\Omega + \Omega \times J \Omega - J(\widehat{\Omega} R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d). \quad (35)$$

Physical intuition. The moment M is a nonlinear PD controller with feed-forward. The $-k_R e_R - k_\Omega e_\Omega$ terms are proportional/derivative, while $\Omega \times J\Omega - J(\widehat{\Omega}R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d)$ cancels the gyroscopic coupling and the terms arising from the time-varying desired attitude. If these cancellations are exact, the closed-loop dynamics reduce to $J\dot{e}_\Omega = -k_R e_R - k_\Omega e_\Omega$. For the thrust, note that $f = A \cdot Re_3$ where $A = -k_x e_x - k_v e_v - mge_3 + m\ddot{x}_d$. When $R = R_d$, $Re_3 = R_d e_3 = -A/\|A\|$, so $-fRe_3 = A$, recovering a PD translational controller with feed-forward. When the attitude error is large, $A \cdot Re_3$ decreases, automatically reducing the thrust to prevent applying a large force in the wrong direction.

A.4 Exponential Stability of the Attitude Dynamics

Proposition 1 (Exponential Stability of Attitude Dynamics). *Consider the control moment M defined in (35) for any positive constants k_R, k_Ω . Suppose the initial conditions satisfy*

$$\Psi(R(0), R_d(0)) < 2, \quad (36)$$

$$\|e_\Omega(0)\|^2 < \frac{2}{\lambda_{\max}(J)} k_R (2 - \Psi(R(0), R_d(0))). \quad (37)$$

Then the zero equilibrium of (e_R, e_Ω) is exponentially stable, and there exist constants $\alpha_2, \beta_2 > 0$ such that $\Psi(R(t), R_d(t)) \leq \min\{2, \alpha_2 e^{-\beta_2 t}\}$.

Remark 2. Condition (36) states that the initial attitude error is less than 180° ; the region of attraction *almost covers* $\text{SO}(3)$. Condition (37) can be relaxed by choosing a larger k_R .

The proof proceeds in four steps.

Step 1: Error Dynamics

Attitude error dynamics. Differentiating (28),

$$\dot{e}_R = \frac{1}{2} \left(\frac{d}{dt}(R_d^\top R) - \frac{d}{dt}(R^\top R_d) \right)^\vee.$$

From (31), $\frac{d}{dt}(R_d^\top R) = (R_d^\top R)\widehat{e_\Omega}$. Transposing,

$$\frac{d}{dt}(R^\top R_d) = -\widehat{e_\Omega} R^\top R_d,$$

using $\widehat{e_\Omega}^\top = -\widehat{e_\Omega}$. Hence

$$\dot{e}_R = \frac{1}{2} \left(R_d^\top R \widehat{e_\Omega} + \widehat{e_\Omega} R^\top R_d \right)^\vee.$$

By direct expansion one verifies the identity

$$(A\widehat{x} + \widehat{x}A^\top)^\vee = (\text{tr}(A)I - A^\top)x. \quad (38)$$

Applying (38) with $A = R_d^\top R$ and $x = e_\Omega$,

$$\dot{e}_R = \frac{1}{2} \left(\text{tr}(R_d^\top R)I - R^\top R_d \right) e_\Omega \equiv C(R_d^\top R) e_\Omega. \quad (39)$$

Key bound on C . Applying Rodrigues to $R^\top R_d$ with axis k , angle θ , and using $\text{tr}(R_d^\top R) = 1 + 2\cos\theta$ from (25),

$$C(R_d^\top R) = \frac{1}{2}(1 + \cos\theta)I - \frac{1}{2}(1 - \cos\theta)kk^\top - \frac{1}{2}\sin\theta\widehat{k}.$$

Decomposing any vector into components along and perpendicular to k :

- *Along k* : $C(R_d^\top R)k = \cos \theta k$, so $\|C(R_d^\top R)k\|_2 \leq \|k\|_2$.
- *Perpendicular to k* : for $v \perp k$, $\|C(R_d^\top R)v\|_2^2 = \frac{1}{2}(1 + \cos \theta)\|v\|_2^2 \leq \|v\|_2^2$.

Therefore $\|C(R_d^\top R)\|_2 \leq 1$, and from (39),

$$\|\dot{e}_R\| \leq \|e_\Omega\|. \quad (40)$$

Angular velocity error dynamics. Differentiating (30) and using $\frac{d}{dt}(R^\top R_d) = -\widehat{e}_\Omega R^\top R_d$,

$$\dot{e}_\Omega = \dot{\Omega} + \widehat{e}_\Omega R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d.$$

Multiplying by J and using $J\dot{\Omega} = M - \widehat{\Omega}J\Omega$ gives

$$J\dot{e}_\Omega = M - \widehat{\Omega}J\Omega + J(\widehat{e}_\Omega R^\top R_d \Omega_d - R^\top R_d \dot{\Omega}_d).$$

Since $\widehat{e}_\Omega = \widehat{\Omega} - (R^\top R_d \Omega_d)^\wedge$ and $\widehat{a}a = 0$, we have $\widehat{e}_\Omega R^\top R_d \Omega_d = \widehat{\Omega} R^\top R_d \Omega_d$. Substituting (35) and cancelling gives

$$J\dot{e}_\Omega = -k_R e_R - k_\Omega e_\Omega. \quad (41)$$

Step 2: Lyapunov Function

For a constant $c_2 > 0$, define

$$V_2 = \frac{1}{2} e_\Omega^\top J e_\Omega + k_R \Psi(R, R_d) + c_2 e_R \cdot e_\Omega. \quad (42)$$

Differentiating and substituting (39), (41):

$$\dot{V}_2 = e_\Omega \cdot J\dot{e}_\Omega + k_R \frac{d}{dt} \Psi + c_2 C(R_d^\top R) e_\Omega \cdot e_\Omega + c_2 e_R \cdot J^{-1}(-k_R e_R - k_\Omega e_\Omega).$$

Using (31), (26), (24), one shows $k_R \frac{d}{dt} \Psi = k_R e_\Omega \cdot e_R$. Substituting (41) and cancelling $\pm k_R e_R \cdot e_\Omega$,

$$\dot{V}_2 = -k_\Omega \|e_\Omega\|^2 + c_2 C(R_d^\top R) e_\Omega \cdot e_\Omega - c_2 k_R e_R \cdot J^{-1} e_R - c_2 k_\Omega e_R \cdot J^{-1} e_\Omega. \quad (43)$$

Applying $\|C\|_2 \leq 1$, $\frac{1}{\lambda_{\max}(J)} I \preceq J^{-1} \preceq \frac{1}{\lambda_{\min}(J)} I$, and Cauchy-Schwarz,

$$\dot{V}_2 \leq -z_2^\top W_2 z_2, \quad (44)$$

with $z_2 = [\|e_R\|, \|e_\Omega\|]^\top$ and

$$W_2 = \begin{bmatrix} \frac{c_2 k_R}{\lambda_{\max}(J)} & -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} \\ -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} & k_\Omega - c_2 \end{bmatrix}.$$

Step 3: $(R(t), R_d(t))$ Remains in $\mathcal{L}_2$

Setting $c_2 = 0$, $V_2|_{c_2=0}$ is non-increasing. At $t = 0$ and using (37),

$$\begin{aligned} k_R \Psi(R(t), R_d(t)) &\leq V_2|_{c_2=0}(0) = \frac{1}{2} e_\Omega(0)^\top J e_\Omega(0) + k_R \Psi(R(0), R_d(0)) \\ &\leq \frac{1}{2} \lambda_{\max}(J) \|e_\Omega(0)\|^2 + k_R \Psi(R(0), R_d(0)) < 2k_R. \end{aligned}$$

So $\Psi < 2$ for all $t \geq 0$, i.e. $(R(t), R_d(t)) \in \mathcal{L}_{\psi_2} \subset \mathcal{L}_2$ for some $\psi_2 < 2$.

From (29), $\|e_R\|^2 = \Psi(2 - \Psi) \leq 2\Psi$, so $\Psi \geq \frac{1}{2}\|e_R\|^2$. Since $\Psi \leq \psi_2$, $\|e_R\|^2 \geq \Psi(2 - \psi_2)$, so $\Psi \leq \|e_R\|^2 / (2 - \psi_2)$. Hence

$$\frac{1}{2} \|e_R\|^2 \leq \Psi \leq \frac{1}{2 - \psi_2} \|e_R\|^2. \quad (45)$$

Step 4: Exponential Stability

Within $\mathcal{L}_{\psi_2}$, applying (45) and $\lambda_{\min}(J)\|e_\Omega\|^2 \leq e_\Omega \cdot J e_\Omega \leq \lambda_{\max}(J)\|e_\Omega\|^2$ to each term of (42),

$$z_2^\top M_{21} z_2 \leq V_2 \leq z_2^\top M_{22} z_2, \quad (46)$$

where

$$M_{21} = \frac{1}{2} \begin{bmatrix} k_R & -c_2 \\ -c_2 & \lambda_{\min}(J) \end{bmatrix}, \quad M_{22} = \frac{1}{2} \begin{bmatrix} \frac{2k_R}{2-\psi_2} & c_2 \\ c_2 & \lambda_{\max}(J) \end{bmatrix}.$$

We choose c_2 to make W_2, M_{21}, M_{22} all positive-definite. For a 2×2 symmetric matrix, positive-definiteness requires the diagonal entries and the determinant to be positive. The three conditions give

$$c_2 < \min \left\{ k_\Omega, \frac{4k_\Omega k_R \lambda_{\min}(J)^2}{k_\Omega^2 \lambda_{\max}(J) + 4k_R \lambda_{\min}(J)^2}, \sqrt{k_R \lambda_{\min}(J)}, \sqrt{\frac{2}{2-\psi_2} k_R \lambda_{\max}(J)} \right\}. \quad (47)$$

From (46), $\lambda_{\min}(M_{21})\|z_2\|^2 \leq V_2 \leq \lambda_{\max}(M_{22})\|z_2\|^2$, and from (44) and $\|z_2\|^2 \geq V_2/\lambda_{\max}(M_{22})$,

$$\dot{V}_2 \leq -\frac{\lambda_{\min}(W_2)}{\lambda_{\max}(M_{22})} V_2 = -\beta_2 V_2.$$

Grönwall's inequality gives $V_2(t) \leq V_2(0)e^{-\beta_2 t}$. The Lyapunov exponential-stability theorem then yields exponential stability of $(e_R, e_\Omega) = (0, 0)$. Moreover,

$$\Psi(R(t), R_d(t)) \leq \frac{1}{\lambda_{\min}(M_{21})(2-\psi_2)} V_2(0) e^{-\beta_2 t},$$

so $\Psi(R(t), R_d(t)) \leq \min\{2, \alpha_2 e^{-\beta_2 t}\}$. $\square$

A.5 Exponential Stability of the Complete Dynamics

Proposition 2 (Exponential Stability of the Complete Dynamics). *Consider f, M from (34), (35), and suppose*

$$\Psi(R(0), R_d(0)) \leq \psi_1 < 1. \quad (48)$$

Define

$$\begin{aligned} \widetilde{W}_1 &= \begin{bmatrix} \frac{c_1 k_x}{m}(1-\alpha) & -\frac{c_1 k_v}{2m}(1+\alpha) \\ -\frac{c_1 k_v}{2m}(1+\alpha) & k_v(1-\alpha) - c_1 \end{bmatrix}, \quad W_{12} = \begin{bmatrix} k_x e_{v,\max} + \frac{c_1}{m} B & 0 \\ B & 0 \end{bmatrix}, \\ W_2 &= \begin{bmatrix} \frac{c_2 k_R}{\lambda_{\max}(J)} & -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} \\ -\frac{c_2 k_\Omega}{2\lambda_{\min}(J)} & k_\Omega - c_2 \end{bmatrix}, \end{aligned} \quad (49)$$

with $\alpha = \sqrt{\psi_1(2-\psi_1)}$, $e_{v,\max} = \max\{\|e_v(0)\|, B/(k_v(1-\alpha))\}$, and $c_1, c_2 \in \mathbb{R}_{>0}$. For any $k_x, k_v > 0$, choose $c_1, c_2, k_R, k_\Omega > 0$ such that

$$c_1 < \min \left\{ k_v(1-\alpha), \frac{4mk_x k_v (1-\alpha)^2}{k_v^2 (1+\alpha)^2 + 4mk_x (1-\alpha)}, \sqrt{k_x m} \right\}, \quad (50)$$

$$c_2 < \min \left\{ k_\Omega, \frac{4k_\Omega k_R \lambda_{\min}(J)^2}{k_\Omega^2 \lambda_{\max}(J) + 4k_R \lambda_{\min}(J)^2}, \sqrt{k_R \lambda_{\min}(J)}, \sqrt{\frac{2}{2-\psi_1} k_R \lambda_{\max}(J)} \right\}, \quad (51)$$

$$\lambda_{\min}(W_2) > \frac{\|W_{12}\|^2}{4\lambda_{\min}(\widetilde{W}_1)}. \quad (52)$$

Then the zero equilibrium of the complete tracking error is exponentially stable, with region of attraction characterised by (48) and

$$\|e_\Omega(0)\|^2 < \frac{2}{\lambda_{\max}(J)} k_R(1 - \Psi(R(0), R_d(0))). \quad (53)$$

Remark 3. The matrix $\widetilde{W}_1$ in (49) differs from the W_1 stated in [1] but is correct according to the bound on $\dot{V}_1$ derived below; see (63).

Remark 4. Condition (52) is a less restrictive bound on $\lambda_{\min}(W_2)$ than the one in [1] and is sufficient for $\widetilde{W} \succ 0$; see (65).

The proof proceeds in six steps.

Step 1: Attitude error remains in the sublevel set $\Psi < 1$

Conditions (48), (53) satisfy the hypotheses of Proposition 1. Setting $c_2 = 0$ in (42), (43),

$$\dot{V}_2|_{c_2=0} = -k_\Omega \|e_\Omega\|^2 \leq 0,$$

so $V_2|_{c_2=0}$ is non-increasing. Using (53),

$$k_R \Psi(R(t), R_d(t)) \leq \frac{1}{2} \lambda_{\max}(J) \|e_\Omega(0)\|^2 + k_R \Psi(R(0), R_d(0)) < k_R,$$

so $\Psi(R(t), R_d(t)) < 1$ for all $t \geq 0$, hence there is $\psi_1 < 1$ with

$$\Psi(R(t), R_d(t)) \leq \psi_1 < 1 \quad \forall t \geq 0. \quad (54)$$

From (25), this implies $\cos \theta(t) > 0$, i.e. $\theta(t) < \pi/2$.

Step 2: Translational error dynamics

From (4),

$$m\dot{e}_v = mge_3 - fRe_3 - m\ddot{x}_d. \quad (55)$$

Define $A = -k_x e_x - k_v e_v - mge_3 + m\ddot{x}_d$. By (32), (34), $R_d e_3 = -A/\|A\|$ and $f = -A \cdot Re_3$. Let $s = (R_d e_3) \cdot (Re_3)$. Since $\theta < \pi/2$, $s \geq \cos \theta > 0$. Adding and subtracting $(f/s)R_d e_3$,

$$m\dot{e}_v = mge_3 - m\ddot{x}_d - \frac{f}{s} R_d e_3 - X, \quad X = \frac{f}{s} (sRe_3 - R_d e_3). \quad (56)$$

Using $f = \|A\|s$ and $R_d e_3 = -A/\|A\|$, $-\frac{f}{s} R_d e_3 = A$. Substituting,

$$m\dot{e}_v = -k_x e_x - k_v e_v - X. \quad (57)$$

Step 3: Bound on the coupling term X

$\|X\| \leq \|A\| \|sRe_3 - R_d e_3\|$. Letting ϕ be the angle between Re_3 and $R_d e_3$, $\|sRe_3 - R_d e_3\|^2 = \sin^2 \phi$, so $\|sRe_3 - R_d e_3\| = \sin \phi$. Since $\phi \leq \theta$, $\sin \phi \leq \sin \theta = \|e_R\|$, giving

$$\|sRe_3 - R_d e_3\| \leq \|e_R\|.$$

Combined with $\|A\| \leq k_x \|e_x\| + k_v \|e_v\| + B$ (from (33)),

$$\|X\| \leq (k_x \|e_x\| + k_v \|e_v\| + B) \|e_R\|. \quad (58)$$

Since $\Psi \leq \psi_1 < 1$ and $g(x) = x(2-x)$ is increasing on $[0, 1)$, $\|e_R\| \leq \alpha = \sqrt{\psi_1(2-\psi_1)}$.

Step 4: Lyapunov function for the translational dynamics

For $c_1 > 0$,

$$V_1 = \frac{1}{2}k_x\|e_x\|^2 + \frac{1}{2}m\|e_v\|^2 + c_1e_x \cdot e_v. \quad (59)$$

Differentiating using $\dot{e}_x = e_v$ and (57),

$$\dot{V}_1 = -(k_v - c_1)\|e_v\|^2 - \frac{c_1k_x}{m}\|e_x\|^2 - \frac{c_1k_v}{m}e_x \cdot e_v - X \cdot \left(\frac{c_1}{m}e_x + e_v\right). \quad (60)$$

Using (58) and $\|e_R\| \leq \alpha$,

$$\begin{aligned} \dot{V}_1 \leq & -(k_v(1 - \alpha) - c_1)\|e_v\|^2 - \frac{c_1k_x}{m}(1 - \alpha)\|e_x\|^2 + \frac{c_1k_v}{m}(1 + \alpha)\|e_x\|\|e_v\| \\ & + \|e_R\| (k_x\|e_x\|\|e_v\| + \frac{c_1}{m}B\|e_x\| + B\|e_v\|). \end{aligned} \quad (61)$$

Step 5: Uniform bound on $\|e_v\|$

The cubic term $k_x\|e_R\|\|e_x\|\|e_v\|$ in (61) must be converted to a quadratic. Setting $c_1 = k_x = 0$ in (59), (61),

$$V_1|_{c_1=k_x=0} = \frac{1}{2}m\|e_v\|^2, \quad \dot{V}_1|_{c_1=k_x=0} \leq -k_v(1 - \alpha)\|e_v\|^2 + B\|e_v\|.$$

Hence whenever $\|e_v\| > B/(k_v(1 - \alpha))$, $\|e_v\|$ decreases, so

$$\|e_v(t)\| < \max\{\|e_v(0)\|, B/(k_v(1 - \alpha))\} \equiv e_{v,\max}. \quad (62)$$

Substituting into the cubic term, (61) becomes

$$\dot{V}_1 \leq -z_1^\top \widetilde{W}_1 z_1 + z_1^\top W_{12} z_2, \quad (63)$$

with $z_1 = [\|e_x\|, \|e_v\|]^\top$, $z_2 = [\|e_R\|, \|e_\Omega\|]^\top$, and $\widetilde{W}_1, W_{12}$ as in (49).

Step 6: Lyapunov function for the complete system

Let $V = V_1 + V_2$. Using (59) and $-\|e_x\|\|e_v\| \leq e_x \cdot e_v \leq \|e_x\|\|e_v\|$,

$$z_1^\top M_{11} z_1 \leq V_1 \leq z_1^\top M_{12} z_1,$$

with

$$M_{11} = \frac{1}{2} \begin{bmatrix} k_x & -c_1 \\ -c_1 & m \end{bmatrix}, \quad M_{12} = \frac{1}{2} \begin{bmatrix} k_x & c_1 \\ c_1 & m \end{bmatrix}.$$

By Step 1, $\Psi \leq \psi_1 < 1$, so the argument of (46) applies with ψ_1 in place of ψ_2 :

$$z_2^\top M_{21} z_2 \leq V_2 \leq z_2^\top M'_{22} z_2, \quad M'_{22} = \frac{1}{2} \begin{bmatrix} \frac{2k_R}{2-\psi_1} & c_2 \\ c_2 & \lambda_{\max}(J) \end{bmatrix}.$$

Combining,

$$z_1^\top M_{11} z_1 + z_2^\top M_{21} z_2 \leq V \leq z_1^\top M_{12} z_1 + z_2^\top M'_{22} z_2. \quad (64)$$

Combining (63) and (44),

$$\dot{V} \leq -z_1^\top \widetilde{W}_1 z_1 + z_1^\top W_{12} z_2 - z_2^\top W_2 z_2 = -z^\top \widetilde{W} z,$$

with $z = [z_1^\top, z_2^\top]^\top$ and

$$\widetilde{W} = \begin{bmatrix} \widetilde{W}_1 & -\frac{1}{2}W_{12} \\ -\frac{1}{2}W_{12}^\top & W_2 \end{bmatrix}.$$

Conditions (50) are exactly the conditions for $M_{11}, M_{12}, \widetilde{W}_1 \succ 0$. Conditions (51) are exactly the conditions for $M_{21}, M'_{22}, W_2 \succ 0$.

Finally, since $\widetilde{W}_1 \succ 0$, the Schur complement gives $\widetilde{W} \succ 0$ if $W_2 - \frac{1}{4}W_{12}^\top \widetilde{W}_1^{-1} W_{12} \succ 0$. For any $x \in \mathbb{R}^2$,

$$x^\top \left(\frac{1}{4} W_{12}^\top \widetilde{W}_1^{-1} W_{12} \right) x \leq \frac{\|W_{12}\|^2}{4\lambda_{\min}(\widetilde{W}_1)} \|x\|^2.$$

Hence

$$W_2 - \frac{1}{4} W_{12}^\top \widetilde{W}_1^{-1} W_{12} \succ 0 \quad (65)$$

if $\lambda_{\min}(W_2) > \|W_{12}\|^2 / (4\lambda_{\min}(\widetilde{W}_1))$, i.e. (52).

With all these matrices positive-definite, V is positive-definite and radially unbounded, and

$$\dot{V} \leq -\lambda_{\min}(\widetilde{W}) \|z\|^2 \leq -\frac{\lambda_{\min}(\widetilde{W})}{\lambda_{\max}(\overline{M})} V = -\beta V,$$

where $\overline{M} = \text{diag}(M_{12}, M'_{22})$. Grönwall's inequality gives $V(t) \leq V(0)e^{-\beta t}$, so the zero equilibrium of $(e_x, e_v, e_R, e_\Omega)$ is exponentially stable. $\square$

A.6 Almost Global Exponential Attractiveness

Proposition 2 requires $\Psi(R(0), R_d(0)) < 1$, i.e. an initial attitude error below 90° . We now relax this to attitude errors up to 180° .

Definition 1 (Exponential Attractiveness). *An equilibrium $z = 0$ is exponentially attractive if, for some $\delta > 0$, there exist $\alpha(\delta), \beta > 0$ such that $\|z(0)\| < \delta \implies \|z(t)\| \leq \alpha(\delta)e^{-\beta t}$ for all $t > 0$. This is weaker than exponential stability because the constant need not be proportional to $\|z(0)\|$.*

Proposition 3 (Almost Global Exponential Attractiveness). *Consider the control system designed according to Proposition 2. Suppose*

$$1 \leq \Psi(R(0), R_d(0)) < 2, \quad (66)$$

$$\|e_\Omega(0)\|^2 < \frac{2}{\lambda_{\max}(J)} k_R (2 - \Psi(R(0), R_d(0))). \quad (67)$$

Then the zero equilibrium of the complete tracking error is exponentially attractive.

Proof. Conditions (66), (67) satisfy the hypotheses of Proposition 1, so $z_2 = [\|e_R\|, \|e_\Omega\|]^\top$ is exponentially stable, and both $\Psi(R(t), R_d(t)) \rightarrow 0$ and $\|e_\Omega(t)\| \rightarrow 0$ exponentially.

Since $\Psi \rightarrow 0$ exponentially, there exists t_1 with $\Psi(R(t_1), R_d(t_1)) < 1/2$. Hence $\frac{2}{\lambda_{\max}(J)} k_R (1 - \Psi(R(t_1), R_d(t_1))) > k_R / \lambda_{\max}(J)$. Since $\|e_\Omega\| \rightarrow 0$ exponentially, there exists $t_2 \geq t_1$ with $\|e_\Omega(t_2)\|^2 < k_R / \lambda_{\max}(J)$. At $t^* = t_2$, the rotational state is in the region of attraction of Proposition 2.

It remains to bound $z_1 = [\|e_x\|, \|e_v\|]^\top$ on $[0, t^*]$. Define

$$V_3 = \frac{1}{2} \|e_x\|^2 + \frac{1}{2} m \|e_v\|^2. \quad (68)$$

Using $\dot{e}_x = e_v$, Cauchy–Schwarz, (33), and $|f| \leq k_x \|e_x\| + k_v \|e_v\| + B$,

$$\begin{aligned} \dot{V}_3 &\leq \|e_x\| \|e_v\| + B \|e_v\| + (k_x \|e_x\| + k_v \|e_v\| + B) \|e_v\| \\ &= k_v \|e_v\|^2 + (2B + (k_x + 1) \|e_x\|) \|e_v\|. \end{aligned}$$

From (68), $\|e_x\| \leq \sqrt{2V_3}$ and $\|e_v\| \leq \sqrt{2V_3/m}$, so

$$\dot{V}_3 \leq d_1 V_3 + d_2 \sqrt{V_3}, \quad d_1 = \frac{2k_v}{m} + \frac{2(k_x+1)}{\sqrt{m}}, \quad d_2 = 2B\sqrt{2/m}.$$

If $V_3 < 1$, it is bounded. If $V_3 \geq 1$ on $[t_a, t_b]$, $\sqrt{V_3} \leq V_3$, so $\dot{V}_3 \leq (d_1 + d_2)V_3$, and Grönwall gives $V_3(t) \leq V_3(t_a)e^{(d_1+d_2)(t-t_a)}$. Hence z_1 is bounded on $[0, t^*]$.

At t^* , Proposition 2 applies, so the error decays exponentially for $t \geq t^*$: $\|z(t)\| \leq C(\delta)e^{-\beta(t-t^*)}$. Let $\alpha(\delta) = \max\{M(\delta)e^{\beta t^*}, C(\delta)e^{\beta t^*}\}$ where $M(\delta)$ is a bound on $\|z\|$ over $[0, t^*]$. Then

$$\|z(t)\| \leq \alpha(\delta)e^{-\beta t} \quad \forall t > 0,$$

which is exponential attractiveness. □

The result is called *almost global* because (66), together with the region covered by Proposition 2, covers all initial attitudes except the measure-zero set of exact 180° relative rotations.

B Feed-Forward Derivation of Ω_d and $\dot{\Omega}_d$

This appendix gives the analytical derivation of the desired body-frame angular velocity Ω_d and acceleration $\dot{\Omega}_d$ used as feed-forward terms in the moment law (11). The derivation is summarised in Section 5.3 of the main report.

The moment law (11) requires Ω_d and $\dot{\Omega}_d$. Rather than numerically differentiating the desired attitude—which amplifies noise—these are derived analytically from the flat outputs $x_d(t)$ and $\psi_d(t)$ and their derivatives up to snap and yaw-acceleration. The derivation below is a self-contained reworking of classical results from differential flatness-based quadrotor trajectory generation [5], tailored to the geometric-controller construction.

B.1 Definitions

From the controller of Section 4 in the Isaac Sim / Z-up convention (Section 5.1),

$$A = -k_x e_x - k_v e_v + m g e_3 + m a_d, \quad (69)$$

$$b_{3d} = \frac{A}{\|A\|}, \quad (70)$$

$$c = [\cos \psi_d, \sin \psi_d, 0]^\top, \quad (71)$$

$$r = b_{3d} \times c, \quad b_{2d} = \frac{r}{\|r\|}, \quad b_{1d} = b_{2d} \times b_{3d}, \quad (72)$$

$$R_d = [b_{1d} \ b_{2d} \ b_{3d}]. \quad (73)$$

B.2 Derivatives of A

With $\dot{e}_x = e_v$ and $\dot{e}_v = a - a_d$,

$$\dot{A} = -k_x e_v - k_v (a - a_d) + m j_d, \quad (74)$$

$$\ddot{A} = -k_x (a - a_d) - k_v (j - j_d) + m s_d, \quad (75)$$

where $j = \dot{a}$, $j_d = \dot{a}_d$, and $s_d = \ddot{a}_d$. These require the controller to have access to the body's jerk j (available from the Pegasus modification described below) as well as the reference jerk j_d and snap s_d (produced by the trajectory publisher).

B.3 Normalisation Lemma

We derive general formulas for the first and second derivatives of $u = p/\|p\|$ for any non-zero $p \in \mathbb{R}^3$, which will be applied to $A \mapsto b_{3d}$ and $r \mapsto b_{2d}$.

Let $s = \|p\| = (p^\top p)^{1/2}$, so $u = s^{-1}p$. Then

$$\dot{u} = \frac{d}{dt}(s^{-1})p + s^{-1}\dot{p} = -s^{-2}\dot{s}p + s^{-1}\dot{p}.$$

Differentiating $s = (p^\top p)^{1/2}$,

$$\dot{s} = \frac{1}{2}(p^\top p)^{-1/2} \cdot 2p^\top \dot{p} = \frac{p^\top \dot{p}}{\|p\|}.$$

Substituting and using $p = su$, $u^\top \dot{p} = p^\top \dot{p}/s$,

$$\dot{u} = \frac{\dot{p}}{s} - u \frac{u^\top \dot{p}}{s} = \frac{1}{s}(I - uu^\top)\dot{p}.$$

Hence

$$\dot{u} = \frac{1}{\|p\|}(I - uu^\top)\dot{p} \quad (76)$$

Define $P = I - uu^\top$, so $\dot{u} = (1/s)P\dot{p}$. Differentiating,

$$\ddot{u} = \frac{d}{dt}\left(\frac{1}{s}\right)P\dot{p} + \frac{1}{s}\dot{P}\dot{p} + \frac{1}{s}P\ddot{p} = -\frac{\dot{s}}{s^2}P\dot{p} + \frac{1}{s}(\dot{P}\dot{p} + P\ddot{p}).$$

Using $P\dot{p} = s\dot{u}$,

$$\ddot{u} = \frac{1}{\|p\|}(\dot{P}\dot{p} + P\ddot{p}) - \frac{\|\dot{p}\|}{\|p\|}\dot{u}, \quad \dot{P} = -\dot{u}u^\top - u\dot{u}^\top, \quad \|\dot{p}\| = \frac{p^\top \dot{p}}{\|p\|} \quad (77)$$

B.4 Derivatives of b_{3d}

Applying (76), (77) with $p = A$, $u = b_{3d}$, and $P_3 = I - b_{3d}b_{3d}^\top$:

$$\dot{b}_{3d} = \frac{1}{\|A\|}P_3\dot{A}, \quad (78)$$

$$\ddot{b}_{3d} = \frac{1}{\|A\|}(\dot{P}_3\dot{A} + P_3\ddot{A}) - \frac{A^\top \dot{A}}{\|A\|^2}\dot{b}_{3d}, \quad (79)$$

with $\dot{P}_3 = -\dot{b}_{3d}b_{3d}^\top - b_{3d}\dot{b}_{3d}^\top$.

B.5 Derivatives of the Yaw Heading c

From $c = [\cos \psi_d, \sin \psi_d, 0]^\top$, direct differentiation gives

$$\dot{c} = [-\sin \psi_d \dot{\psi}_d, \cos \psi_d \dot{\psi}_d, 0]^\top, \quad (80)$$

$$\ddot{c} = \begin{bmatrix} -\cos \psi_d \dot{\psi}_d^2 - \sin \psi_d \ddot{\psi}_d \\ -\sin \psi_d \dot{\psi}_d^2 + \cos \psi_d \ddot{\psi}_d \\ 0 \end{bmatrix}. \quad (81)$$

B.6 Derivatives of r and b_{2d}

From $r = b_{3d} \times c$, the product rule gives

$$\dot{r} = \dot{b}_{3d} \times c + b_{3d} \times \dot{c}, \quad (82)$$

$$\ddot{r} = \ddot{b}_{3d} \times c + 2\dot{b}_{3d} \times \dot{c} + b_{3d} \times \ddot{c}. \quad (83)$$

Applying (76), (77) with $p = r$, $u = b_{2d}$, and $P_2 = I - b_{2d}b_{2d}^\top$:

$$\dot{b}_{2d} = \frac{1}{\|r\|}P_2\dot{r}, \quad (84)$$

$$\ddot{b}_{2d} = \frac{1}{\|r\|}(\dot{P}_2\dot{r} + P_2\ddot{r}) - \frac{r^\top \dot{r}}{\|r\|^2}\dot{b}_{2d}, \quad (85)$$

with $\dot{P}_2 = -\dot{b}_{2d}b_{2d}^\top - b_{2d}\dot{b}_{2d}^\top$.

B.7 Derivatives of b_{1d}

From $b_{1d} = b_{2d} \times b_{3d}$,

$$\dot{b}_{1d} = \dot{b}_{2d} \times b_{3d} + b_{2d} \times \dot{b}_{3d}, \quad (86)$$

$$\ddot{b}_{1d} = \ddot{b}_{2d} \times b_{3d} + 2\dot{b}_{2d} \times \dot{b}_{3d} + b_{2d} \times \ddot{b}_{3d}. \quad (87)$$

B.8 Derivation of Ω_d

By definition of the body-frame angular velocity,

$$\hat{\Omega}_d = R_d^\top \dot{R}_d. \quad (88)$$

With $R_d = [b_{1d} \ b_{2d} \ b_{3d}]$ and $\dot{R}_d = [\dot{b}_{1d} \ \dot{b}_{2d} \ \dot{b}_{3d}]$,

$$R_d^\top \dot{R}_d = \begin{bmatrix} b_{1d}^\top \dot{b}_{1d} & b_{1d}^\top \dot{b}_{2d} & b_{1d}^\top \dot{b}_{3d} \\ b_{2d}^\top \dot{b}_{1d} & b_{2d}^\top \dot{b}_{2d} & b_{2d}^\top \dot{b}_{3d} \\ b_{3d}^\top \dot{b}_{1d} & b_{3d}^\top \dot{b}_{2d} & b_{3d}^\top \dot{b}_{3d} \end{bmatrix}.$$

Orthonormality of R_d 's columns gives $b_{id}^\top b_{id} = 1$ and $b_{id}^\top b_{jd} = 0$ for $i \neq j$. Differentiating, $b_{id}^\top \dot{b}_{id} = 0$ (diagonal entries vanish) and $b_{jd}^\top \dot{b}_{id} = -b_{id}^\top \dot{b}_{jd}$ (off-diagonal entries are antisymmetric). Hence

$$R_d^\top \dot{R}_d = \begin{bmatrix} 0 & b_{1d}^\top \dot{b}_{2d} & b_{1d}^\top \dot{b}_{3d} \\ -b_{1d}^\top \dot{b}_{2d} & 0 & b_{2d}^\top \dot{b}_{3d} \\ -b_{1d}^\top \dot{b}_{3d} & -b_{2d}^\top \dot{b}_{3d} & 0 \end{bmatrix}.$$

The hat of $\Omega_d = [\Omega_{d,1}, \Omega_{d,2}, \Omega_{d,3}]^\top$ is

$$\hat{\Omega}_d = \begin{bmatrix} 0 & -\Omega_{d,3} & \Omega_{d,2} \\ \Omega_{d,3} & 0 & -\Omega_{d,1} \\ -\Omega_{d,2} & \Omega_{d,1} & 0 \end{bmatrix}.$$

Matching entries:

$$\Omega_d = \begin{bmatrix} b_{3d}^\top \dot{b}_{2d} \\ b_{1d}^\top \dot{b}_{3d} \\ b_{2d}^\top \dot{b}_{1d} \end{bmatrix} \quad (89)$$

Note the use of the antisymmetry $b_{3d}^\top \dot{b}_{2d} = -b_{2d}^\top \dot{b}_{3d}$ (which follows from $b_{2d}^\top b_{3d} = 0$) to convert the (2,3) entry into the form shown in the first component, and similarly for the other components.

B.9 Derivation of $\dot{\Omega}_d$

Differentiating each component of (89) component-wise,

$$\dot{\Omega}_d = \begin{bmatrix} \dot{b}_{3d}^\top \dot{b}_{2d} + b_{3d}^\top \ddot{b}_{2d} \\ \dot{b}_{1d}^\top \dot{b}_{3d} + b_{1d}^\top \ddot{b}_{3d} \\ \dot{b}_{2d}^\top \dot{b}_{1d} + b_{2d}^\top \ddot{b}_{1d} \end{bmatrix} \quad (90)$$

B.10 Summary of the Computation Chain

The implementation evaluates, in order: (i) $A, \dot{A}, \ddot{A}$; (ii) $b_{3d}, \dot{b}_{3d}, \ddot{b}_{3d}$ via (78), (79); (iii) $c, \dot{c}, \ddot{c}$; (iv) $r, \dot{r}, \ddot{r}$; (v) $b_{2d}, \dot{b}_{2d}, \ddot{b}_{2d}$ via (84), (85); (vi) $b_{1d}, \dot{b}_{1d}, \ddot{b}_{1d}$ via (86), (87); (vii) $\Omega_d, \dot{\Omega}_d$ via (89), (90).

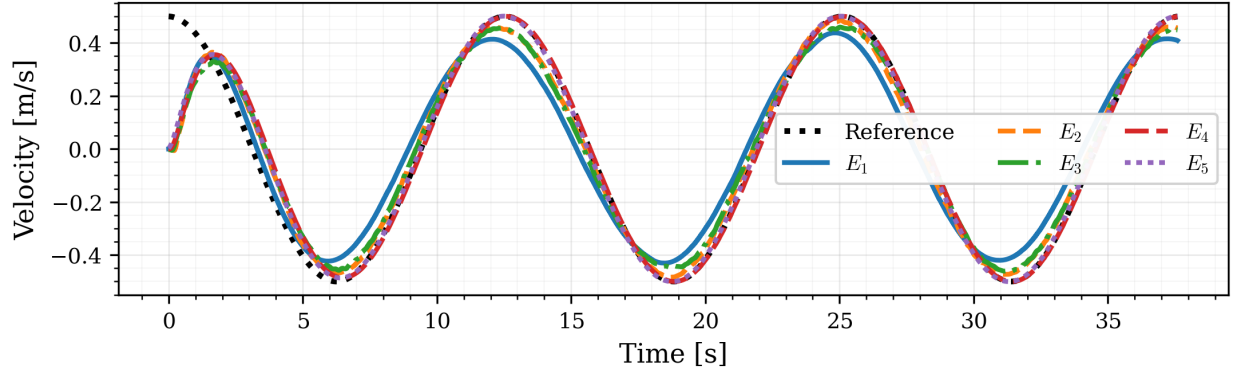
B.11 Assumptions and Degeneracy Handling

The derivation assumes $\|A\| \neq 0$ and $\|b_{3d} \times c\| \neq 0$. The first fails only in the unphysical case where the controller commands zero net thrust (including gravity compensation); the second fails when the commanded thrust axis b_{3d} is parallel or anti-parallel to the yaw heading c . Since c lies in the horizontal plane ($c_z = 0$), this occurs exactly when b_{3d} is also horizontal, which corresponds to commanding the body- z axis to be horizontal, i.e. a 90° pitch or roll attitude. Both conditions are guarded in the implementation by a small numerical threshold, with safe fall-backs: b_{3d} reverts to e_3 with zero derivatives, and b_{2d} reverts to the yaw-perpendicular direction with zero derivatives.

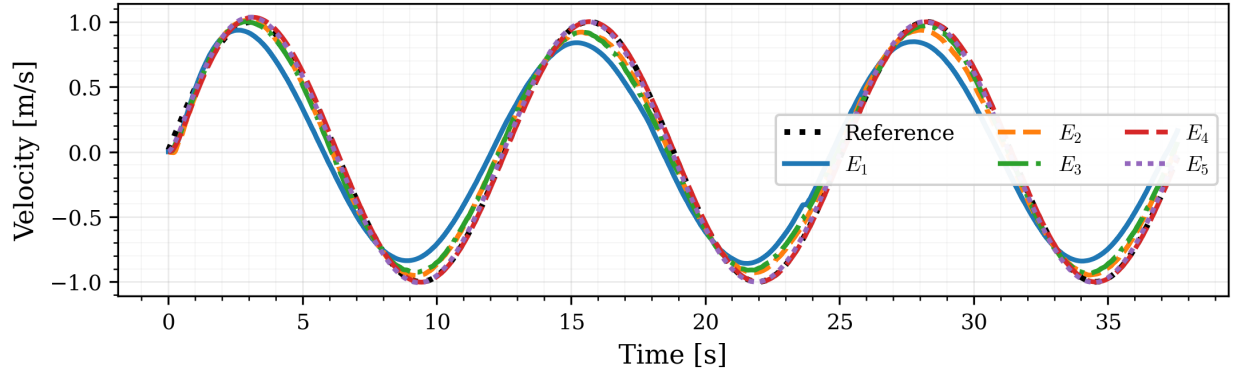
C Supplementary Plots

This appendix collects the supplementary plots for the five-configuration benchmark that were not shown in Section 6.4. Each plot retains the same overlay convention as the main-report figures: all five experiments E1–E5 and (where applicable) the reference signal share the same axes.

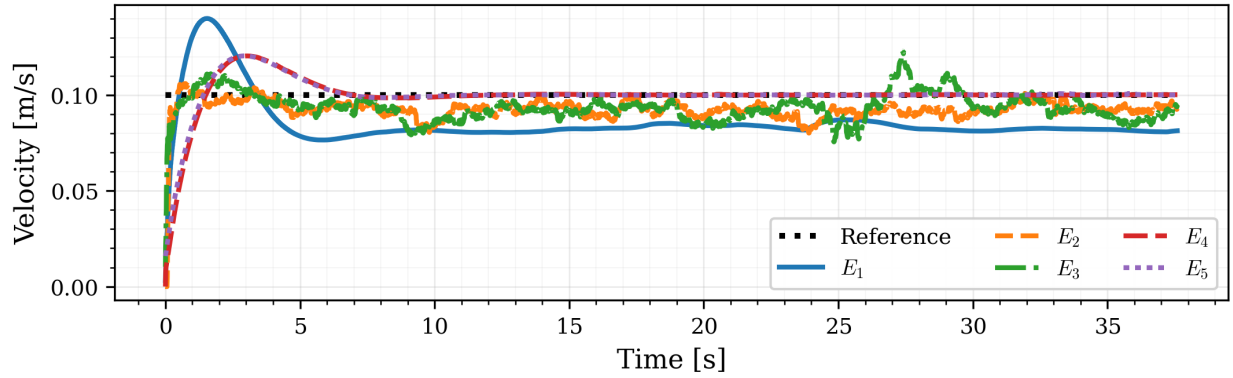
C.1 Per-Axis Velocity Tracking



(a) $v_x(t)$.



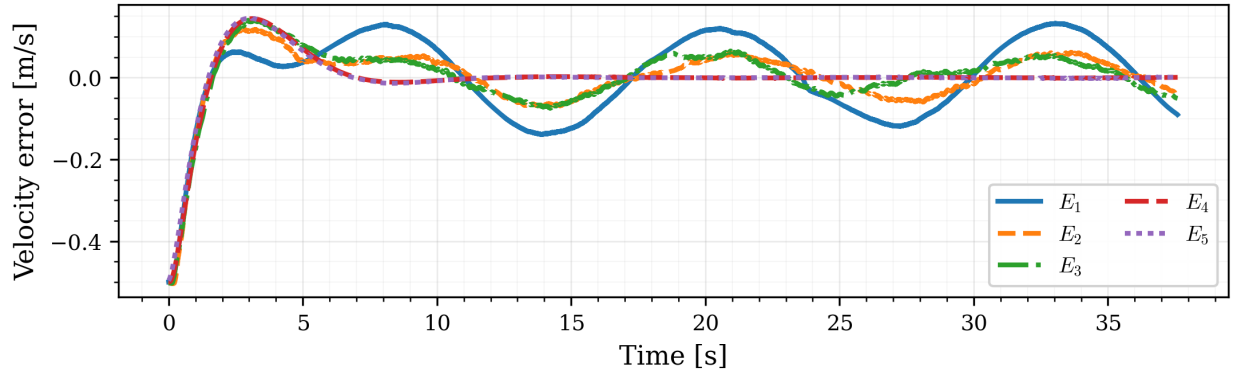
(b) $v_y(t)$.



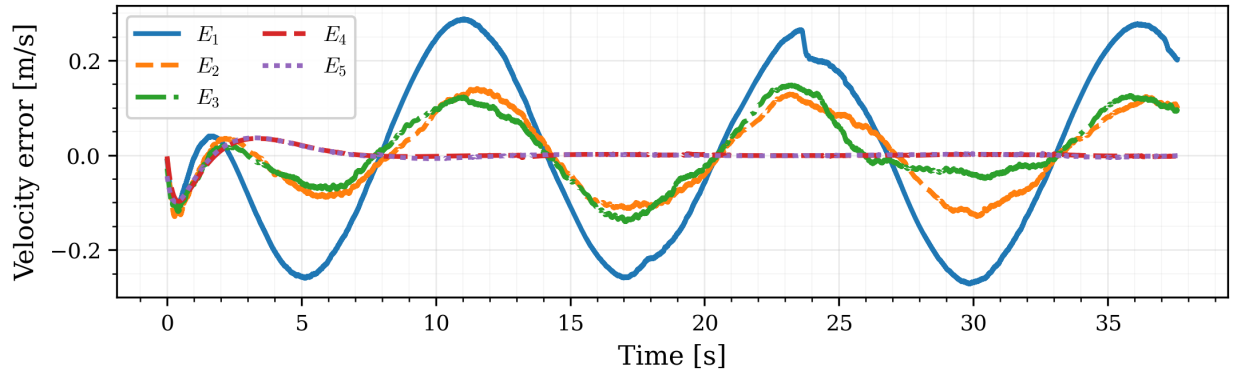
(c) $v_z(t)$.

Figure 31: Per-axis velocity tracking.

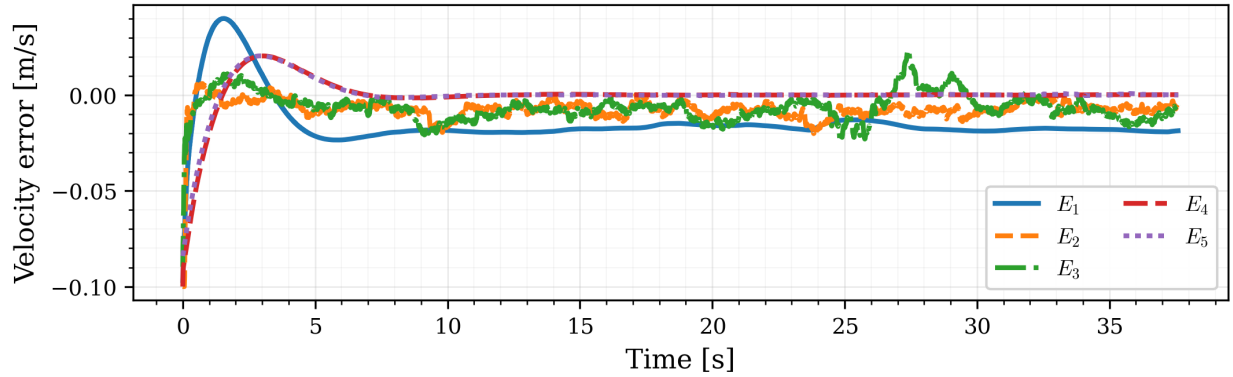
C.2 Per-Axis Velocity Error



(a) $e_{v,1}(t)$.



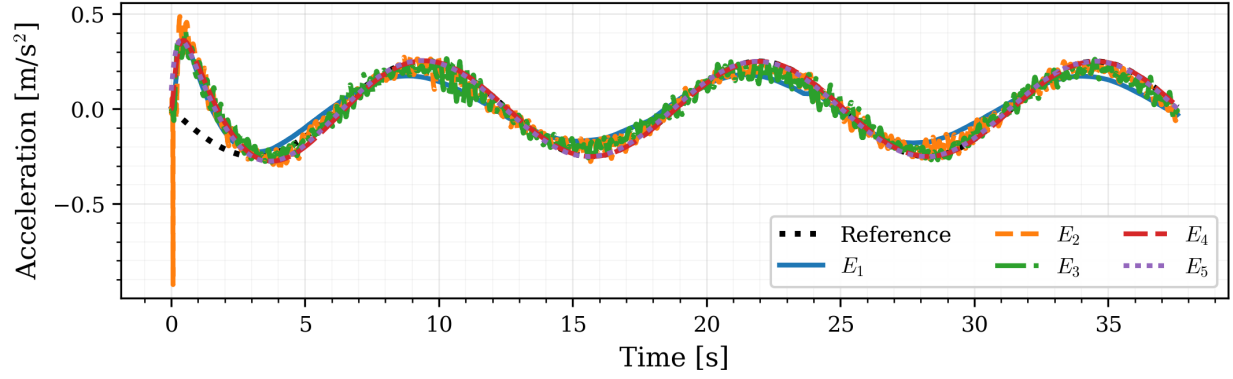
(b) $e_{v,2}(t)$.



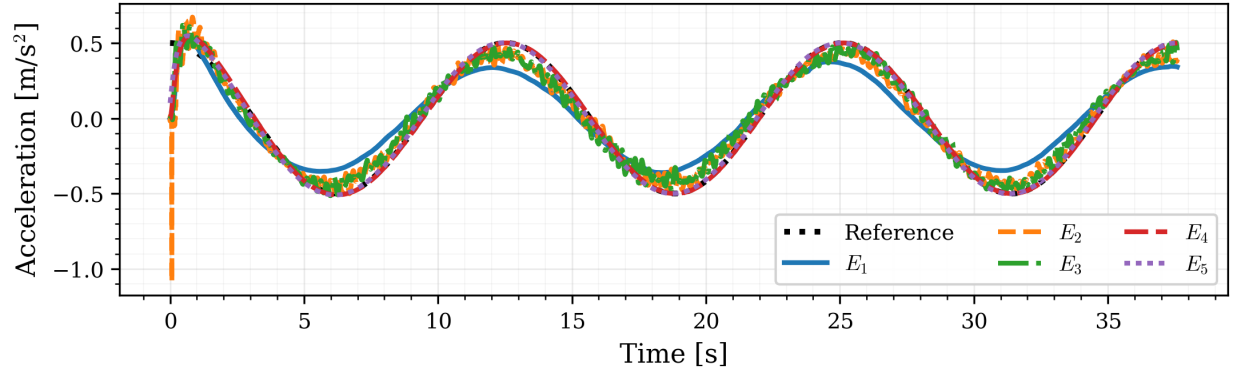
(c) $e_{v,3}(t)$.

Figure 32: Velocity-error components per inertial axis.

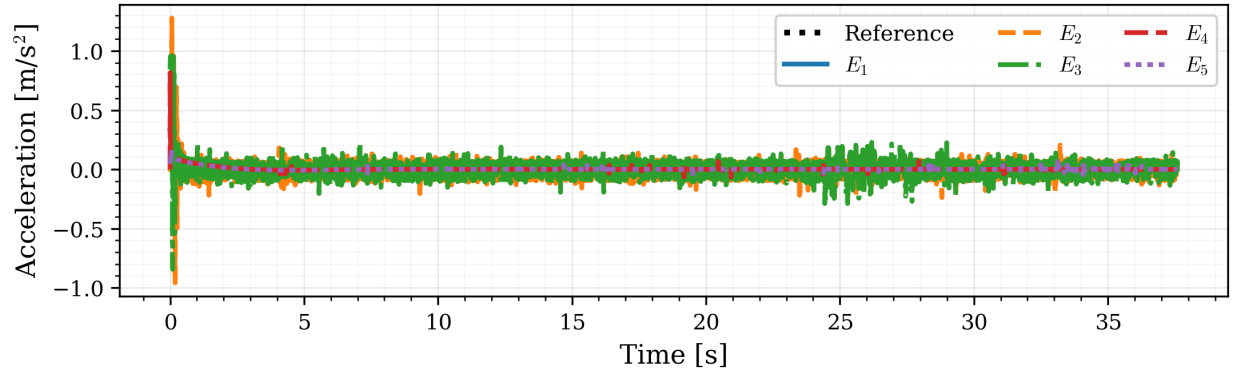
C.3 Per-Axis Acceleration Tracking



(a) $a_x(t)$.



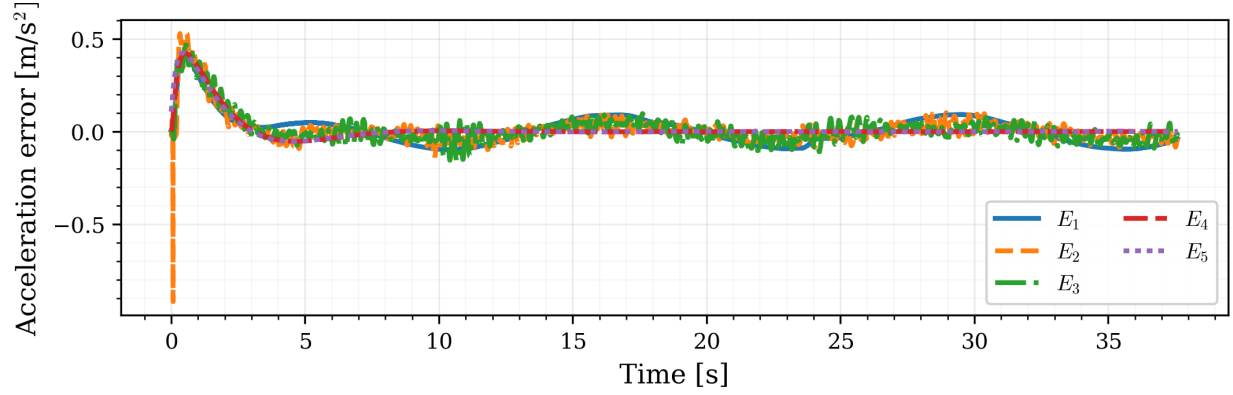
(b) $a_y(t)$.



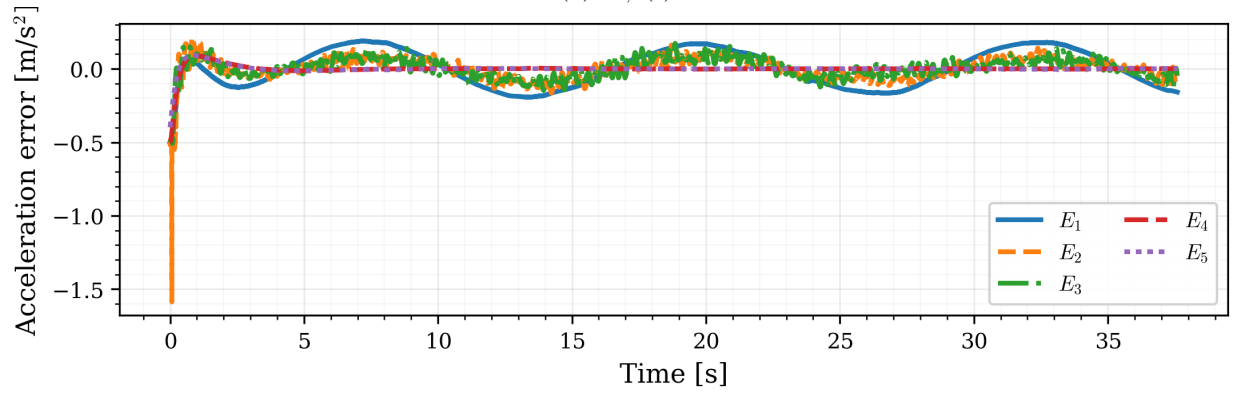
(c) $a_z(t)$.

Figure 33: Per-axis acceleration tracking.

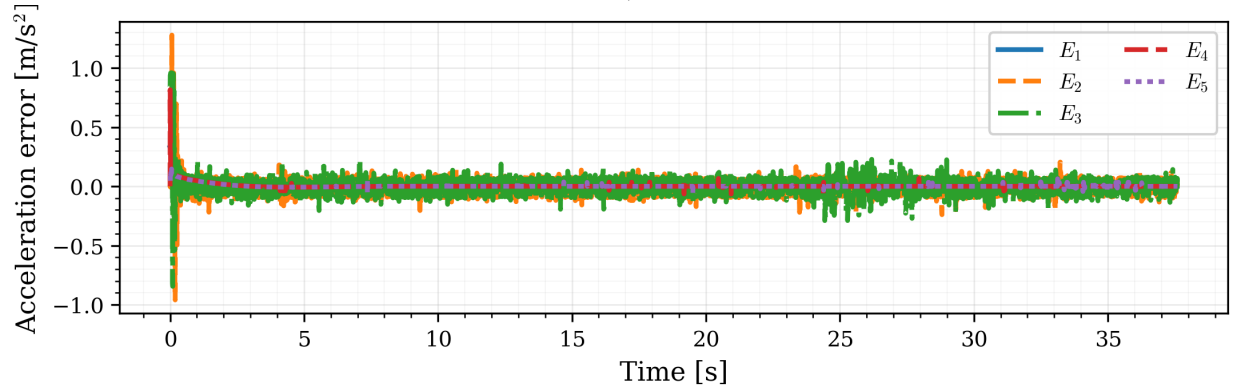
C.4 Per-Axis Acceleration Error



(a) $\dot{e}_{v,1}(t)$.



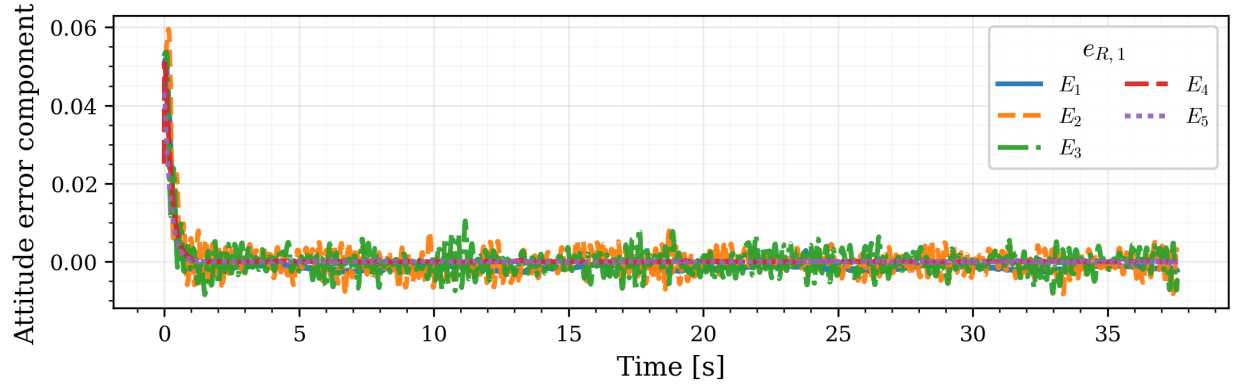
(b) $\dot{e}_{v,2}(t)$.



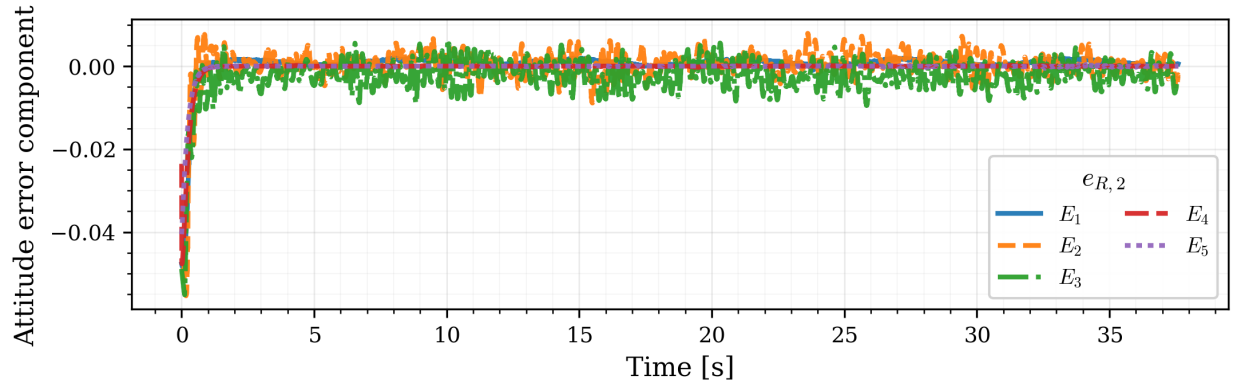
(c) $\dot{e}_{v,3}(t)$.

Figure 34: Acceleration-error components per inertial axis.

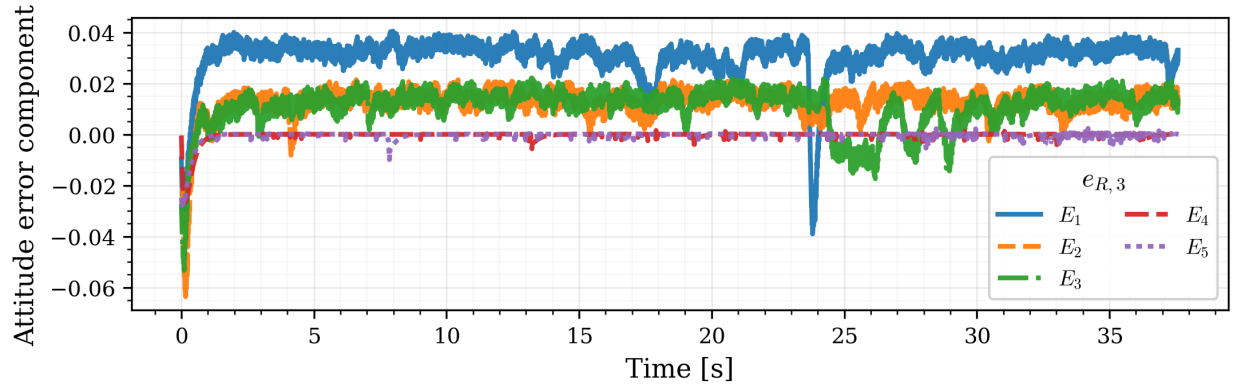
C.5 Per-Axis Attitude-Error Components



(a) $e_{R,1}(t)$.



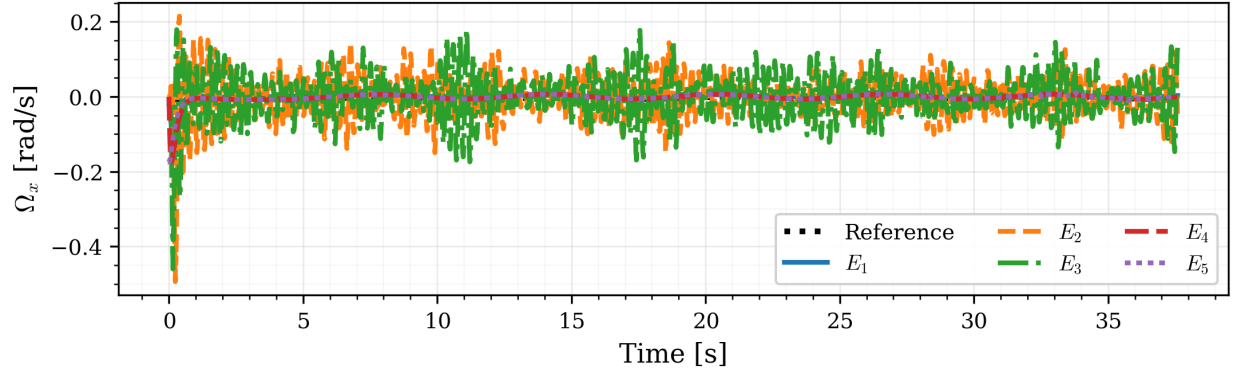
(b) $e_{R,2}(t)$.



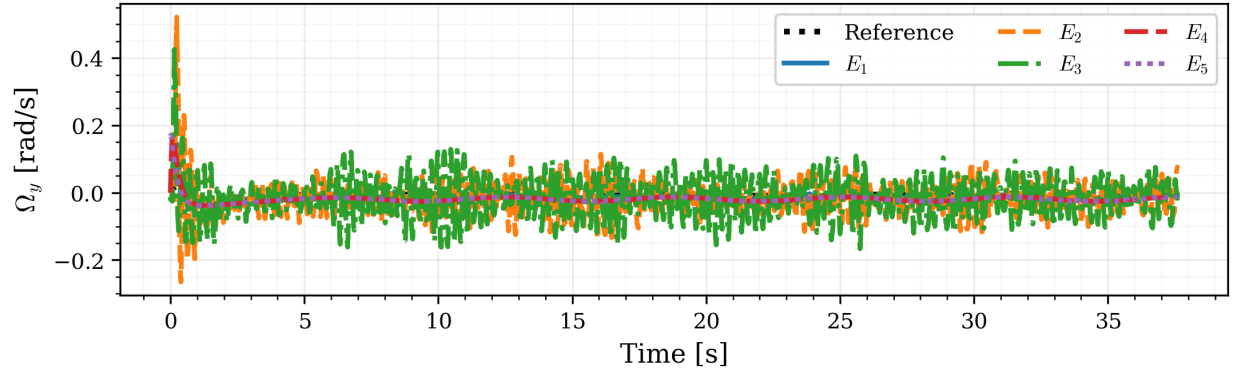
(c) $e_{R,3}(t)$.

Figure 35: Per-body-axis components of the attitude error vector e_R .

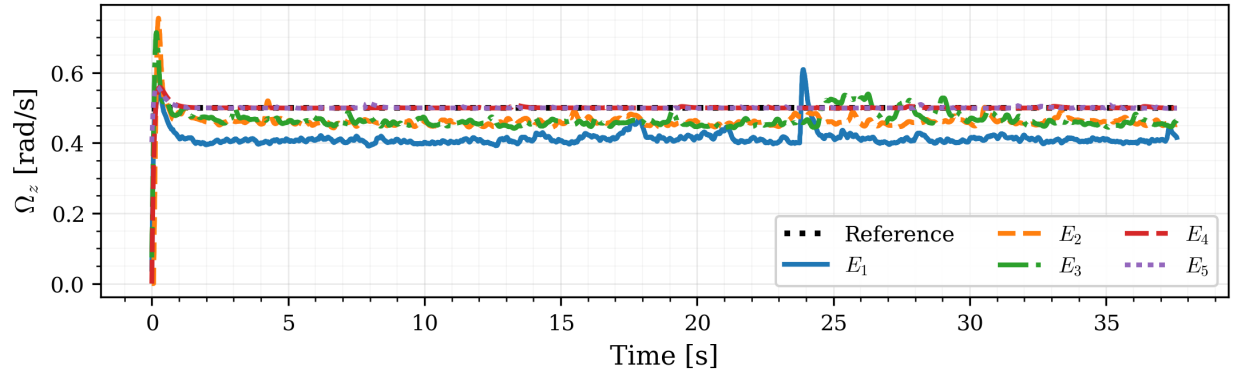
C.6 Per-Axis Angular-Rate Tracking



(a) $\Omega_x(t)$.



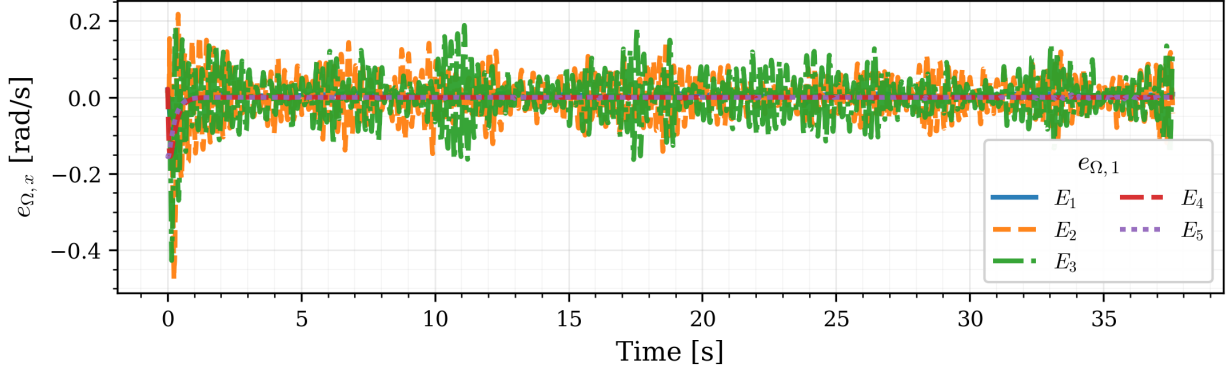
(b) $\Omega_y(t)$.



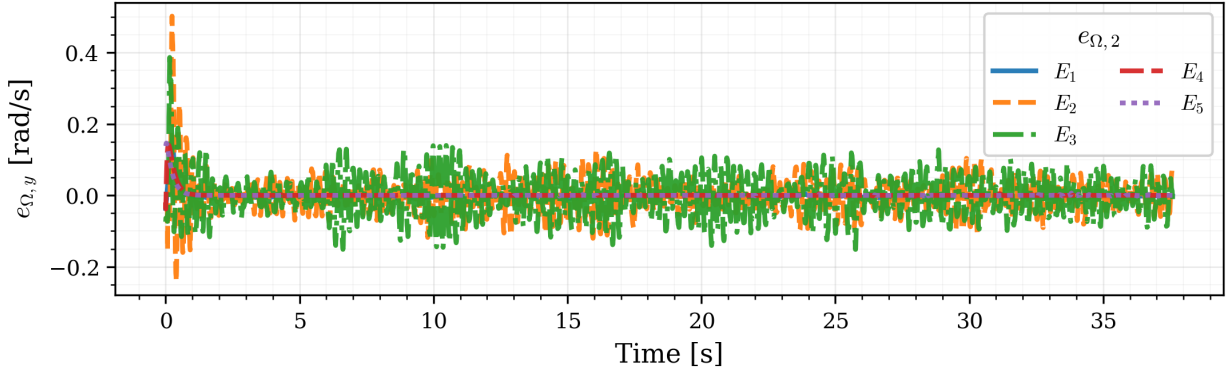
(c) $\Omega_z(t)$.

Figure 36: Per-body-axis angular-rate tracking.

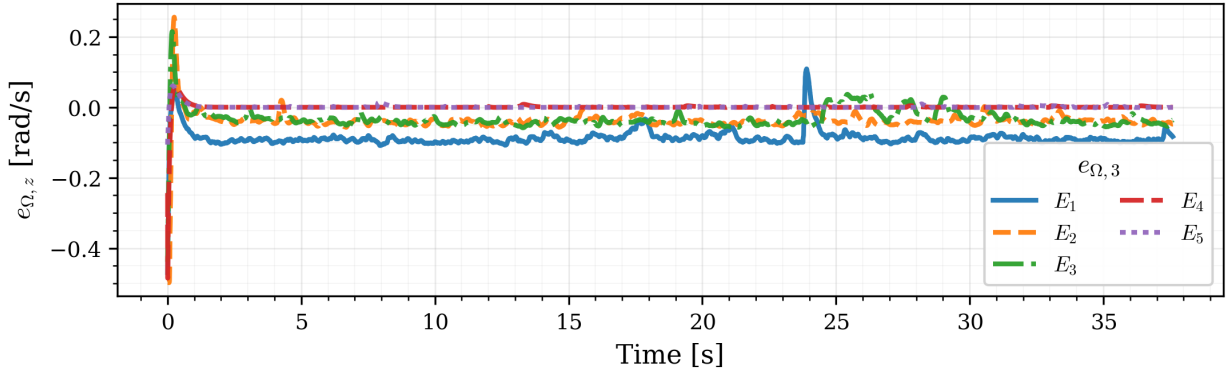
C.7 Per-Axis Angular-Velocity-Error Components



(a) $e_{\Omega,1}(t)$.



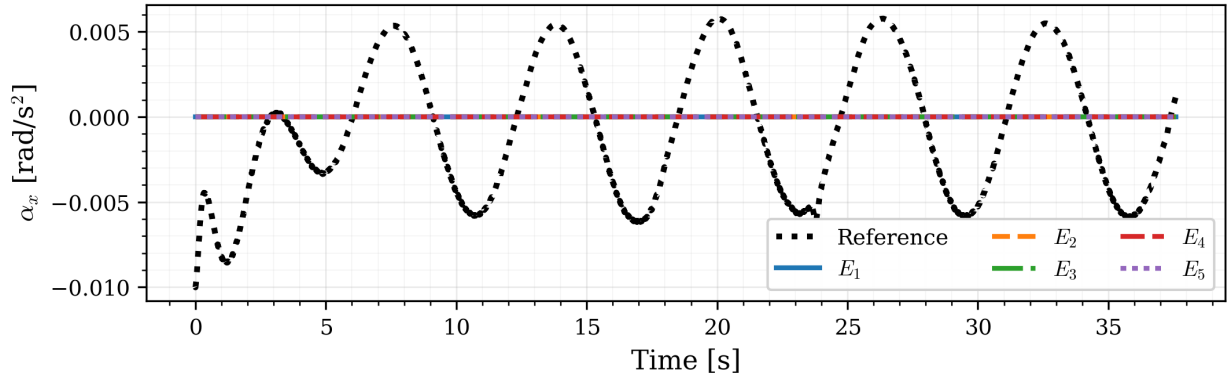
(b) $e_{\Omega,2}(t)$.



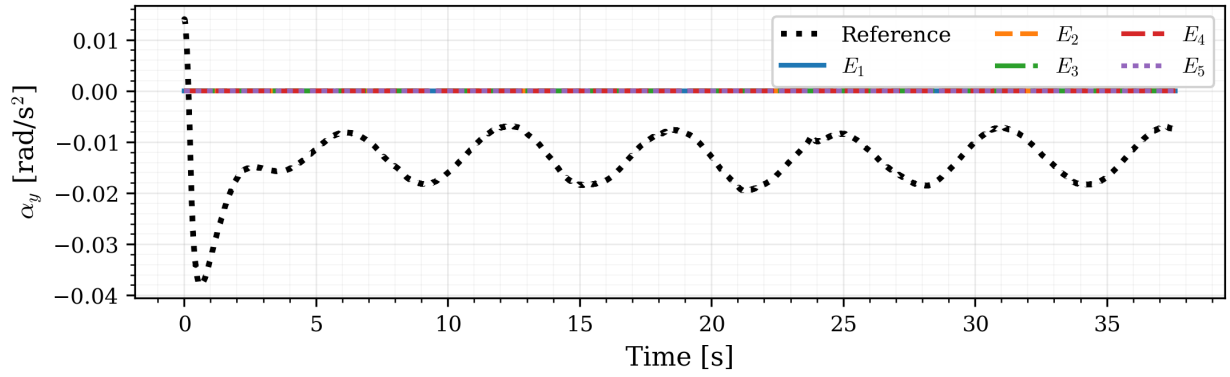
(c) $e_{\Omega,3}(t)$.

Figure 37: Per-body-axis components of the angular-velocity error vector e_{Ω} .

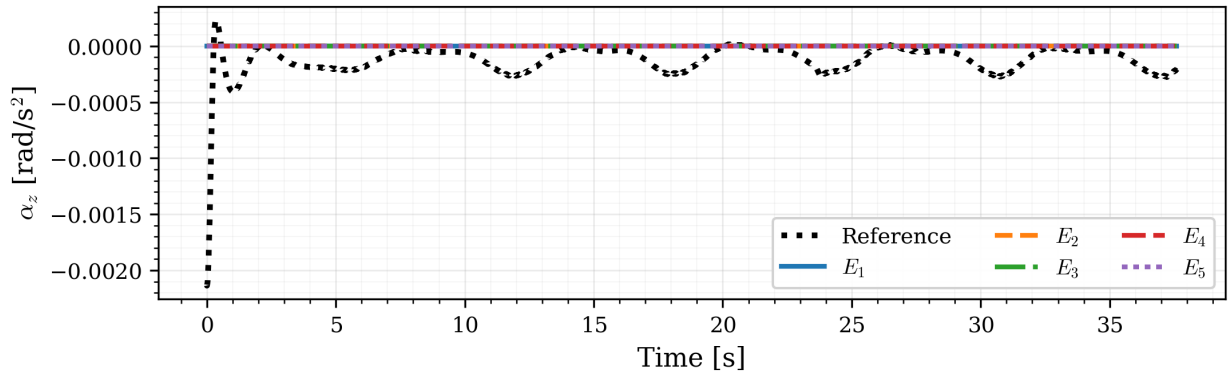
C.8 Per-Axis Angular-Acceleration Tracking



(a) $\dot{\Omega}_x(t)$.



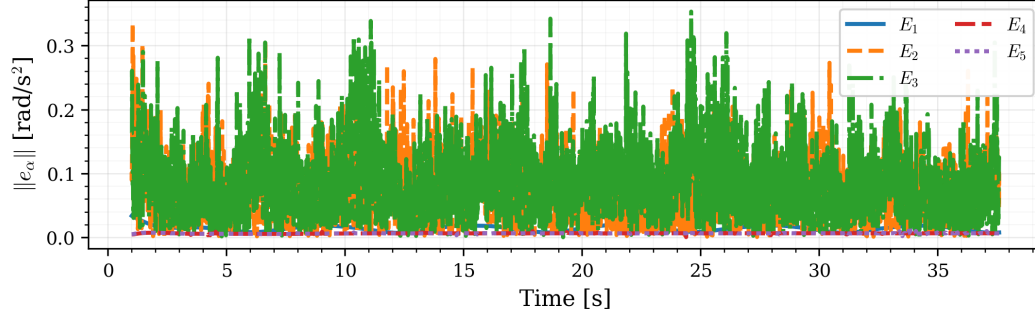
(b) $\dot{\Omega}_y(t)$.



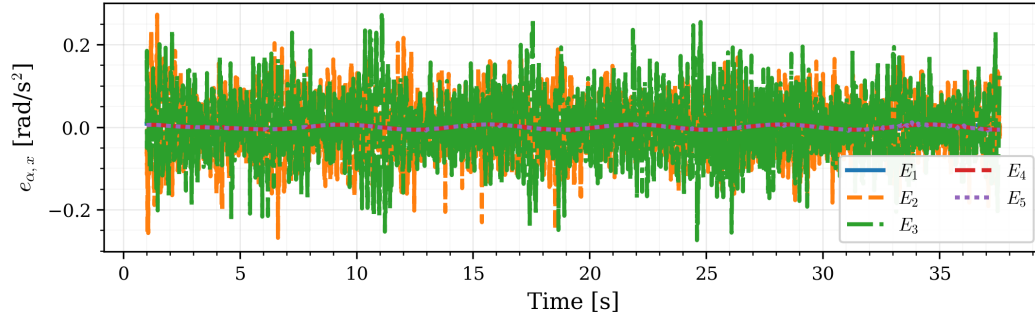
(c) $\dot{\Omega}_z(t)$.

Figure 38: Per-body-axis angular-acceleration tracking.

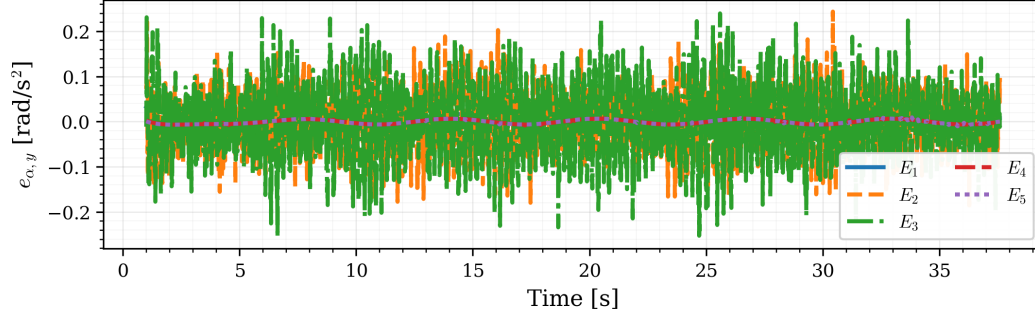
C.9 Angular-Acceleration Errors



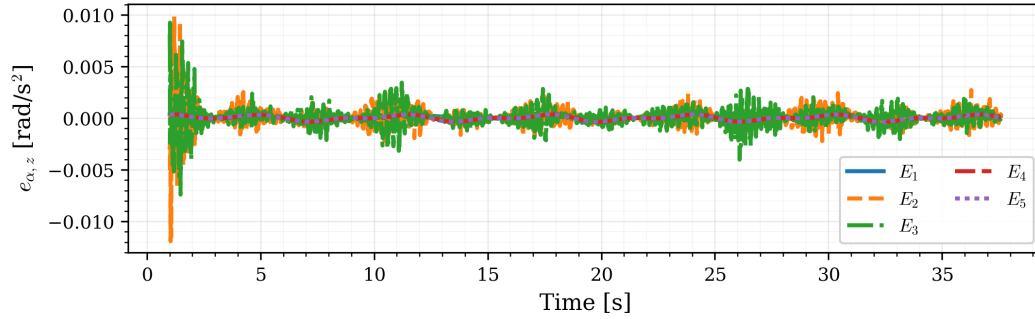
(a) Angular-acceleration-error norm $\|\dot{\Omega} - \dot{\Omega}_d\|(t)$.



(b) x -component.



(c) y -component.



(d) z -component.

Figure 39: Angular-acceleration tracking errors. The norm is dominated by short transients at the start of the trajectory; the steady-state behaviour is most informative in the per-axis breakdown.